



产品故事 · 竞品研究 · 技术栈 · 数据库 · REST API · 四人执行计划

两天规划 + 两天编码

文档日期：2026-07-30

目录

HoldHive 团队项目执行指南	8
1. 项目目标	8
1.1.1.0.0 已交付范围	8
2. 竞品介绍和链接	8
竞品结论	9
HoldHive 的差异化机会	9
3. 产品设计	9
3.1 背景故事	9
3.2 产品愿景与价值主张	9
3.3 目标用户与非目标用户	10
3.3.1 MVP 资产类型	10
3.3.2 基金穿透与解耦	10
3.3.3 大模型分析	11
3.4 设计原则	11
3.5 信息架构	11
3.6 核心用户流程	11
流程 A：建立第一个组合	11
流程 B：理解组合	11
流程 C：删除错误持仓	12
3.7 页面和状态设计	12
3.8 界面参考、现代交互与动画	12
页面布局建议	12
图表设计	13
用户友好提示	13
动态与过渡	14
3.9 功能需求与验收标准	14
3.10 计算口径	14
3.11 API 边界建议	15
3.12 非功能与风险设计	15
3.13 成功指标与范围边界	15
4. MVP 范围	16
必须完成	16
可选增强	16
5. 最小数据模型	16
6. 四人分工	16
6.1 后端两人细分	17
6.2 每人实现目录	17
目录责任总览	18

成员 A：后端 API 与数据库	18
成员 B：后端业务计算与质量	19
成员 C：前端页面与交互	20
成员 D：QA、交付与自动化	20
共享目录规则	21
低耦合与依赖规则	21
7. 四天计划	22
Day 1：规划 - 需求与仓库基线	22
Day 2：规划 - 任务、测试和演示设计	23
Day 3：编码 - MVP 主流程	23
Day 4：编码 - 图表、稳定性与交付	23
8. Git 与 PR 规范	24
分支策略	24
Git Commit 规范	24
提交与 PR 前检查	25
9. 工程质量要求	25
10. 每日协作节奏	25
11. 最终交付检查清单	25
HoldHive 成员目录分工速查	26
成员 A：后端 API + 数据库	26
成员 B：后端业务计算 + 质量	26
成员 C：前端页面 + 联调	26
成员 D：QA + CI + 交付	27
共享规则	27
HoldHive 蓝湖协作设计稿	28
文件说明	28
主题与 Logo	28
页面清单	28
Gateway 动效标注	28
蓝湖上传建议	28
增量修改方式	28
蓝湖画板 01：Gateway 夜间主题 - 单个六边形门户，六个顶点入口和 hover 示意	30
蓝湖画板 02：Gateway 日间主题 - 同结构主题适配	31
蓝湖画板 03：Dashboard 夜间主题 - KPI、配置图、趋势和持仓摘要	32
蓝湖画板 04：Dashboard 日间主题 - 首屏组合总览	33
蓝湖画板 05：Holdings 夜间主题 - 持仓表和筛选	34
蓝湖画板 06：Holdings 日间主题 - 记录维护	35
蓝湖画板 07：Performance 夜间主题 - 价值趋势和贡献提示	36

蓝湖画板 08：Performance 日间主题 - 表现视图	37
蓝湖画板 09：Analysis 夜间主题 - 配置和集中度检查	38
蓝湖画板 10：Analysis 日间主题 - X-Ray 分析	39
蓝湖画板 11：Add Holding 夜间主题 - 表单和实时预览	40
蓝湖画板 12：Add Holding 日间主题 - 新增持仓流程	41
蓝湖画板 13：Settings 夜间主题 - 主题、数据模式和动效偏好	42
蓝湖画板 14：Settings 日间主题 - 主题切换配置	43
蓝湖画板 15：States 夜间主题 - 空态、部分估值、失败和确认	44
蓝湖画板 16：States 日间主题 - 用户友好状态	45
HoldHive 技术栈定案	46
1. 决策摘要	46
2. 选型前置条件	46
3. 前端技术栈	47
3.1 核心工具	47
3.2 前端额外库建议	47
3.3 前端不采用的技术	48
3.4 建议目录	48
3.5 前端数据策略	48
3.6 前端依赖边界	48
4. 后端技术栈	49
4.1 核心工具	49
4.2 建议包结构	49
4.3 后端职责边界	49
4.4 后端依赖方向	49
4.5 配置策略	50
5. 数据库与迁移	50
6. 测试技术栈与分层	51
6.1 后端测试	51
6.2 前端测试	51
6.3 E2E 测试	51
6.4 测试数据	51
7. API 与前后端契约	52
8. 本机协作、变更同步与联调	52
8.1 基本原则	52
8.2 代码、数据库和接口的同步边界	52
8.3 每日同步节奏	53
8.4 前端无等待开发	53
8.5 每位成员的本机启动方式	53
8.6 调试流程	53

8.7 联调完成标准	54
9. 本地开发与 CI	54
10. 明确不采用	54
11. Day 1 技术栈确认清单	54
HoldHive Git 分支与 GitHub 自动化流程	56
1. 决策摘要	56
2. 分支何时创建	56
3. 初始化命令	56
4. 日常使用流程	57
4.1 功能开发	57
4.2 QA 集成到 main	57
4.3 main 发布到 prod	57
4.4 Hotfix 流程	58
5. 分支保护规则	58
6. Merge 策略	59
7. GitHub Actions 自动化设计	59
7.1 工作流文件	59
7.2 PR 门禁	59
7.3 QA 集成	59
7.4 Release 验证	60
8. 当前 GitHub Actions	60
8.1 可选后续增强	60
9. 自动化门禁矩阵	61
10. 四天落地计划	61
11. 常见冲突处理	61
12. 参考链接	61
HoldHive 实时股价 API 方案	62
1. 决策摘要	62
2. 候选接口与使用结论	63
2.1 资产类型与行情策略	63
3. 东方财富接口用法	63
3.1 secid 规则	63
3.2 批量报价	64
3.3 单标的报价	64
3.4 搜索建议	65
4. 腾讯与新浪 fallback	65
5. 官方 API 备选	65
5.1 Tiingo Starter 免费备选	65

5.2 Finnhub	66
5.3 Alpha Vantage 不推荐作为主链路	66
5.4 AKShare / AKTools 判断	66
5.5 加密资产行情	66
6. 后端统一接口	67
6.1 搜索证券	67
6.2 批量获取报价	67
6.3 前端使用规则	68
7. 缓存、限流和失败策略	68
7.1 后端缓存	68
7.2 后端限流	68
7.3 失败降级顺序	68
8. 配置建议	68
9. 测试与验收	69
10. 实测记录	69
11. 最终推荐	69

HoldHive 数据库设计 70

1. 设计目标	70
2. 分阶段模型	70
MVP 必须实现	70
后续扩展	70
3. 实体关系	71
4. 表结构	71
4.1 portfolio	71
4.2 instrument	72
4.3 holding	72
4.4 price_snapshot	73
5. 参考 DDL	73
6. 索引设计	74
7. 一致性与事务	74
8. 扩展到交易账本	74
9. 数据库验收清单	75

HoldHive REST API 文档 76

1. 文档范围	76
2. 通用约定	76
2.1 标识符	76
2.2 数值与币种	76
2.3 资产类型	76

2.4 数据状态	76
2.5 错误响应	77
2.6 错误码	77
3. 数据模型	78
3.1 Holding	78
3.2 PortfolioSummary	78
4. MVP 接口	79
4.1 查询全部持仓	79
4.2 查询单个持仓	79
4.3 创建持仓	79
4.4 修改持仓数量和平均成本	81
4.5 删除持仓	81
4.6 查询组合摘要	81
4.7 搜索证券	81
4.8 批量获取报价	82
4.9 查询基金穿透	82
4.10 查询组合穿透暴露	83
4.11 健康检查	84
4.12 组合分析	84
4.13 当前组合分析 — 结构化事实	84
4.14 当前组合分析 — AI 流式解读	85
4.15 当前组合分析 — 合并 SSE (推荐)	85
5. 计算规则	86
6. 一致性、缓存与失败处理	86
7. 后续扩展接口	86
7.1 基金穿透展示规则	86
7.2 大模型组合分析接口	87
8. 安全约束	87
9. API 验收与测试矩阵	88
10. 完成标准	88

HoldHive 团队项目执行指南

项目周期：两天规划 + 两天编码

当前交付：main 已同步 qa 并打 tag 1.0.0。后续修订仍从 qa 创建 docs/*、bugfix/* 或 feature/* 分支，合并回 qa 后再按需同步到 main。

配套设计文档：

- 技术栈定案
- 成员目录分工速查
- Git 分支与 GitHub 自动化流程
- 实时股价 API 方案
- 数据库设计
- REST API 文档

1. 项目目标

HoldHive 是一个面向单一用户的多资产投资组合管理应用。用户可以查看、添加和删除股票、场内基金、场外基金、加密资产、现金和银行存款，并查看组合市值、盈亏和图表化表现。

执行原则是先交付稳定、可解释、可演示的 MVP，再考虑增强功能。任何扩展都不得影响持仓管理、估值计算、图表展示和本地启动流程。

1.1 1.0.0 已交付范围

1.0.0 版本已覆盖从录入、估值、穿透到分析展示的完整演示链路：

- 后端：MySQL + Flyway schema/seed、持仓 CRUD、行情搜索/报价、缓存降级、基金穿透、组合暴露、规则型组合分析和 DeepSeek 兼容 LLM 解读。
- 前端：Gateway 六边形入口、Dashboard、Holdings、Performance、Analysis、Add Holding、Settings 多页面，以及 day/night 主题、可拖动侧边栏、data mode、loading/thinking 动画和用户友好 toast。
- 数据：Flyway V4__seed_demo_portfolio_data.sql 提供股票、ETF、场外基金、crypto、cash、bank deposit 和 unpriced holding，团队成员同步后即可读取演示数据。
- 流程：所有功能通过 feature/* -> qa -> main PR 合并，GitHub Actions 覆盖 backend、backend mysql smoke、frontend 和 docs。

2. 竞品介绍和链接

以下调研基于各产品官网或官方帮助中心，重点观察大型投资组合产品如何解决“数据录入、组合总览、收益解释、风险认知”四类问题。竞品能力用于启发范围取舍，不代表 HoldHive 要在四天内复制这些产品。

产品	市场定位与代表能力	HoldHive 可借鉴	本期不复制
Empower Personal Dashboard	聚合投资、现金和负债账户，并把资产配置、现金流、退休规划放进统一财富视图。	首页先给用户整体结论，再允许查看明细。	多机构账户聚合、预算及退休规划。
Morningstar Investor Portfolio	通过 Portfolio X-Ray、基准对比、行业暴露和持仓重合分析解释组合风险。	图表必须回答一个问题，不能只展示数字；后续可扩展集中度分析。	评级体系、基金穿透和专业研究数据。
Sharesight	覆盖多市场、多币种、股息、公司行动、收益归因和税务报告；强调准确的业绩追踪。	将投入成本、当前价值和盈亏分开，清楚说明计算口径。	税务、公司行动、多币种收益归因。

产品	市场定位与代表能力	HoldHive 可借鉴	本期不复制
TradingView Portfolios	将组合价值、收益历史、基准、资产/行业/币种配置，以及 Sharpe、Sortino 等风险指标集中展示。	采用“总览-持仓-分析”的渐进式信息架构。	专业技术指标、新闻、基准模拟和高级风险模型。
Yahoo Finance Portfolios	支持手工录入、CSV、券商连接、实际/模拟/观察组合，以及可定制视图。	新用户必须能手工快速建立第一个组合，不依赖复杂连接。	券商同步、交易批次、股息和现金流水。
Kubera	从资产、债务、净值和配置角度提供全局财富趋势，并可汇总多个子组合。	产品语言应围绕“我的整体情况”，而不是堆砌证券术语。	实物资产、债务和家庭/实体级组合。
Portfolio Performance	开源桌面工具，支持交易、历史价格、自定义分类、目标配置、再平衡、TTWROR 和 IRR。	计算需要可解释、可测试，并在界面说明口径。	完整交易账本、TTWROR/IRR 和高级再平衡。

竞品结论

1. 首页需要先回答“我现在怎么样”：总价值、投入成本、盈亏和资产配置应在首屏出现。
2. 数据录入决定首次体验：大型产品普遍支持连接、导入或手工录入；本期只做好最低风险的手工录入。
3. 分析价值来自解释而非指标数量：一个清晰的配置图比十个无人理解的专业指标更适合 MVP。
4. 收益口径必须透明：不同产品可能采用资金加权或时间加权收益；本期只计算基于当前价格和平均成本的未实现盈亏，并明确标注。
5. 可信度是金融产品的核心体验：必须显示价格更新时间、数据来源、加载失败和演示数据状态，不能用静默降级制造虚假的精确感。

HoldHive 的差异化机会

HoldHive 不追求“功能最多”，而定位为一个可解释、低门槛、适合投资初学者的组合快照：用户在一分钟内完成三件事——看懂组合价值、识别资产集中情况、维护一项持仓。它用更少的字段和更明确的语言换取更低的学习成本，也让团队能够在两天编码时间内交付一个可以完整解释和可靠演示的产品。

3. 产品设计

3.1 背景故事

刚开始工作的年轻职员 Alex 通过不同平台买了几只股票、一只场内基金和一只场外基金，也持有少量 BTC，并保留了一部分现金和银行存款。他知道每个 App 里的单项涨跌，却回答不了三个更重要的问题：自己一共投入了多少钱、整个组合现在值多少、风险是否因为基金底层持仓而集中在某些股票上。

Alex 尝试过电子表格，但价格需要手动更新，公式也容易出错；成熟的财富管理工具能力强，却包含账户聚合、税务、退休规划和复杂风险指标，第一次使用时很难判断应该先看什么。于是他逐渐回到“分别打开几个应用看价格”的旧习惯，拥有大量行情信息，却没有形成组合视角。

HoldHive 的名字来自蜂巢：每项资产像一个独立蜂房，只有汇集起来才能看见完整结构。Alex 只需选择资产类型，录入代码、数量和平均买入价，系统就把股票、基金、加密资产、现金和存款整理成一张可信的组合快照。打开首页时，他首先看到总价值和盈亏，然后通过资产配置图识别集中风险；当添加基金时，系统提醒基金本身可能包含股票，并在后续阶段提供基金穿透分析。

演示故事以 Alex 为主线：他首次打开空组合，加入一只股票、一只场内基金、一只场外基金、一项加密资产、一笔现金和一笔银行存款，看到组合价值和配置图形成；随后发现基金可能与直持股票存在重叠，并删除一条错误记录。整个过程展示 HoldHive 如何把“零散资产”变为“可理解的组合”。

3.2 产品愿景与价值主张

愿景：让投资初学者无需掌握专业金融术语，也能快速建立并理解自己的投资组合快照。

价值主张：HoldHive 用最少的输入，清楚呈现“持有什么、现在值多少、盈亏如何、是否集中”，并对数据来源和计算口径保持透明。

一句话介绍：把零散持仓放进一个蜂巢，一分钟看懂你的组合。

3.3 目标用户与非目标用户

主要用户：持有少量股票、基金、加密资产、现金和银行存款，想集中查看整体情况，但暂时不需要专业交易、税务或账户聚合系统的投资初学者。

核心需求：

- 快速录入股票、场内基金、场外基金、加密资产、现金和银行存款，不需要连接真实券商、基金平台、交易所、银行或钱包账户。
- 一眼看懂总投入、当前价值和未实现盈亏。
- 识别单一持仓或单一资产类型占比过高的情况。
- 在市场数据不可用时知道发生了什么，而不是看到错误数字。

非目标用户：高频交易者、专业基金经理、需要报税/审计报表的用户、需要多账户多币种精确归因的用户。本期也不提供认证、真实交易、买卖建议、自动调仓、链上钱包读取、交易所账户同步或现金流管理。

3.3.1 MVP 资产类型

HoldHive MVP 支持六类资产，范围必须明确，避免四天项目膨胀成完整财富管理系统：

资产类型	示例	MVP 能力	不做内容
STOCK	AAPL、600519	行情或演示价格、市值、盈亏、占比	专业研报、财报因子、交易下单
ETF	VOO、SPY、510300	场内基金，与股票同样录入和估值，配置图单独分类；添加时提示可能有底层股票；1.0.0 已支持基金穿透和组合暴露	基金评级、申赎流水、完整披露历史
MUTUAL_FUND	开放式基金、货币基金	场外基金，MVP 可用手工/演示净值估值；添加时提示披露滞后和底层资产	申赎流水、费率和基金评级
CRYPTO	BTC、ETH	支持展示、手工录入和 CoinGecko/cache/demo 价格估值	钱包连接、链上交易历史、币币兑换
CASH	USD	固定按 1.00000000 估值，纳入总值和配置	存取款流水、利息、外汇换算
BANK_DEPOSIT	HSBC_USD、USD_DEPOSIT	固定按本金估值，纳入总值和配置	利息自动计提、到期收益、银行账户同步

本期默认基准币种为 USD。不同币种之间不做自动 FX 换算；如果添加现金或银行存款，建议只添加组合基准币种。

3.3.2 基金穿透与解耦

基金应与股票估值解耦处理。用户添加 ETF 或 MUTUAL_FUND 时，系统先把它作为一项独立资产保存和估值；基金底层股票、债券或现金只进入“穿透分析”模块，不写回持仓，不改变基金市值，也不自动生成股票持仓。

前端必须在添加基金时显示提醒：

This fund may contain stocks already held directly. Lookthrough data is for exposure analysis only and may lag the latest disclosure.

规划阶段分期规则如下；1.0.0 已完成 P0，并把 P1/P2 中最关键的穿透展示和组合暴露纳入演示链路：

- P0：允许添加场内/场外基金，并显示“可能包含底层股票”的提示。
- P1：后端提供基金穿透接口，展示基金最新披露的 Top Holdings 和权重。
- P2：组合暴露页支持 `lookthrough=true`，把直持股票和基金派生股票暴露分开展示，并提示重叠。

3.3.3 大模型分析

大模型分析在规划时被放在最后阶段；1.0.0 已实现为可降级的 Analysis 页面增强功能。该功能只消费后端整理后的结构化快照，包括资产类型、市值占比、价格状态、基金穿透摘要和风险提示；它不直接访问数据库、不调用第三方行情、不生成交易建议。

前端展示原则：

- 标题使用“AI Portfolio Brief”或“组合解读”，避免写成“投资建议”。
- 每条结论必须能追溯到输入数据，例如资产占比、基金重叠、现金/存款比例。
- 固定显示免责声明：分析仅用于教育和解释，不构成买卖建议。
- 大模型调用失败时，页面保留规则型分析，不影响持仓、估值和图表。

3.4 设计原则

1. 结论优先：首页先展示整体组合，再展示单项持仓。
2. 渐进披露：默认只显示关键指标，详细数据在表格中展开。
3. 透明可信：每个市场价格显示来源、时间和实时/演示状态。
4. 防止误操作：表单即时校验，删除需要二次确认。
5. 可解释优先：只展示团队可以解释、测试和演示的金融计算。
6. 无数据也是完整状态：空组合、加载中、失败和部分数据缺失均有明确界面。
7. 低耦合与明确依赖：每个模块只依赖更稳定、更底层的接口，不跨层直接调用实现细节；新增依赖必须说明调用方向、替换方式和测试隔离方式。

3.5 信息架构

MVP 采用单页 Dashboard，避免在两天编码时间内引入不必要的路由和状态复杂度：

```
HoldHive Dashboard
├── 顶部栏：品牌、价格更新时间、数据状态
├── 组合摘要：总投入、当前价值、未实现盈亏、持仓数
├── 资产配置：按资产类型和持仓市值展示的环形图和图例
├── 持仓列表：assetType、ticker、数量、均价、当前价、市值、盈亏、占比、操作
├── 添加持仓：按钮打开表单或侧栏
└── 全局反馈：成功提示、错误提示、删除确认
```

如果时间允许，再平衡提示放在摘要和图表之间，明确展示触发规则和计算依据。

3.6 核心用户流程

流程 A：建立第一个组合

1. 用户看到空状态和“添加第一项持仓”按钮。
2. 选择 assetType，输入 ticker、quantity、averagePurchasePrice；现金使用币种代码作为 ticker，银行存款使用可读代码。
3. 前端即时检查必填、数值范围和格式。
4. 后端再次验证并保存。
5. 页面刷新摘要、图表和持仓表，并显示成功反馈。

流程 B：理解组合

1. 用户打开 Dashboard。
2. 首先读取当前价值和未实现盈亏。
3. 查看资产配置图，识别最大持仓。
4. 在持仓表中查看单项市值、盈亏和占比。
5. 如果价格缺失，系统明确标记受影响持仓，不把未知价格当作 0。
6. 如果组合中包含基金，系统在分析页解释基金可能包含股票，并区分直持暴露和基金派生暴露。

流程 C：删除错误持仓

- 1. 用户在持仓行选择删除。
- 2. 确认框显示 ticker，并说明操作结果。
- 3. 用户确认后调用删除 API。
- 4. 成功后同步刷新摘要、图表和列表；失败则保留原数据并显示原因。

3.7 页面和状态设计

区域	展示内容	交互与状态
顶部栏	HoldHive 品牌、数据状态、最后更新时间	数据为 mock 时显示“演示数据”；失败时提供重试
摘要卡	总投入、当前价值、未实现盈亏、持仓数	盈亏同时使用正负号、文字和颜色，避免只依赖颜色
配置图	资产类型和每项持仓的市值占比	图例含 assetType、ticker、比例和金额；无有效价格时解释无法计算
持仓表	assetType、ticker、数量、均价、现价、市值、盈亏、占比	支持删除；移动端允许横向滚动或折叠次要列
添加表单	assetType、ticker、数量、平均买入价	保留用户输入；基金、现金、存款显示专属提示；错误定位到字段；提交时防止重复点击
基金穿透卡	基金 Top Holdings、披露日期、重叠股票提醒	1.0.0 已通过基金穿透和 portfolio exposure 展示，穿透数据不改变主持仓
AI 分析卡	大模型生成的组合解读、关键发现、数据限制	1.0.0 已接入 DeepSeek 兼容 SSE；失败时保留规则型分析，必须带非投资建议免责声明
空状态	产品价值说明和主行动按钮	不显示全是 0 的无意义图表
错误状态	可理解的原因和恢复动作	API 失败可重试；表单失败不关闭表单

3.8 界面参考、现代交互与动画

[docs/design/assets/sample_screenshot.png](#) 展示了一个适合本项目的基础 Portfolio Dashboard：左侧导航、顶部组合 KPI、两张图表卡、中部持仓表和底部资产卡。HoldHive 不需要逐像素复刻，但应保留这种信息层级，并做得更现代、直观和用户友好。

竞品调研给出的界面方向如下：Empower 的个人 Dashboard 强调整体投资配置和账户级总览；Morningstar Portfolio X-Ray 用资产类别、地域、行业和 Top Holdings 等视角解释组合风险；Sharesight 的报表强调按日期范围查看组合 performance；TradingView Portfolios 同时提供组合价值/表现视图、distribution 视图和 holdings 数据视图。HoldHive 的 MVP 应吸收这些成熟产品的共同点：首屏先给组合结论，图表解释配置和变化，表格承担可审计明细，状态提示解释数据可信度。

页面布局建议

区域	视觉目标	MVP 实现
顶部 KPI	让用户 3 秒内看懂当前状态	总市值、未实现盈亏、持仓数、价格状态、添加持仓按钮
左侧导航	形成现代产品感，但不增加路由复杂度	保留 Dashboard；Holdings/Performance 可显示为禁用或作为页面锚点
图表卡片	比表格更快解释组合	必做资产配置 donut；时间允许加组合价值 line/area chart
持仓表	提供可审计明细	asset type、ticker、quantity、average price、current price、market value、P/L、allocation、delete
Insight 卡片	把指标翻译成人话	最大持仓、资产类型占比、基金重叠、未定价持仓、演示价格、集中度提示
底部卡片	做二级摘要	priced holdings、unpriced holdings、top holding、data source

图表设计

图表	优先级	前端库建议	数据来源	说明
Asset Allocation Donut	P0	Recharts PieChart 或 Chart.js doughnut	summary.allocations	展示股票、场内基金、场外基金、加密资产、现金、银行存款及各持仓市值占比，必须配文字图例
Fund Lookthrough Card	P1 / 已实现	普通卡片 + 表格，不强依赖图表库	fundLookthrough.holdings、portfolio/exposure	展示基金底层 Top Holdings、权重、披露日期和重叠提醒
AI Portfolio Brief	Final / 已实现	普通卡片 + 状态标签	portfolio/analysis/insights/full	展示解释性结论、数据限制和免责声明，不做买卖建议
Portfolio Value Trend	P1	Recharts AreaChart / LineChart 或 Chart.js line	演示历史估值或后续 portfolio_valuation	用于增强展示，不作为 P0 阻塞
Gain/Loss Bar	P1	Recharts BarChart 或 Chart.js bar	holdings	展示每个持仓盈亏，帮助识别贡献项
Concentration Indicator	P1	CSS progress/ring 或 Recharts radial	summary 最大占比	超过 40% 时提示，不做投资建议

图表必须同时提供文字图例和数值，不得只依赖颜色。正收益使用绿色、负收益使用红色，但必须同时使用 +/- 符号和文字说明，照顾色弱用户。

用户友好提示

场景	文案示例	位置
空组合	Add your first holding to create a portfolio snapshot.	空状态
添加成功	AAPL added. Portfolio totals refreshed.	Toast
删除成功	AAPL removed. Allocation and totals updated.	Toast
表单校验	Quantity must be greater than 0.	字段下方
重复持仓	AAPL already exists. Edit the existing holding instead of adding a duplicate.	表单顶部
场内基金	ETF trades like a stock, but it may contain underlying stocks.	资产类型提示
场外基金	Mutual fund data may update by disclosure or NAV schedule, not intraday.	资产类型提示
基金重叠	This fund may contain stocks you already hold directly.	添加表单和分析页
现金录入	Cash is valued at 1.00 in the portfolio base currency.	资产类型提示
银行存款	Bank deposit is shown at principal value. Interest is not accrued automatically.	资产类型提示
加密资产	Crypto prices use demo or cached data in MVP.	表单或状态标签
大模型分析	AI analysis explains the current snapshot and is not investment advice.	AI 分析卡
演示价格	Demo prices are shown for training. They are not live market data.	顶部状态条
部分估值	Some holdings have no price. Totals only include priced holdings.	摘要区
价格失败	Price service is unavailable. Saved holdings are still safe.	Banner
集中度提示	AAPL is 46% of priced market value. Consider whether this concentration matches your plan.	Insight 卡片

动态与过渡

- 页面首次加载时，KPI、图表和表格使用 120-180ms 的轻微 fade/slide-in。
- 新增或删除持仓后，总市值、盈亏和持仓数做 300ms 数字高亮或 count-up。
- Donut 和 line chart 使用 400-700ms 内置动画；避免循环动画。
- 新增表格行使用浅色背景高亮 1 秒；删除必须先确认。
- 表单提交按钮显示 loading 并禁用，防止重复提交。
- 加载中使用 skeleton，比全屏 spinner 更稳定。
- 尊重 prefers-reduced-motion；系统要求减少动画时关闭非必要过渡。

不建议在 MVP 做大面积粒子、视频背景、复杂路由转场或遮挡数据的动效。金融 Dashboard 的美观来自清晰层级、稳定布局、准确反馈和细腻过渡，不是装饰性动画。

3.9 功能需求与验收标准

ID	优先级	用户故事	验收标准
HH-01	P0	作为用户，我可以查看全部持仓。	页面和 API 返回股票、场内基金、场外基金、加密资产、现金和银行存款；无数据时显示可行动的空状态。
HH-02	P0	作为用户，我可以添加持仓。	assetType 合法；ticker 非空并标准化为大写；quantity 大于 0；平均买入价大于等于 0；成功后所有区域同步更新。
HH-03	P0	作为用户，我可以删除错误持仓。	删除前确认；成功后记录消失并重新计算；不存在的 ID 返回明确错误。
HH-04	P0	作为用户，我可以查看组合摘要。	正确显示总投入、当前价值、未实现盈亏和持仓数；空组合不报错。
HH-05	P0	作为用户，我可以查看资产配置。	图表比例基于有效市值，至少区分股票、场内基金、场外基金、加密资产、现金和银行存款；合计受舍入影响时约为 100%；缺失价格被明确标记。
HH-06	P0	作为用户，我知道数据是否可信。	显示价格来源、更新时间、实时/缓存/演示状态；市场服务失败不返回伪造的实时价格。
HH-07	P1	作为用户，我可以看到集中度提示。	任一持仓占有效总市值超过 40% 时显示解释性提示，并说明该提示仅用于风险认知。
HH-08	P0	作为用户，我可以把现金纳入组合。	现金以基准币种固定价格 1.00 纳入总值、配置和持仓表，不请求外部行情。
HH-09	P0	作为用户，我可以把银行存款纳入组合。	银行存款以本金固定价格 1.00 纳入总值、配置和持仓表，不自动计算利息。
HH-10	P0	作为用户，我添加基金时会收到底层持仓提醒。	选择 ETF 或 MUTUAL_FUND 时出现基金可能包含股票和披露滞后的提示。
HH-11	P1	作为用户，我可以查看基金底层股票。	基金穿透接口返回 Top Holdings、权重、披露日期和数据来源；穿透数据不改变主持仓。
HH-12	P2	作为用户，我可以查看穿透后的组合暴露。	页面区分直持暴露和基金派生暴露，并提示重叠股票。
HH-13	Final	作为用户，我可以查看 AI 组合解读。	大模型结论带数据依据、限制和免责声明；失败时不影响规则型分析。

3.10 计算口径

对股票、场内基金、场外基金和加密资产：

投入成本 = quantity × averagePurchasePrice
当前市值 = quantity × currentPrice
未实现盈亏 = 当前市值 - 投入成本
未实现盈亏率 = 未实现盈亏 ÷ 投入成本 × 100%

持仓占比 = 当前市值 ÷ 全部有效持仓当前市值 × 100%

现金和银行存款使用固定规则：

当前价格 = 1.00000000
投入成本 = quantity × 1.00000000
当前市值 = quantity × 1.00000000
未实现盈亏 = 0
价格状态 = FIXED

组合总值为所有具有有效价格的持仓市值之和。现金和银行存款始终属于有效估值；股票、场内基金、场外基金和加密资产如果价格缺失，摘要必须显示“部分估值”，并列出不计入项。基金穿透只用于分析暴露，不参与主持仓市值重复计算。投入成本为 0 时不计算盈亏率，显示 **N/A**，避免除零。金额统一使用两位小数；内部计算不得过早舍入。

本期的“表现”是当前快照下的未实现盈亏，不等于考虑资金流、股息、费用和持有期的真实投资回报率。界面和演示必须说明该限制。

3.11 API 边界建议

方法与路径	目的	关键响应
GET /api/holdings	查询全部持仓及估值	200；空组合返回空数组
POST /api/holdings	新增持仓	201；无效字段返回 400
PATCH /api/holdings/{id}	修改数量和平均成本	200；无效字段返回 400；不存在返回 404
DELETE /api/holdings/{id}	删除持仓	204；不存在返回 404
GET /api/portfolio/summary	获取摘要、配置和数据状态	200；价格服务异常时返回部分结果和状态

错误响应采用一致结构，例如 `code`、`message`、`fieldErrors` 和 `timestamp`。技术堆栈确定后，字段名和完整 schema 应写入 OpenAPI，并由产品负责人和技术评审方确认。

3.12 非功能与风险设计

- 性能：演示数据规模下，主要查询应在正常本地环境中快速响应；外部价格请求设置超时，避免页面无限等待。
- 可访问性：表单包含 label；键盘可操作；焦点可见；盈亏不只依赖红绿颜色表达；图表提供文字图例。
- 数据安全：MVP 无认证且只假设单用户，只能用于本地或受控演示环境；禁止部署为公开的真实资产管理服务。
- 隐私：不录入姓名、账户号、券商凭据或其他个人敏感信息；演示只使用虚构数据。
- 可靠性：数据库是持仓事实来源；市场价格失败不得破坏已保存持仓。
- 可维护性：市场数据通过适配器隔离，使真实数据和演示数据可以替换；业务计算保持纯函数并由单元测试覆盖。

3.13 成功指标与范围边界

本期成功不以功能数量衡量，而以以下结果衡量：

- 新用户可以在 60 秒内添加第一项持仓。
- 用户可以在首页回答“总值多少、盈亏多少、最大持仓是什么”。
- 全部 P0 验收标准通过，后端行覆盖率至少 70%。
- 价格服务断网时仍可演示已保存持仓，并明确解释数据状态。
- 四位成员都能解释产品取舍、计算公式和自己负责或审查的代码。

明确不在本期范围内：登录和多用户、真实交易、券商连接、交易所连接、钱包连接、银行连接、CSV 导入、现金流水、基金申赎流水、存款利息自动计提、股息、税务、公司行动、多币种自动换算、历史交易账本、基准比较、专业风险指标和个性化投资建议。基金穿透、组合暴露和大模型解读在 1.0.0 中作为演示增强实现，但它们不改变持仓 CRUD、基础估值和数据可信度标识这条核心链路。

4. MVP 范围

必须完成

- 持久化保存持仓数据。
- REST API 支持持仓查询、新增和删除。
- UI 支持浏览组合、新增持仓和删除持仓。
- 支持 STOCK、ETF、MUTUAL_FUND、CRYPTO、CASH、BANK_DEPOSIT 六类资产的展示和基础估值。
- 添加基金时给出底层持仓和重叠暴露提醒。
- 展示组合总市值、总盈亏及至少一个图表。
- 后端单元测试行覆盖率不低于 70%。
- 提供 API 文档、架构图、ERD 和可运行说明。

可选增强

- 接入股票/ETF 市场价格数据；外部服务不可用时使用明确标识的演示数据。
- 接入基金净值或基金持仓披露数据，并显示穿透后的 Top Holdings。
- 接入实时加密资产价格接口；MVP 可先使用演示或缓存价格。
- 规则型再平衡提示，例如单一资产占比超过 40%。
- 规则型集中度提示、价格缓存状态和更完整的组合历史视图。
- 大模型组合解读；只作为最后阶段功能，不影响核心估值链路。

5. 最小数据模型

初版不要引入复杂金融模型，只保存最小持仓字段：

字段	说明
id	持仓唯一标识
assetType	资产类型：STOCK、ETF、MUTUAL_FUND、CRYPTO、CASH、BANK_DEPOSIT
ticker	资产代码，例如 AAPL、VOO、BTC、USD、HSBC_USD
quantity	持仓数量，必须大于 0
averagePurchasePrice	平均买入价，必须大于等于 0
createdAt	创建时间

当前价格、市值和盈亏在查询时由价格服务及业务计算获得，不作为 MVP 的核心持久化字段。

6. 四人分工

成员	主责	主要交付物	审查责任
成员 A：后端与架构	数据模型、数据库、核心 API	ERD、CRUD API、数据库迁移、OpenAPI 文档	审查成员 B 的后端 PR
成员 B：业务逻辑与质量	组合计算、价格服务、后端测试	市值/盈亏计算、异常处理、单元测试、覆盖率报告	审查成员 A 的后端 PR
成员 C：前端与体验	页面、图表、前后端联调	持仓列表、表单、删除、汇总、图表	审查成员 D 的文档或配置 PR
成员 D：产品、QA 与交付	需求管理、验收、集成、演示	看板、验收清单、ADR、测试记录、PPT	审查成员 C 的前端 PR

分工不等于隔离。每位成员必须至少提交一个功能 PR、人工审查一个队友 PR、参与测试或文档维护，并在最终展示中发言。

6.1 后端两人细分

后端两人不要按 **Controller / Service / Repository** 这种横切层分工，否则会频繁修改同一批文件，产生冲突并拖慢联调。最高效的分法是按“数据主链路”和“业务质量链路”拆分：

后端成员	核心职责	具体任务	交付边界
成员 A：API + 数据库负责人	让持仓数据可靠进出 MySQL	Flyway 建表、Entity/Repository 或 JDBC DAO、 GET /holdings 、 POST /holdings 、 PATCH /holdings/{id} 、 DELETE /holdings/{id} 、assetType 字段落库、基础 Swagger/OpenAPI	前端可以通过 API 完成股票、ETF、场外基金、加密资产、现金和银行存款的查询、新增、修改和删除
成员 B：业务计算 + 质量负责人	让数据被正确计算、校验、测试并稳定返回	PortfolioCalculator 、summary API、demo/market price adapter、现金/存款固定估值、加密资产演示价格、基金穿透 demo endpoint、统一错误响应、validation、JUnit/Mockito/MockMvc 测试、JaCoCo 覆盖率	前端可以拿到多资产 summary、基金底层持仓提示、图表数据和稳定错误格式

协作规则：

- 成员 A 先打通 CRUD 主链路，即使价格先返回 **UNAVAILABLE**。
- 成员 B 的估值和占比计算先写成可测试的 domain/service 方法，不直接堆在 Controller 中。
- 成员 A 负责数据库 migration 追加和本地启动脚本；成员 B 负责测试矩阵和覆盖率门槛。
- 两人每天合并 2-3 个小 PR，不把所有后端代码攒到一个大 PR。
- API DTO、错误响应和字段名一旦给前端使用，修改前必须先通知前端并同步文档。
- 已经共享出去的 Flyway migration 不反复改，后续变化用新版本 migration。
- 每个后端 PR 互相 review：A 检查 B 的业务计算是否能接上数据库，B 检查 A 的 CRUD 是否覆盖错误和边界。

按天拆分建议：

时间	成员 A：API + 数据库	成员 B：业务计算 + 质量
Day 1-2 Planning	定 MySQL 表结构、Flyway 文件、Holding/Instrument/PriceSnapshot 字段、assetType 约束、ERD	定错误响应、summary response、priceStatus、valuationStatus、现金/股票/ETF/加密资产计算规则、测试矩阵
Day 3 Coding	实现 migration、持仓实体/DAO、CRUD API、基础 API 文档	实现 portfolio summary、price adapter、现金固定估值、加密资产 demo quote、validation、global exception handler、核心单元测试
Day 4 Coding	修 CRUD/数据库 bug、支持 Swagger/curl 联调、维护启动说明	补 MockMvc/API 错误测试、覆盖率达到 70%+、支持图表字段、多资产 allocation 和演示数据

一句话边界：成员 A 负责“数据能可靠进出 MySQL”，成员 B 负责“数据能被正确计算、校验、测试并稳定返回给前端”。

6.2 每人实现目录

以下目录结构用于编码任务、PR review 和联调验收。Day 1 创建项目骨架时应按本节先建立目录边界，不要求一次写满所有文件；Day 3-4 再按任务逐步实现。目录归属表示主要负责人，不表示其他成员不能修改；跨负责人目录的改动必须在 PR 描述中说明原因，并请对应负责人 review。

仓库中已为主要工作区保留入口说明：后端见 `backend/src/main/java/com/holdhive/*`/README.md 和 `backend/src/test/java/com/holdhive`/README.md；前端见 `frontend/src`/README.md、`frontend/src/api`/README.md 和 `frontend/src/features/portfolio`/README.md；成员拉取仓库后可先阅读 [成员目录分工速查](#)，再进入自己的 feature 分支编码。

目录责任总览

成员	主要实现目录	主要文件类型	不建议直接负责
成员 A	backend/src/main/java/.../holdhive/portfolio/api、portfolio/persistence、src/main/resources/db/migration	Controller、DTO、Entity、Repository、Flyway SQL、OpenAPI 基础配置、assetType 持久化	组合估值公式和价格适配器内部逻辑
成员 B	backend/src/main/java/.../holdhive/portfolio/application、portfolio/domain、pricing、common/error	Service、Calculator、Price Adapter、现金固定估值、加密资产演示报价、异常处理、后端测试	Flyway 建表主迁移和前端页面组件
成员 C	frontend/src/api、frontend/src/features/portfolio、frontend/src/components、frontend/src/styles	React 组件、Hook、DTO、样式、图表、前端测试、多资产标签和筛选	后端数据库迁移和核心业务计算
成员 D	.github/workflows、docs/qa、docs/demo、docs/adr、scripts/mysql、.env.example、docs/guideline	CI 配置、验收清单、缺陷记录、演示脚本、ADR、交付文档	未经负责人确认直接改 API 字段或数据库结构

每个成员的 feature 分支建议和目录对应：成员 A 使用 `feature/backend-crud-db`，成员 B 使用 `feature/backend-summary-pricing`，成员 C 使用 `feature/frontend-dashboard`，成员 D 使用 `feature/qa-ci-docs`。所有分支最终通过 PR 合并到 `qa`，不直接推送到 `main` 或 `prod`。

后端包名前缀以实际 `groupId` 为准，示例中用 `.../holdhive` 表示。

成员 A：后端 API 与数据库

成员 A 负责数据库结构、持仓 CRUD 主链路、JPA 映射和后端启动配置。目标是让持仓数据能够可靠写入 MySQL、从 MySQL 读出，并通过基础 REST API 暴露给前端。

```

backend/
├── pom.xml                # 与成员 B 共同维护依赖和插件
├── src/main/java/.../holdhive/
│   ├── HoldHiveApplication.java    # Spring Boot 入口
│   ├── common/
│   │   └── config/
│   │       ├── OpenApiConfig.java    # Swagger/OpenAPI 基础配置
│   │       └── CorsConfig.java        # 本地前后端联调配置
│   └── portfolio/
│       ├── api/
│       │   ├── HoldingController.java    # GET/POST/DELETE holdings
│       │   └── dto/
│       │       ├── CreateHoldingRequest.java
│       │       ├── HoldingResponse.java
│       │       └── HoldingMapper.java
│       └── persistence/
│           ├── entity/
│           │   ├── PortfolioEntity.java
│           │   ├── InstrumentEntity.java
│           │   ├── HoldingEntity.java
│           │   └── PriceSnapshotEntity.java
│           └── repository/
│               ├── PortfolioRepository.java
│               ├── InstrumentRepository.java
│               ├── HoldingRepository.java
│               └── PriceSnapshotRepository.java
├── src/main/resources/
│   ├── application.yml
│   ├── application-local.yml
│   ├── application-test.yml
│   └── db/migration/
│       ├── V1__create_portfolio_tables.sql
│       ├── V2__create_price_snapshot_table.sql
│       └── V3__seed_demo_portfolio.sql
└── src/test/java/.../holdhive/
    ├── portfolio/persistence/
    │   ├── HoldingRepositoryTest.java
    │   └── FlywayMigrationTest.java
  
```

成员 A 的交付边界：

- MySQL 表结构、索引、外键和唯一约束可由 Flyway 从空库创建。
- `GET /api/v1/holdings`、`POST /api/v1/holdings`、`PATCH /api/v1/holdings/{id}`、`DELETE /api/v1/holdings/{id}` 可运行。
- Repository 测试覆盖唯一约束、基础查询和持仓删除。
- 本地启动说明中的数据库连接和 profile 可用。

成员 B：后端业务计算与质量

成员 B 负责组合估值、价格适配器、统一错误响应和后端测试矩阵。目标是让 API 返回的市值、盈亏、占比、价格状态和错误格式稳定、可测试。

```
backend/
├── src/main/java/.../holdhive/
│   ├── common/
│   │   └── error/
│   │       ├── ApiErrorResponse.java
│   │       ├── GlobalExceptionHandler.java
│   │       └── ErrorCode.java
│   ├── portfolio/
│   │   ├── api/
│   │   │   ├── PortfolioSummaryController.java
│   │   │   └── dto/
│   │   │       ├── PortfolioSummaryResponse.java
│   │   │       ├── AllocationResponse.java
│   │   │       └── UnpricedHoldingResponse.java
│   │   ├── application/
│   │   │   ├── HoldingCommandService.java
│   │   │   ├── HoldingQueryService.java
│   │   │   └── PortfolioSummaryService.java
│   │   └── domain/
│   │       ├── PortfolioCalculator.java
│   │       ├── HoldingValuation.java
│   │       ├── PortfolioValuation.java
│   │       ├── ValuationStatus.java
│   │       └── MoneyMath.java
│   ├── pricing/
│   │   ├── application/
│   │   │   ├── PricingService.java
│   │   │   └── PriceMode.java
│   │   └── domain/
│   │       ├── MarketQuote.java
│   │       └── PriceStatus.java
│   └── infrastructure/
│       ├── EastMoneyPricingAdapter.java
│       ├── DemoPricingAdapter.java
│       └── PricingAdapter.java
├── src/test/java/.../holdhive/
│   ├── portfolio/domain/
│   │   └── PortfolioCalculatorTest.java
│   ├── portfolio/application/
│   │   ├── HoldingCommandServiceTest.java # Mockito mock Repository/PricingAdapter
│   │   └── PortfolioSummaryServiceTest.java # Mockito mock Repository/PricingAdapter
│   ├── portfolio/api/
│   │   ├── HoldingControllerTest.java # MockMvc
│   │   └── PortfolioSummaryControllerTest.java # MockMvc
│   ├── pricing/
│   │   └── PricingServiceTest.java
│   ├── common/error/
│   │   └── GlobalExceptionHandlerTest.java
```

成员 B 的交付边界：

- `GET /api/v1/portfolio/summary` 返回总成本、总市值、未实现盈亏、配置比例和价格状态。
- 业务计算使用 `BigDecimal`，不使用 `double`。
- Mockito 只用于隔离 Service 和 Adapter 依赖，不替代数据库、migration 或 Controller JSON 测试。

- 后端测试覆盖率达到 JaCoCo $\geq 70\%$ 。

成员 C：前端页面与交互

成员 C 负责 React 前端应用、页面状态、图表和前后端联调。目标是让用户可以完成查看组合、添加持仓、删除持仓、查看摘要和图表的完整流程。

```
frontend/
├── package.json
├── vite.config.ts
├── index.html
└── src/
    ├── main.tsx
    ├── App.tsx
    ├── api/
    │   ├── httpClient.ts
    │   ├── portfolioApi.ts
    │   ├── marketApi.ts
    │   └── types.ts
    ├── features/
    │   ├── portfolio/
    │   │   ├── PortfolioDashboard.tsx
    │   │   └── components/
    │   │       ├── SummaryCards.tsx
    │   │       ├── AllocationDonut.tsx
    │   │       ├── HoldingsTable.tsx
    │   │       ├── AddHoldingForm.tsx
    │   │       ├── DeleteHoldingDialog.tsx
    │   │       ├── PriceStatusBadge.tsx
    │   │       └── EmptyPortfolioState.tsx
    │   ├── hooks/
    │   │   ├── usePortfolioData.ts
    │   │   └── useHoldingActions.ts
    │   └── fixtures/
    │       └── portfolioFixtures.ts
    ├── components/
    │   ├── AppShell.tsx
    │   ├── Navigation.tsx
    │   ├── ToastHost.tsx
    │   └── LoadingSkeleton.tsx
    ├── styles/
    │   ├── tokens.css
    │   ├── theme.css
    │   └── global.css
    └── test/
        ├── setup.ts
        ├── PortfolioDashboard.test.tsx
        ├── AddHoldingForm.test.tsx
        └── HoldingsTable.test.tsx
```

成员 C 的交付边界：

- 前端通过 `src/api` 访问后端，不在组件中硬编码第三方行情 URL。
- Dashboard 支持 loading、empty、partial valuation、error 和 success 状态。
- 添加和删除持仓后刷新 holdings 与 summary。
- 图表使用 Recharts，并提供文字图例。
- 前端测试覆盖表单校验、删除确认、价格状态和关键渲染。

成员 D：QA、交付与自动化

成员 D 负责验收资产、协作流程、CI/CD 配置和演示材料。目标是让项目可从零启动、可验收、可回归，并在最终展示时有稳定交付版本。

```
.
├── .github/
│   └── workflows/
│       └── pr-check.yml
```

```

├─── qa-integration.yml
├─── release.yml
├── docs/
│   ├── adr/
│   │   ├── ADR-001-technology-stack.md
│   │   ├── ADR-002-market-data-provider.md
│   │   └── ADR-003-branching-and-ci.md
│   ├── qa/
│   │   ├── acceptance-checklist.md
│   │   ├── defect-log.md
│   │   └── regression-record.md
│   ├── demo/
│   │   ├── demo-script.md
│   │   ├── demo-data.md
│   │   └── fallback-plan.md
│   ├── architecture/
│   │   ├── c4-context.md
│   │   └── erd.md
│   ├── scripts/
│   │   └── mysql/
│   ├── .env.example
│   └── docs/
│       ├── guideline/
│       │   ├── README.md
│       │   ├── project/
│       │   ├── references/
│       │   ├── output/
│       │   └── tools/
│       ├── design/
│       │   ├── lanhu/
│       │   └── assets/

```

成员 D 的交付边界：

- GitHub Actions 能在 PR、**qa**、**main**、**prod** 场景下执行对应检查。
- 验收清单覆盖新增、查询、删除、组合摘要、价格失败和空状态。
- 缺陷记录包含环境、commit、复现步骤、预期、实际和修复验证。
- Demo 脚本、演示数据和失败备用方案可直接用于最终展示。

共享目录规则

目录或文件	主要负责人	共同修改规则
backend/pom.xml	A、B	新依赖必须说明用途；测试依赖优先使用 Spring Boot starter 已包含能力
backend/src/main/resources/db/migration/	A	已合并 migration 不修改；新增变更使用新版本文件
backend/src/main/java/.../portfolio/api/dto/	A、B	字段变更必须同步 API 文档和前端 types.ts
frontend/src/api/types.ts	C	后端字段变更后由 C 更新，A/B review
.github/workflows/	D	影响 CI 门禁的变更必须由对应模块负责人确认
scripts/mysql/.env.example	D、A	数据库和 profile 配置变更必须可本地复现
docs/guideline/project/*.md	D	涉及技术栈、API 或数据库的改动需对应负责人 review

低耦合与依赖规则

本项目按“接口稳定、实现可替换”的原则组织代码。PR 中只要新增跨目录调用、第三方库或共享类型，都必须说明依赖原因；如果无法说明替换方式，默认不合并。

- Frontend UI
 - > frontend hooks
 - > frontend api client + DTO
 - > REST API / OpenAPI

- > backend controller
- > backend application service
- > backend domain service / calculator
- > repository interface
- > JPA entity / MySQL

- backend application service
- > pricing service interface
- > pricing adapter implementation
- > external market data API

依赖约束：

- 前端组件只能依赖 `src/api` 暴露的客户端和 DTO，不直接拼接后端 URL，也不直接访问第三方行情接口。
- 后端 `domain` 不依赖 Spring、JPA、HTTP、数据库或外部行情 API；它只处理业务规则和可测试计算。
- `application` 可以依赖 `domain`、`repository` 接口和 `pricing service` 接口，但不直接解析外部 API 原始响应。
- `persistence` 负责 Entity、Repository 和数据库映射，不写组合估值、价格降级或 HTTP 状态码逻辑。
- `pricing.infrastructure` 可以依赖外部 API SDK 或 HTTP client，但必须通过 `PricingAdapter` 接口对内暴露。
- `common` 只能放错误响应、配置和通用工具，不放 portfolio 或 pricing 的业务规则。
- DTO、Entity、Domain Object 不混用：Controller 返回 DTO，Repository 管 Entity，计算使用 Domain Object。
- 若一个文件需要同时修改三个以上业务方向，优先拆小模块，而不是继续堆逻辑。

PR review 检查点：

- 是否出现 UI 组件直接调用 `fetch`、直接写 URL 或复制后端估值公式。
- 是否出现 Controller 直接访问 Repository 并跳过 application service。
- 是否出现 domain 层 import `org.springframework`、`jakarta.persistence` 或 HTTP client。
- 是否出现价格适配器修改持仓数据，或数据库层决定价格状态。
- 是否新增依赖但没有单元测试、mock/stub 或降级策略。

测试隔离规则：

- Domain 测试不启动 Spring，不连数据库，不访问网络。
- Service 测试用 Mockito mock Repository 和 PricingAdapter。
- Controller 测试用 MockMvc 验证 HTTP 契约，不依赖真实外部行情。
- Repository 测试只验证数据库映射和约束，不验证组合收益算法。
- 前端组件测试 mock `src/api`，不让组件测试依赖真实后端启动状态。

7. 四天计划

Day 1：规划 - 需求与仓库基线

全员完成以下事项：

1. 与产品负责人和技术评审方确认 MVP 字段、价格数据来源和验收优先级。
2. 创建 Jira、Trello 或等价看板；所有工作必须从带优先级和验收标准的 User Story 开始。
3. 创建 `main`、`qa`、`feature/*` 分支。
4. 确认技术栈、目录结构、错误响应格式和 API 命名约定。
5. 创建并审查初版 ERD、C4/架构图、API 契约和演示数据集。
6. 使用 ADR 记录关键技术选择，例如价格服务的降级策略。

完成标准

- MVP 范围、字段和接口经团队确认。
- 看板中每条 P0 用户故事都包含负责人、优先级和验收标准。
- 四人均可启动项目骨架。

- 首个文档或配置 PR 已获队友人工审查。

Day 2：规划 - 任务、测试和演示设计

成员 A 与 B

- 拆分 API 工作：持仓 CRUD、校验、组合计算、价格服务和错误处理。
- 设计测试：正常流程、空组合、无效数量、无效价格、不存在资源、价格服务失败。
- 确认后端覆盖率工具及 70% 门槛。

成员 C 与 D

- 完成低保真页面流程：组合总览、持仓表、添加表单、删除确认、空态、错误态。
- 建立逐条对应验收标准的人工测试清单。
- 确定演示故事线、每人讲解部分及录屏/截图备用方案。

全员完成标准

- 每个编码任务均有负责人、对应分支、验收标准和测试方案。
- API 契约与页面流程对齐。
- 只在 P0 完成后才开始 P1/P2。

Day 3：编码 - MVP 主流程

成员	当日工作
A	实现数据库、数据访问层、持仓查询/新增/删除 API 和 API 文档。
B	实现参数校验、统一错误响应、组合市值/盈亏计算及业务单元测试。
C	实现持仓列表、添加表单、删除、加载态、空态和错误态；完成 API 联调。
D	执行 API 验收，记录缺陷，维护启动说明、测试记录和演示数据。

完成标准

- 用户可以通过 API 和 UI 新增、查询和删除持仓。
- **qa** 可集成当日功能。
- 每个已完成功能均通过 PR 和一名队友的真实人工审查。

Day 4：编码 - 图表、稳定性与交付

成员	当日工作
A、B	实现当前价格或演示价格降级、边界处理，并将后端覆盖率补至 70% 以上。
C	完成一个高价值图表，优先资产分布或组合市值；完成页面回归。
D	执行全量验收，维护架构图、ADR、演示脚本、PPT 和故障备份材料。

全员完成标准

- 所有 P0 验收标准通过。
- 后端测试通过，行覆盖率不低于 70%。
- **main** 可从零启动并完成演示流程。
- 已完成至少一次完整演练，并准备录屏或截图备用。

8. Git 与 PR 规范

分支策略

分支	用途
main	可交付版本，始终可运行
qa	功能集成和回归测试
feature/<story-id>-<short-name>	单一用户故事或任务开发
bugfix/<issue-id>-<short-name>	未发布问题修复
hotfix/<issue-id>-<short-name>	已发布/演示版本紧急修复
prod	最终演示或部署版本

- 禁止直接提交到 main 或 qa。
- 每个功能或任务使用独立 feature/* 分支。
- 所有合并必须经过 PR 和至少一名队友的人工审查。
- 审查者必须阅读 diff、提出必要问题，并核对验收标准；不得只做形式化批准。
- 完整分支生命周期、main/qa/feature/hotfix/prod 创建时机、合并方向和 GitHub Actions 门禁见 [Git 分支与 GitHub 自动化流程](#)。

Git Commit 规范

提交信息使用以下统一格式：

<type>(<scope>): <简短英文祈使句描述>

scope 可选，建议使用模块名，例如 holding、portfolio、api、ui、docs 或 test。标题不超过 72 个字符，不使用句号，不写无意义信息。

Type	适用场景
feat	新增用户可见功能或 API 能力
fix	修复缺陷、异常处理或错误结果
test	新增或修改测试，不改变生产行为
docs	README、ADR、架构图、API 文档等变更
refactor	不改变外部行为的代码重构
chore	构建、依赖、配置、CI 或仓库维护
style	不影响逻辑的格式化或样式调整

合格示例：

```
feat(holding): add endpoint to create a holding
fix(portfolio): handle empty portfolio valuation
test(api): cover invalid holding quantity
docs(readme): document local startup steps
chore(ci): enforce backend coverage threshold
```

禁止示例：

```
fix
update
wip
change stuff
final version
```


提交与 PR 前检查

提交前，作者必须确认：

- [] 提交只包含一个清晰目的，不混入无关格式化或他人文件。
- [] 相关测试已运行并通过；若无法运行，已在 PR 中说明原因。
- [] 未提交密钥、令牌、本地配置、构建产物或 IDE 临时文件。
- [] 提交内容与对应 User Story 的验收标准有关。
- [] 作者能够解释本次变更的关键实现、测试覆盖和边界处理。

一个功能可以有多个小提交。合并 PR 时可选择保留提交或 squash，但 `qa` 和 `main` 的历史必须清晰、可追溯。每个 PR 描述至少包含：关联 User Story、实现摘要、测试结果、验收标准核对和已知限制。

9. 工程质量要求

- 后端必须包含单元测试，行覆盖率至少 70%。
- 测试必须覆盖异常输入、边界条件和失败路径，不能只测试 happy path。
- 修复缺陷前先通过日志、断点、测试和请求响应记录定位原因。
- 任何提交都必须由作者理解并能够解释；无法解释的实现不得合并。
- 编译通过不等于完成，负责人必须依据验收标准手动验证。
- 架构、数据模型、代码结构、类职责和方法命名由团队决定，并同步更新文档。

10. 每日协作节奏

- 每日站会 10 分钟：Yesterday / Today / Blockers。
- 每天结束前更新看板、PR 状态、测试结果、风险和阻塞项。
- 遇到超过 30 分钟无法推进的阻塞，立即在团队频道提出并记录处理结果。
- Day 4 进行简短复盘：记录一个做得好的实践、一个问题和一个后续改进项。

11. 最终交付检查清单

- [] `main` 可启动，且有完整的启动、测试和 API 使用说明。
- [] UI 可演示持仓查询、新增、删除和组合汇总。
- [] 至少一个组合表现图表可用。
- [] 后端测试通过，行覆盖率 $\geq 70\%$ 。
- [] API 文档、ERD、架构图与实际实现一致。
- [] 每个已合并 PR 均有队友人工审查。
- [] `qa`、`main`、`prod` 的分支保护和 GitHub Actions required checks 已启用。
- [] Git 提交持续、清晰，且四人贡献相对均衡。
- [] 四位成员均参与展示，并能解释自己负责和审查的代码。
- [] 已准备现场 Demo 失败时的录屏或截图备用方案。

HoldHive 成员目录分工速查

本文件用于成员拉取仓库后的第一天启动。详细职责、时间计划和 PR 规则见 [team_project_guideline_zh.md](#)；这里保留最短路径，避免成员改错目录。

成员 A：后端 API + 数据库

主要目录：

- [backend/src/main/java/com/holdhive/portfolio/api/](#)
- [backend/src/main/java/com/holdhive/portfolio/persistence/](#)
- [backend/src/main/resources/db/migration/](#)
- [scripts/mysql/](#)

优先任务：

1. 用 Flyway 建表，保证空 MySQL 可自动迁移。
2. 实现 holdings 查询、新增、删除 API，并让 `assetType`、`providerQuoteId`、`currency` 正确落库。
3. 保持 DTO 与 [docs/guideline/project/api_documentation_zh.md](#) 一致。

不要直接改：

- [portfolio/domain/](#) 中的估值公式。
- [pricing/infrastructure/](#) 中的外部行情适配器。

成员 B：后端业务计算 + 质量

主要目录：

- [backend/src/main/java/com/holdhive/portfolio/application/](#)
- [backend/src/main/java/com/holdhive/portfolio/domain/](#)
- [backend/src/main/java/com/holdhive/pricing/](#)
- [backend/src/main/java/com/holdhive/common/error/](#)
- [backend/src/test/java/com/holdhive/](#)

优先任务：

1. 实现股票、ETF、场外基金、加密资产、现金和银行存款的市值、成本、盈亏和配置占比计算。
2. 实现 `demo/market pricing adapter` 的可替换边界；现金和银行存款固定按 1.00 估值，加密资产先支持 `demo/cache price`。
3. 实现基金穿透 `demo endpoint`，用于前端展示基金底层股票提醒；真实披露源后续替换。
4. 用 JUnit、Mockito、MockMvc 把核心路径覆盖到 JaCoCo 70% 以上。

不要直接改：

- 已合并的 Flyway migration。
- 前端页面组件，除非接口字段已经变更且需要同步最小类型或联调展示。

成员 C：前端页面 + 联调

主要目录：

- [frontend/src/api/](#)
- [frontend/src/features/portfolio/](#)
- [frontend/src/components/](#)
- [frontend/src/styles/](#)

- frontend/src/test/

优先任务：

1. 通过 `src/api` 统一访问后端，不在组件里拼接 URL。
2. 实现 Dashboard、持仓表、添加表单、删除确认、状态提示和图表，并展示 `STOCK / ETF / CRYPTO / CASH` 标签和筛选。
3. 用 `fixture/mock` 先开发，再切换到真实 API 联调。

不要直接改：

- 后端数据库 migration。
- 后端业务计算公式。

成员 D：QA + CI + 交付

主要目录：

- .github/workflows/
- .github/pull_request_template.md
- docs/qa/
- docs/demo/
- docs/adr/
- docs/guideline/
- docs/design/
- .env.example

优先任务：

1. 维护 PR 模板、GitHub Actions、验收清单和缺陷记录。
2. 验证每位成员可从本机 MySQL、后端、前端完整启动。
3. 维护 PDF、蓝湖设计图和演示脚本。

不要直接改：

- API 字段、数据库字段或价格计算逻辑；需要修改时先开契约变更 PR。

共享规则

- 跨负责人目录改动必须在 PR 描述中说明原因。
- API 字段变更必须同步后端 DTO、前端 `types.ts`、API 文档和测试；`assetType` 新增或改名必须由 A/B/C 共同确认。
- 数据库变更只新增 migration，不修改已被他人 pull 的旧 migration。
- 本地启动不依赖 Docker；每人使用自己的本机 MySQL。
- `main`、`qa`、`prod` 不直接 push，全部通过 PR。

HoldHive 蓝湖协作设计稿

文件说明

- `png/`：可直接上传蓝湖的页面设计图，尺寸均为 1440 x 1024。
- `svg/`：同名可编辑矢量源稿，适合后续增量修改。
- `source/generate_holdhive_lanhu_design.py`：生成脚本，主题色、页面数据、组件均集中在代码中。
- `HoldHive_Lanhu_Design_Overview.png`：全部画板总览图。

主题与 Logo

- 日间主题使用浅色页面、蜂蜜金强调色，并配合 `FullLogoWhite.png / LogoWhite.png`。
- 夜间主题使用黑金页面、低亮度蜂巢背景，并配合 `FullLogoBlack.png / LogoBlack.png`。
- 两套主题只切换颜色变量与 logo 资源，不改变页面结构，便于前端用 CSS variables 实现。

页面清单

1. `01-gateway`：单个蜂巢六边形门户页，六个入口固定在六边形六个顶点上。
2. `02-dashboard`：组合总览，参考 Empower/Yahoo Finance 的首屏 KPI 和总览信息。
3. `03-holdings`：持仓列表和维护页，参考 Yahoo Finance/TradingView 的 holdings 视图。
4. `04-performance`：组合表现页，参考 Sharesight/TradingView 的趋势视图。
5. `05-analysis`：Hive X-Ray 分析页，参考 Morningstar X-Ray 的风险拆解方式。
6. `06-add-holding`：添加持仓表单页。
7. `07-settings`：日/夜主题、数据模式和动效设置页。
8. `08-states`：空状态、部分估值、价格失败、删除确认等友好状态页。

Gateway 动效标注

- 主入口不是列表，而是一个中心六边形：Dashboard、Holdings、Performance、Analysis、Add Holding、Settings 分别落在六个顶点。
- 默认态：顶点为描边六边形，中心点表示可点击入口。
- Hover 态：目标顶点放大到约 110%，出现蜂蜜金填充、外圈 halo 和 tooltip。
- Click 态：沿当前入口导航到对应页面；前端实现时使用 `transition: transform 160ms ease, opacity 160ms ease`。
- PNG 是静态交付图，SVG 中保留了 `:hover` 与 `hivePulse` CSS 标注，方便设计和开发理解动效。

蓝湖上传建议

- 首次上传：上传 `png/` 下全部 16 张页面图，并用页面名前缀排序。
- 标注交互：Gateway 页面使用静态 hover 示意条补充鼠标悬停、点击导航和 reduce-motion 行为。
- 增量修改：只重新上传改动页面对应的 PNG；SVG 和源脚本保留在仓库中作为可追溯源稿。
- 切图资产：优先复用 `static/img` 的四个 logo，不从设计图里二次截取低清图片。

增量修改方式

新增页面或改页面数据时，优先修改 `source/generate_holdhive_lanhu_design.py`：

- 改颜色：修改 `THEMES`。
- 改持仓示例：修改 `HOLDINGS`。
- 改图表分类：修改 `ALLOC`。

- 改单个页面：修改对应函数，例如 `dashboard()` 或 `analysis()`。

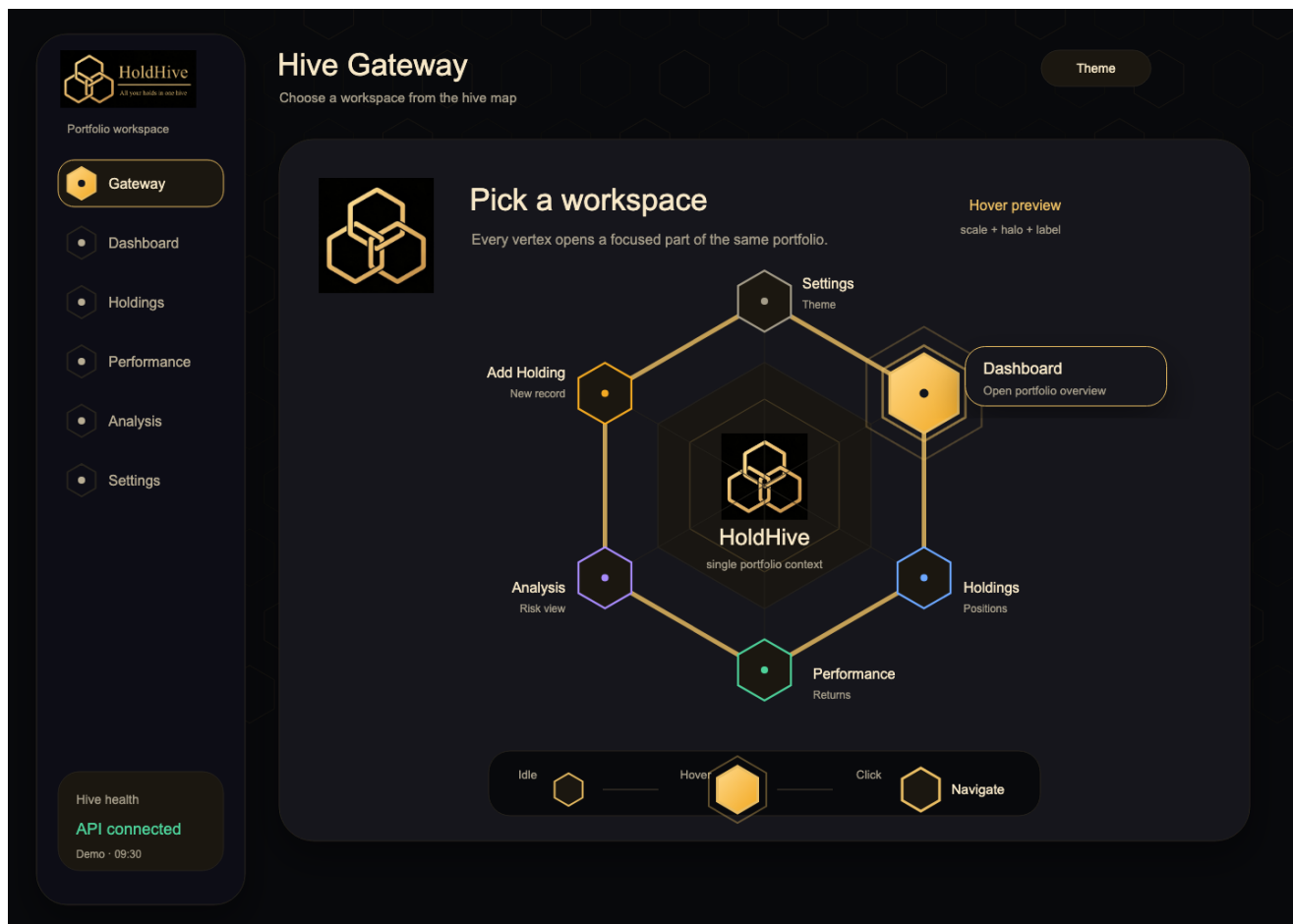
修改后运行：

```
python3 docs/design/lanhu/source/generate_holdhive_lanhu_design.py
```

脚本会覆盖生成 `svg/` 和 `png/` 中的设计稿，方便重新上传蓝湖。

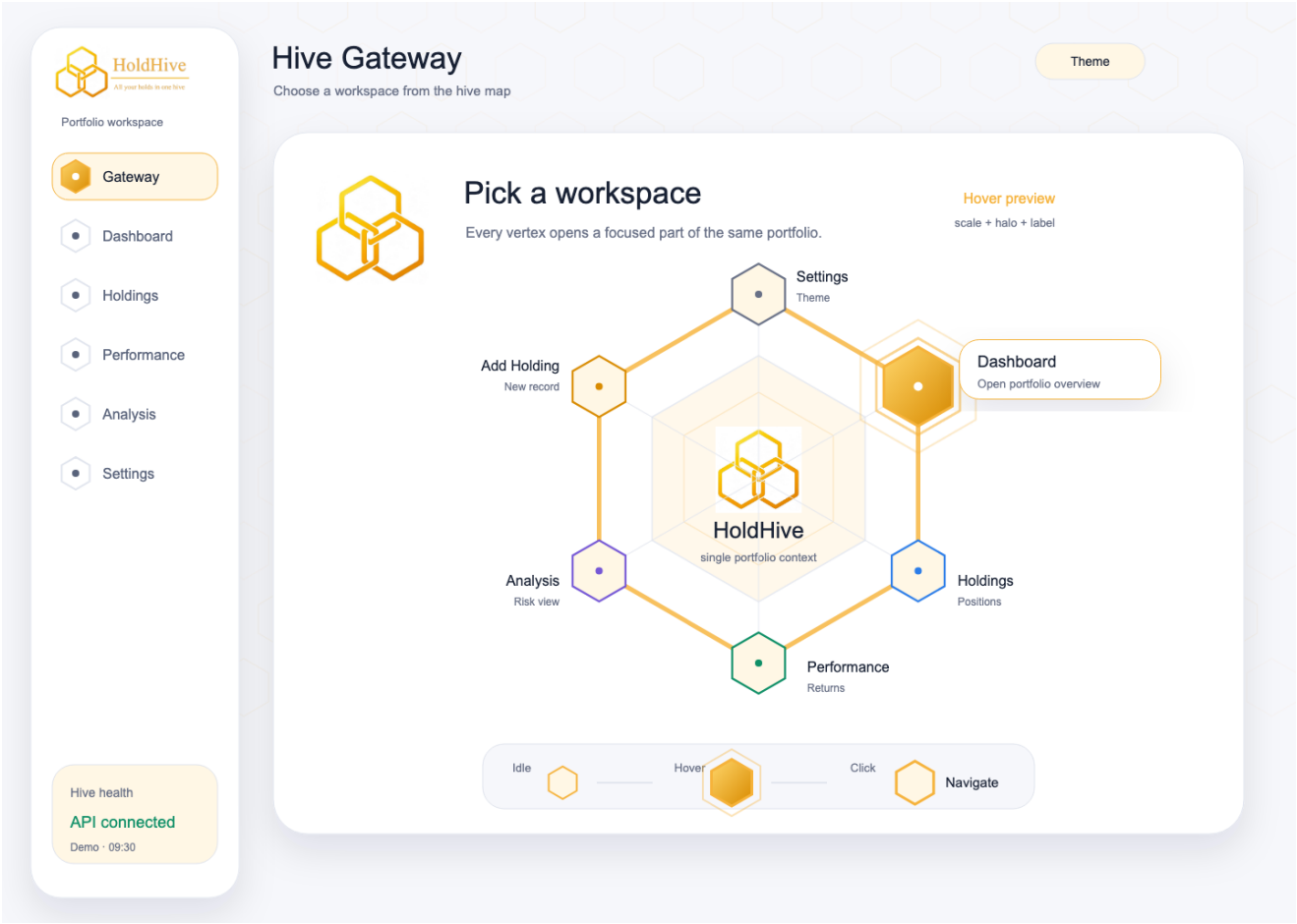
蓝湖画板 01：Gateway 夜间主题 - 单个六边形门户，六个顶点入口和 hover 示意

文件：docs/design/lanhu/png/01-gateway-dark.png。上传蓝湖时保留文件名前缀排序。



蓝湖画板 02：Gateway 日间主题 - 同结构主题适配

文件：docs/design/lanhu/png/01-gateway-light.png。上传蓝湖时保留文件名前缀排序。



蓝湖画板 03：Dashboard 夜间主题 - KPI、配置图、趋势和持仓摘要

文件：docs/design/lanhu/png/02-dashboard-dark.png。上传蓝湖时保留文件名前缀排序。



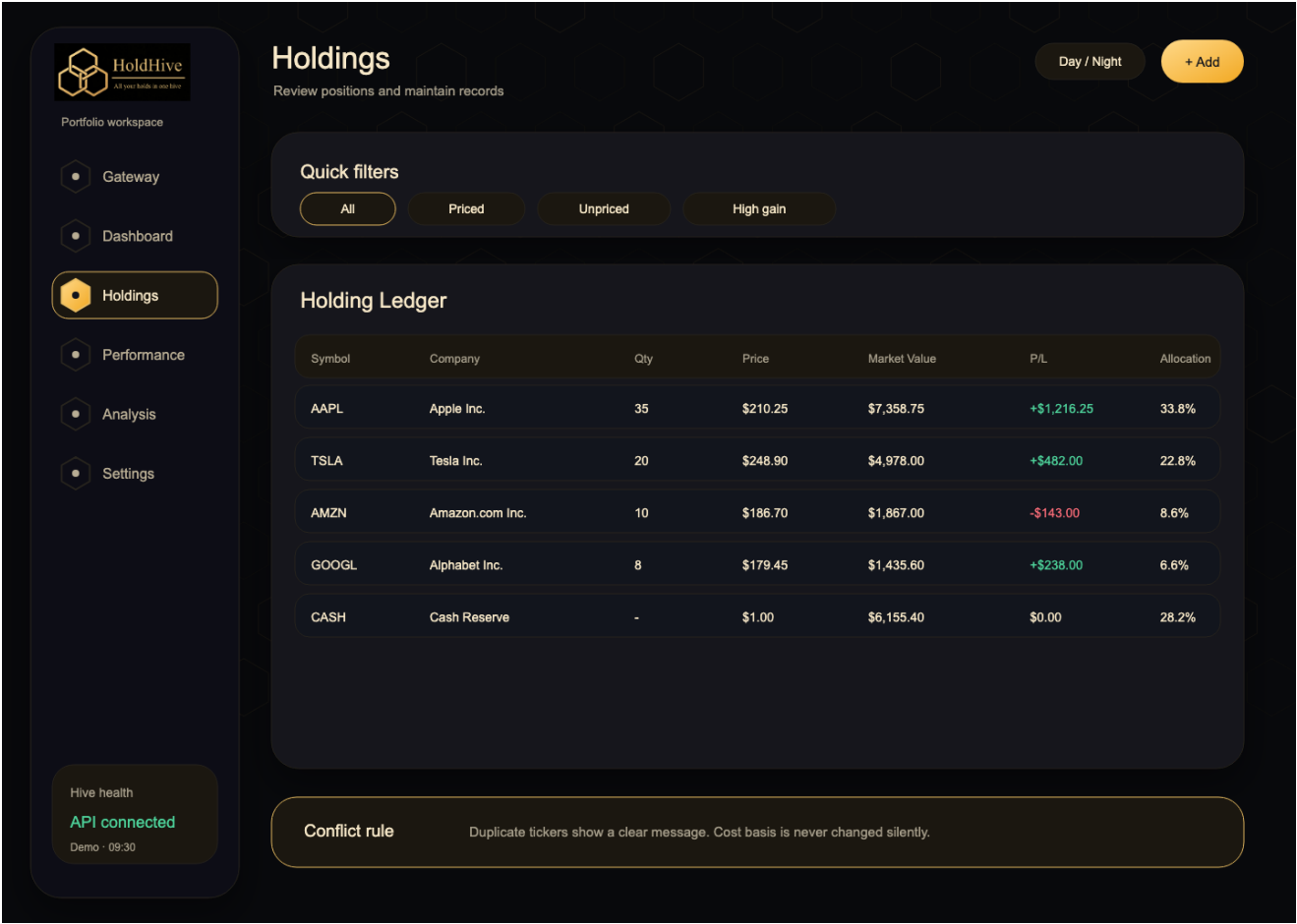
蓝湖画板 04：Dashboard 日间主题 - 首屏组合总览

文件：docs/design/lanhu/png/02-dashboard-light.png。上传蓝湖时保留文件名前缀排序。



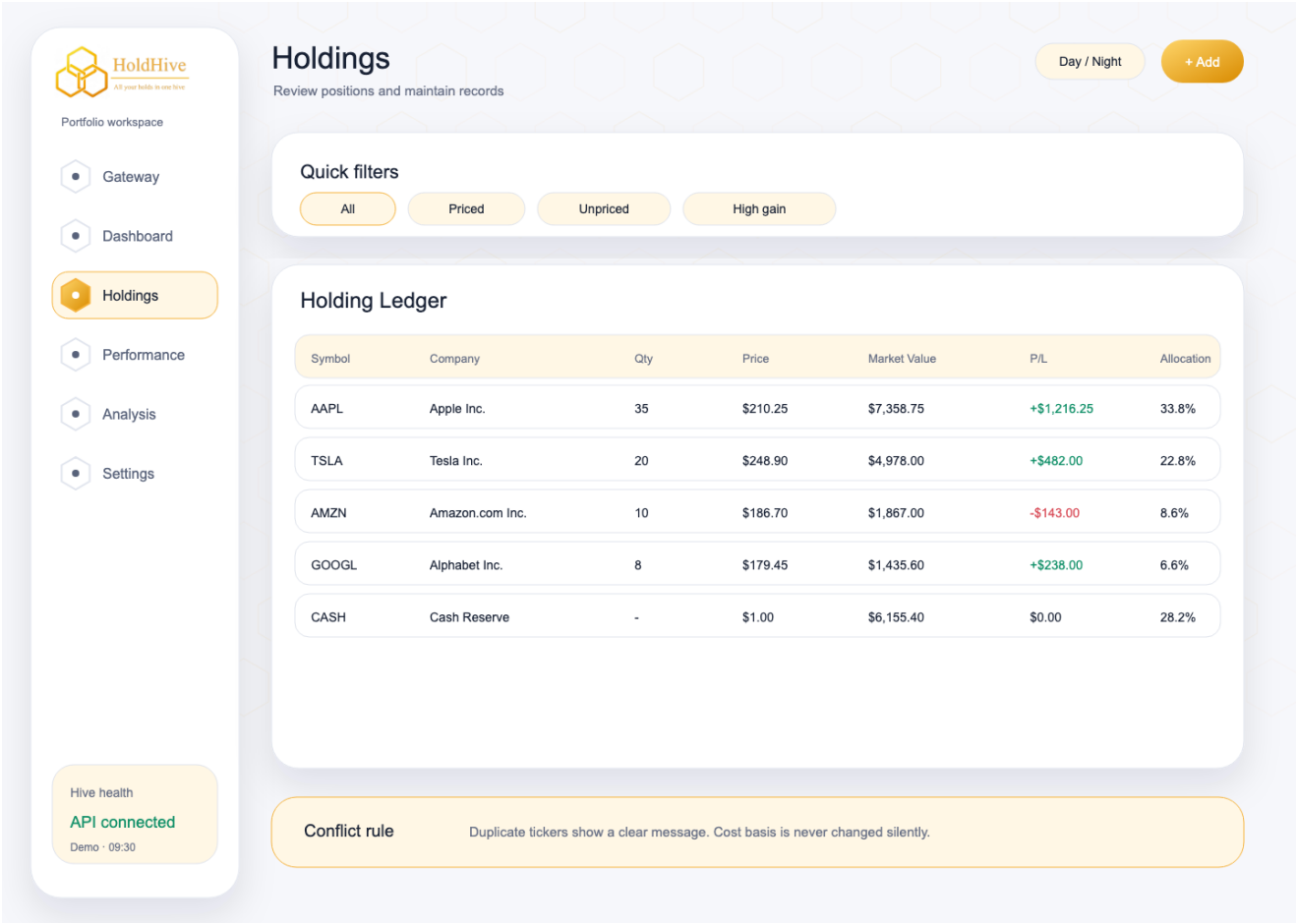
蓝湖画板 05：Holdings 夜间主题 - 持仓表和筛选

文件：docs/design/lanhu/png/03-holdings-dark.png。上传蓝湖时保留文件名前缀排序。



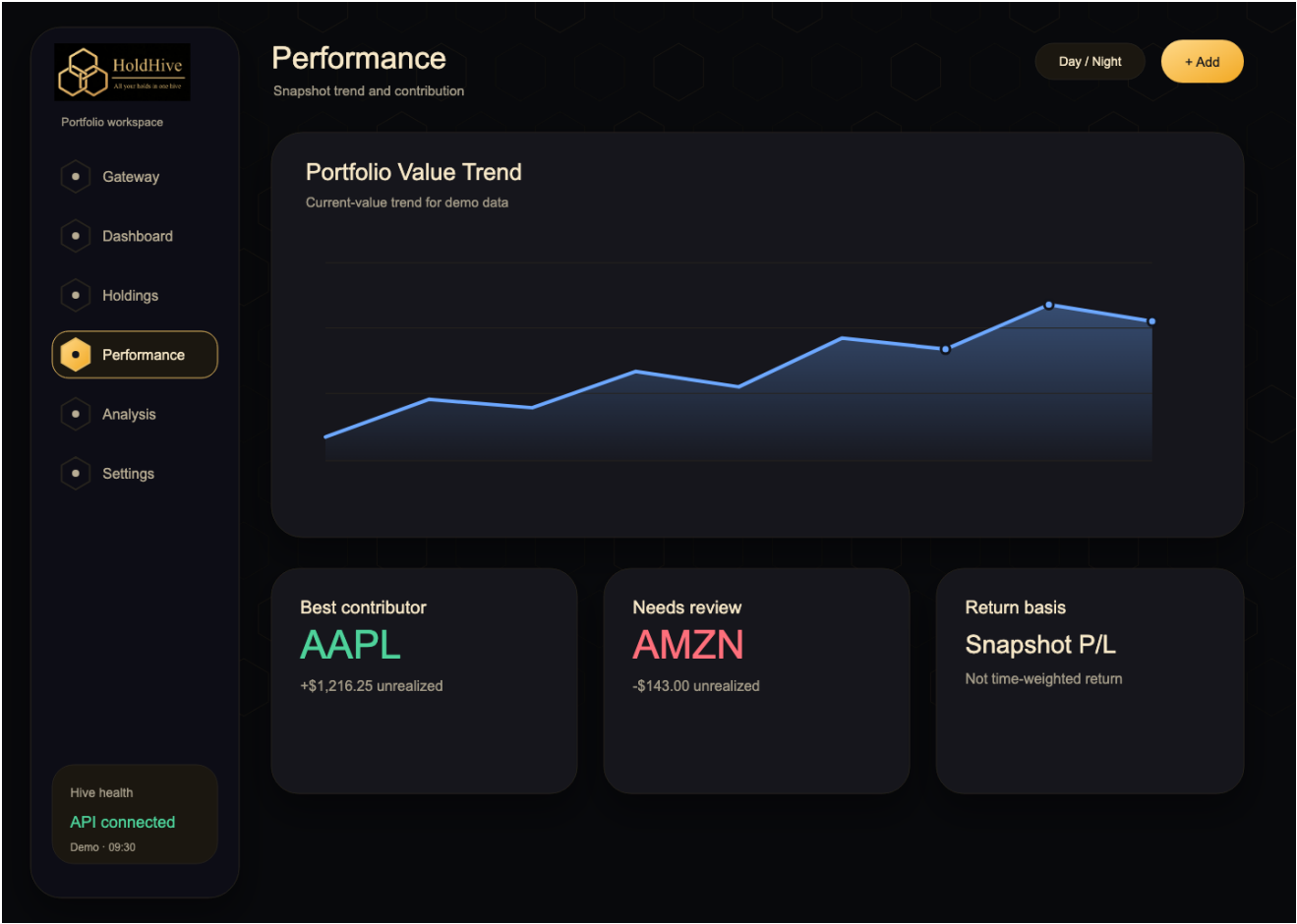
蓝湖画板 06：Holdings 日间主题 - 记录维护

文件：docs/design/lanhu/png/03-holdings-light.png。上传蓝湖时保留文件名前缀排序。



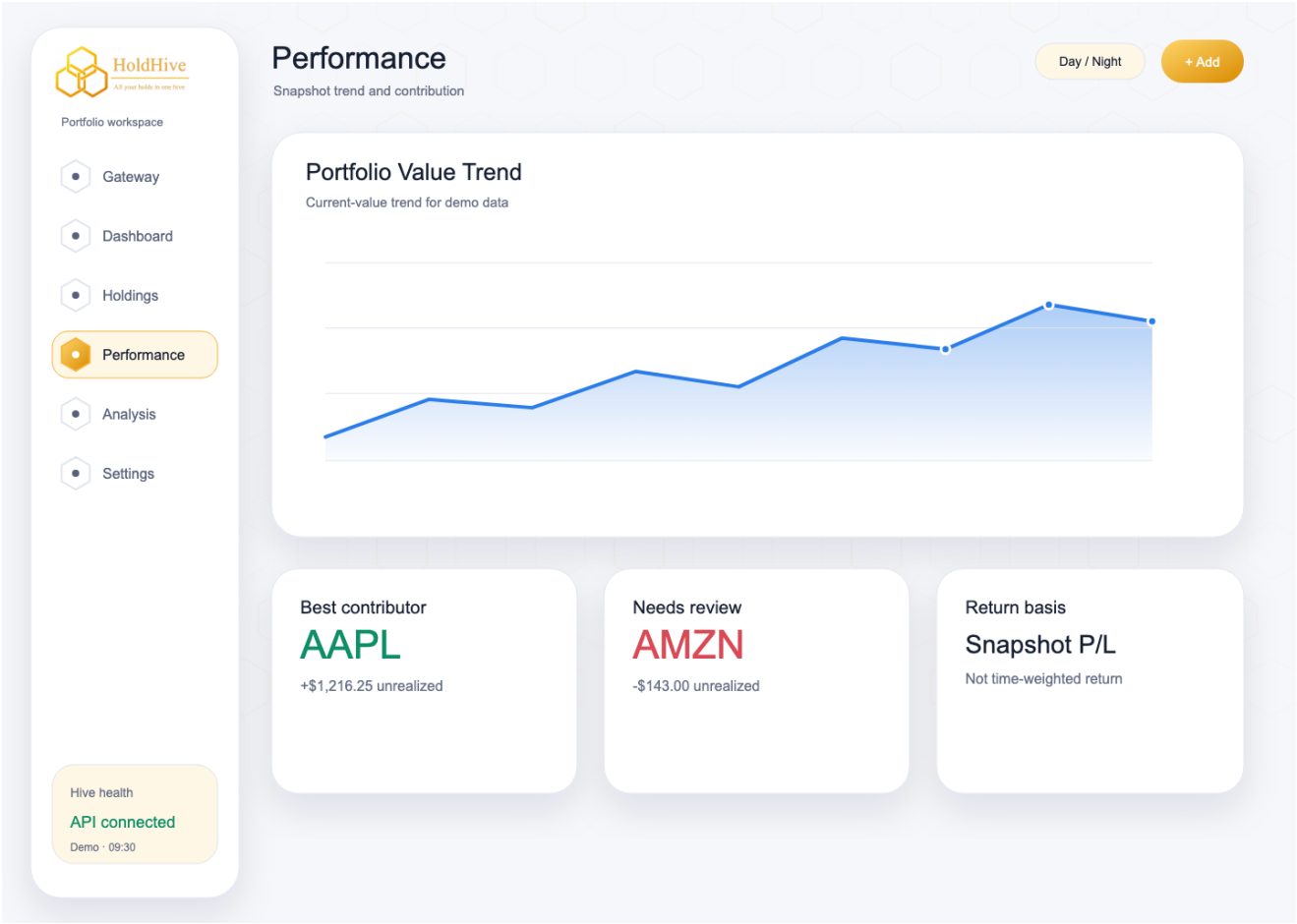
蓝湖画板 07：Performance 夜间主题 - 价值趋势和贡献提示

文件：docs/design/lanhu/png/04-performance-dark.png。上传蓝湖时保留文件名前缀排序。



蓝湖画板 08：Performance 日间主题 - 表现视图

文件：docs/design/lanhu/png/04-performance-light.png。上传蓝湖时保留文件名前缀排序。



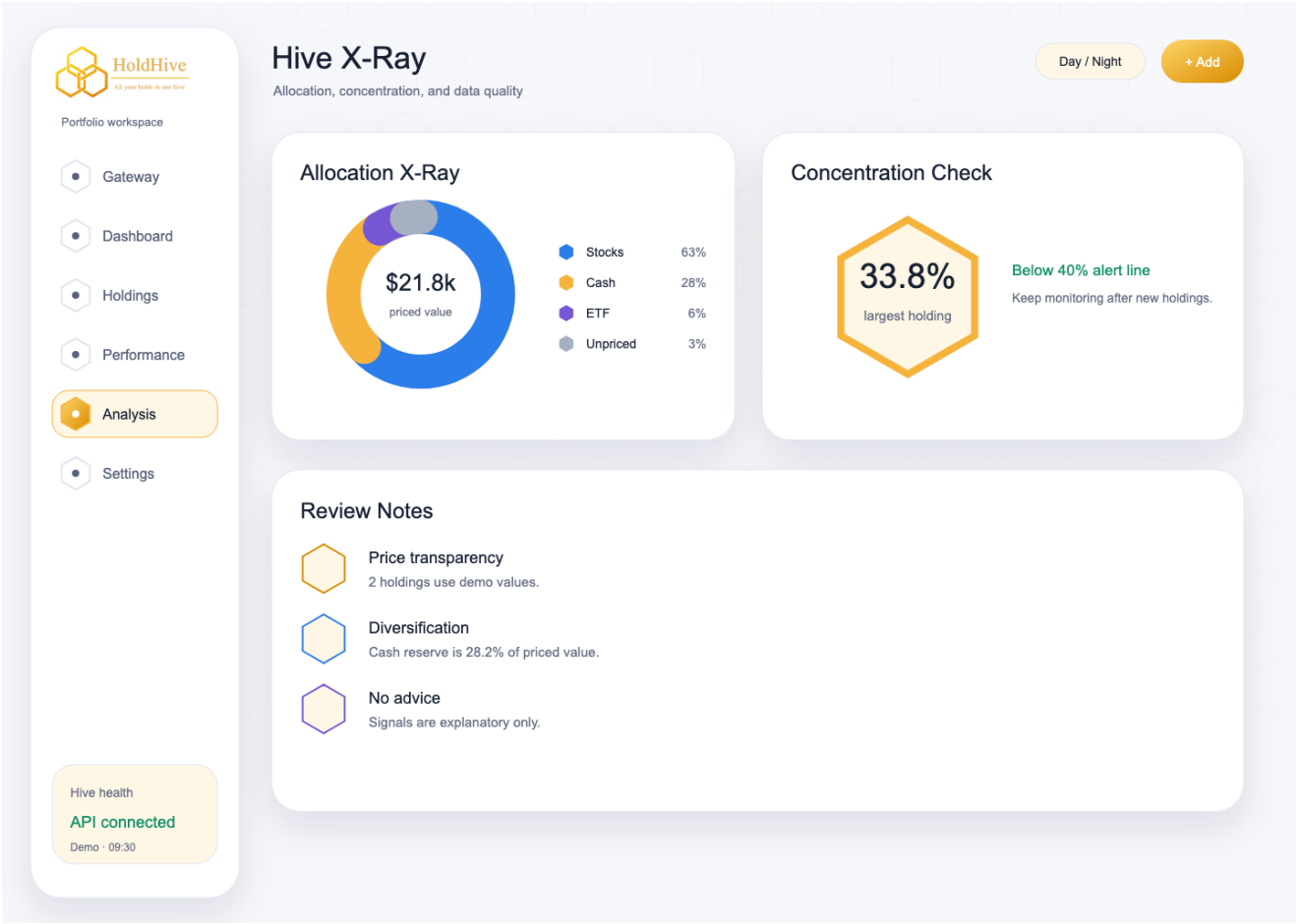
蓝湖画板 09：Analysis 夜间主题 - 配置和集中度检查

文件：docs/design/lanhu/png/05-analysis-dark.png。上传蓝湖时保留文件名前缀排序。



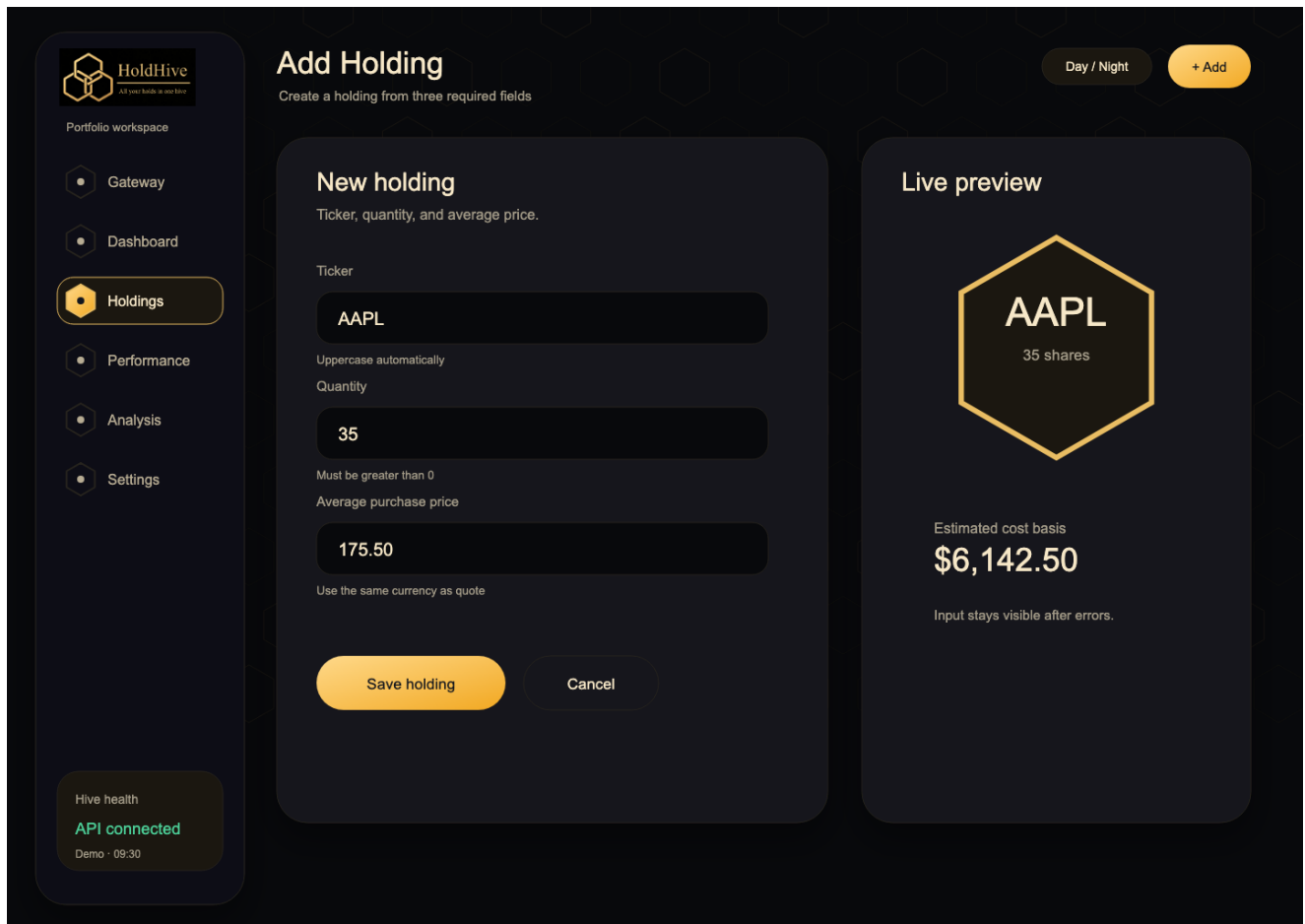
蓝湖画板 10：Analysis 日间主题 - X-Ray 分析

文件：docs/design/lanhu/png/05-analysis-light.png。上传蓝湖时保留文件名前缀排序。



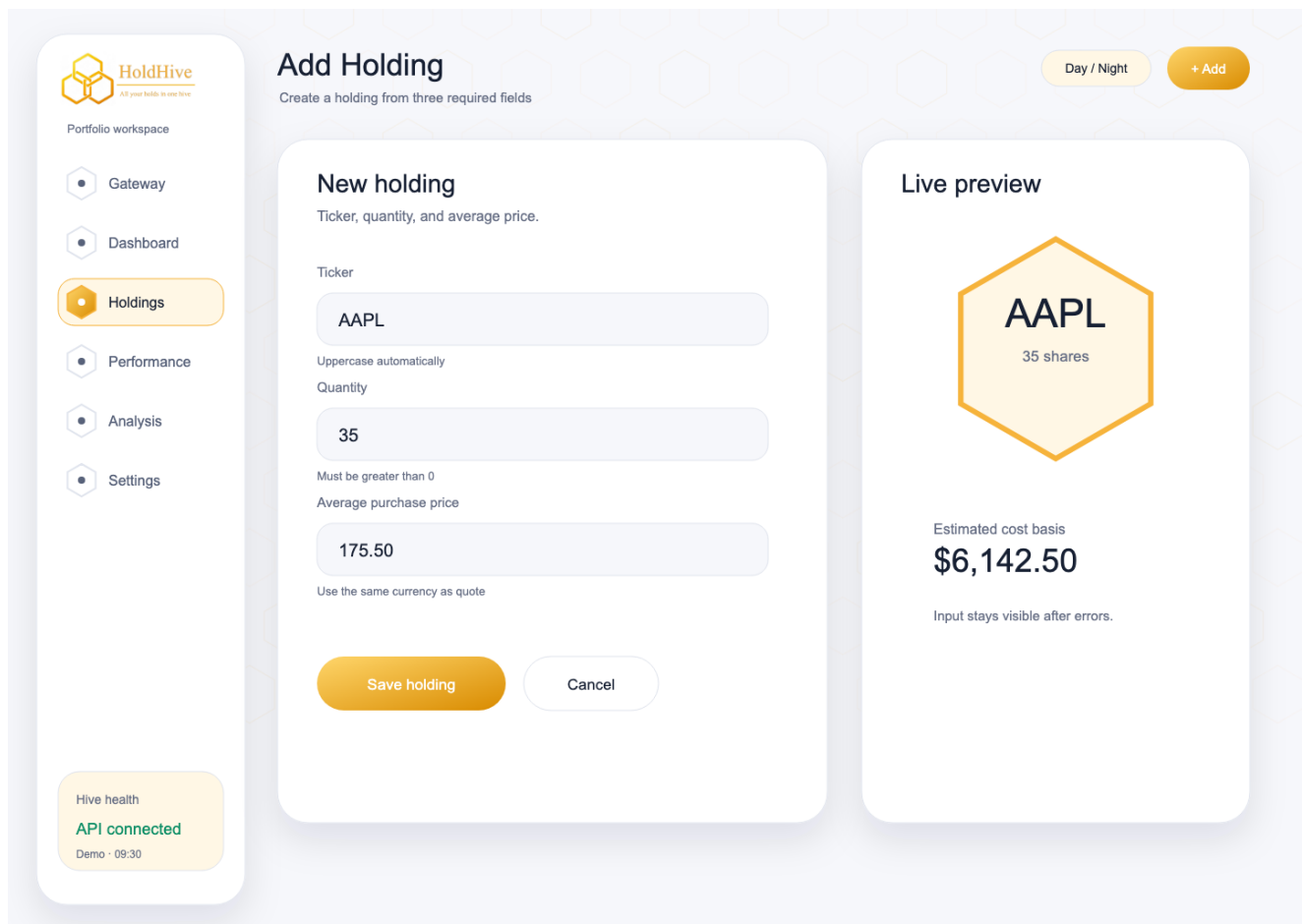
蓝湖画板 11：Add Holding 夜间主题 - 表单和实时预览

文件：docs/design/lanhu/png/06-add-holding-dark.png。上传蓝湖时保留文件名前缀排序。



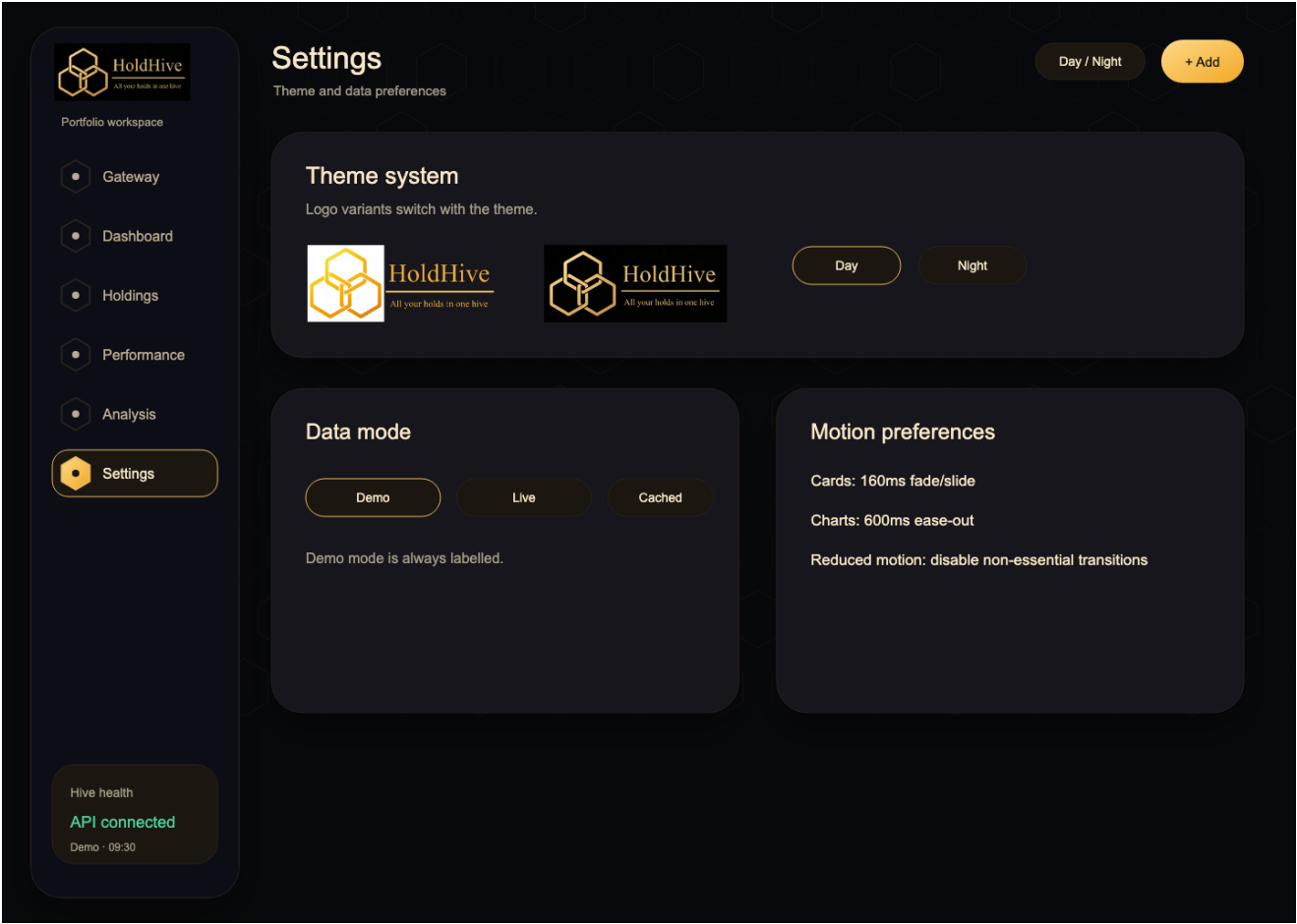
蓝湖画板 12：Add Holding 日间主题 - 新增持仓流程

文件：docs/design/lanhu/png/06-add-holding-light.png。上传蓝湖时保留文件名前缀排序。



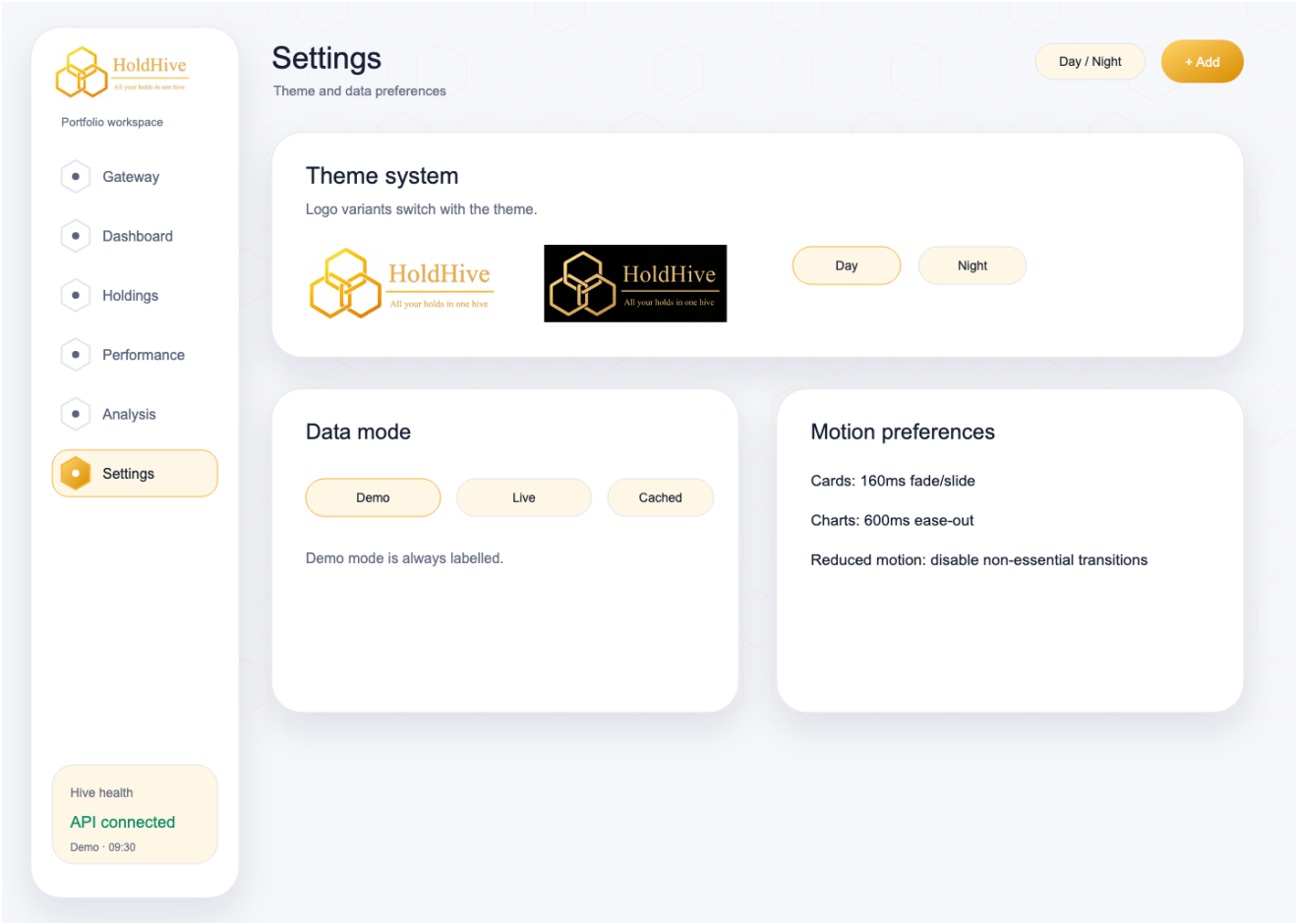
蓝湖画板 13：Settings 夜间主题 - 主题、数据模式和动效偏好

文件：docs/design/lanhu/png/07-settings-dark.png。上传蓝湖时保留文件名前缀排序。



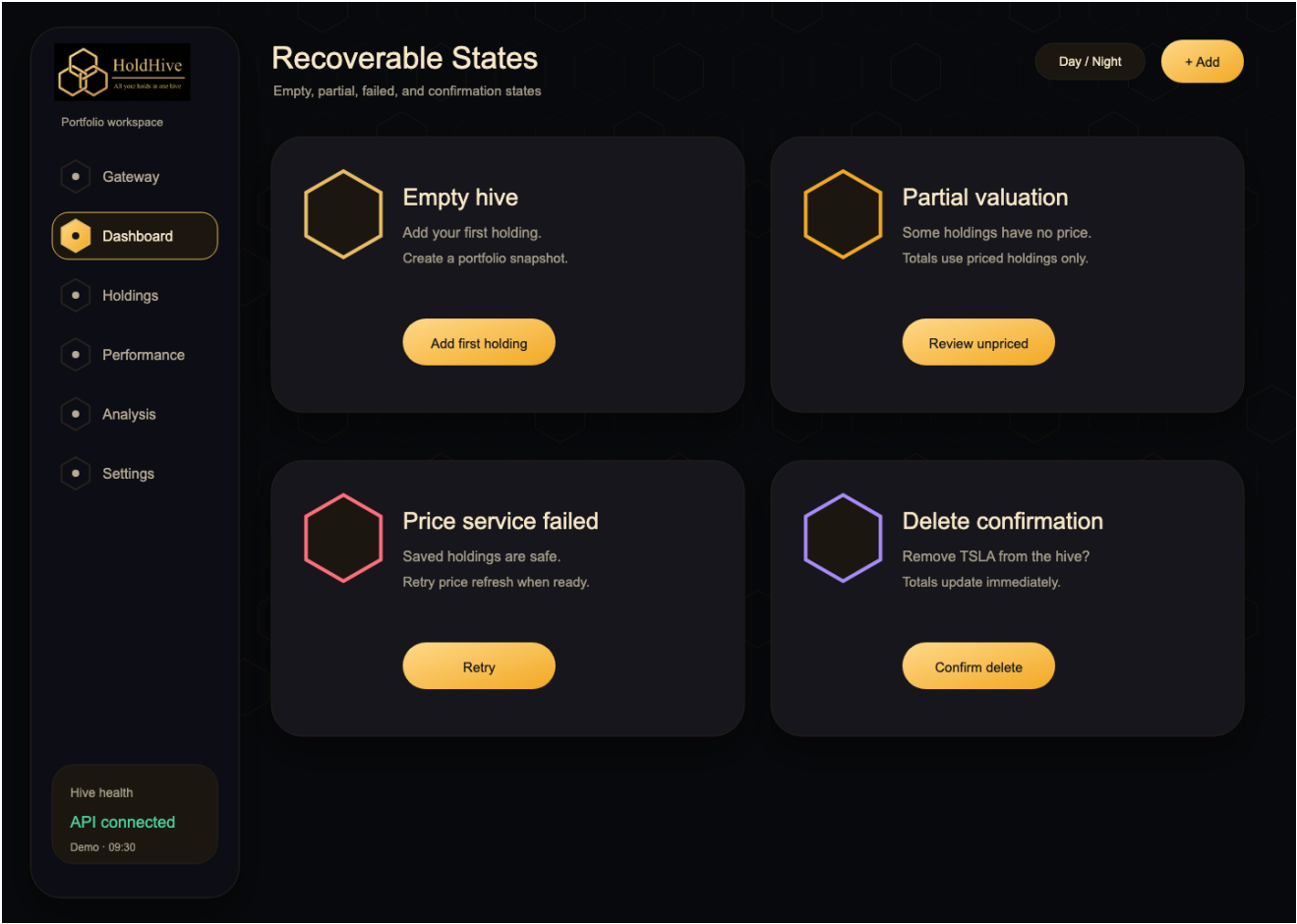
蓝湖画板 14：Settings 日间主题 - 主题切换配置

文件：docs/design/lanhu/png/07-settings-light.png。上传蓝湖时保留文件名前缀排序。



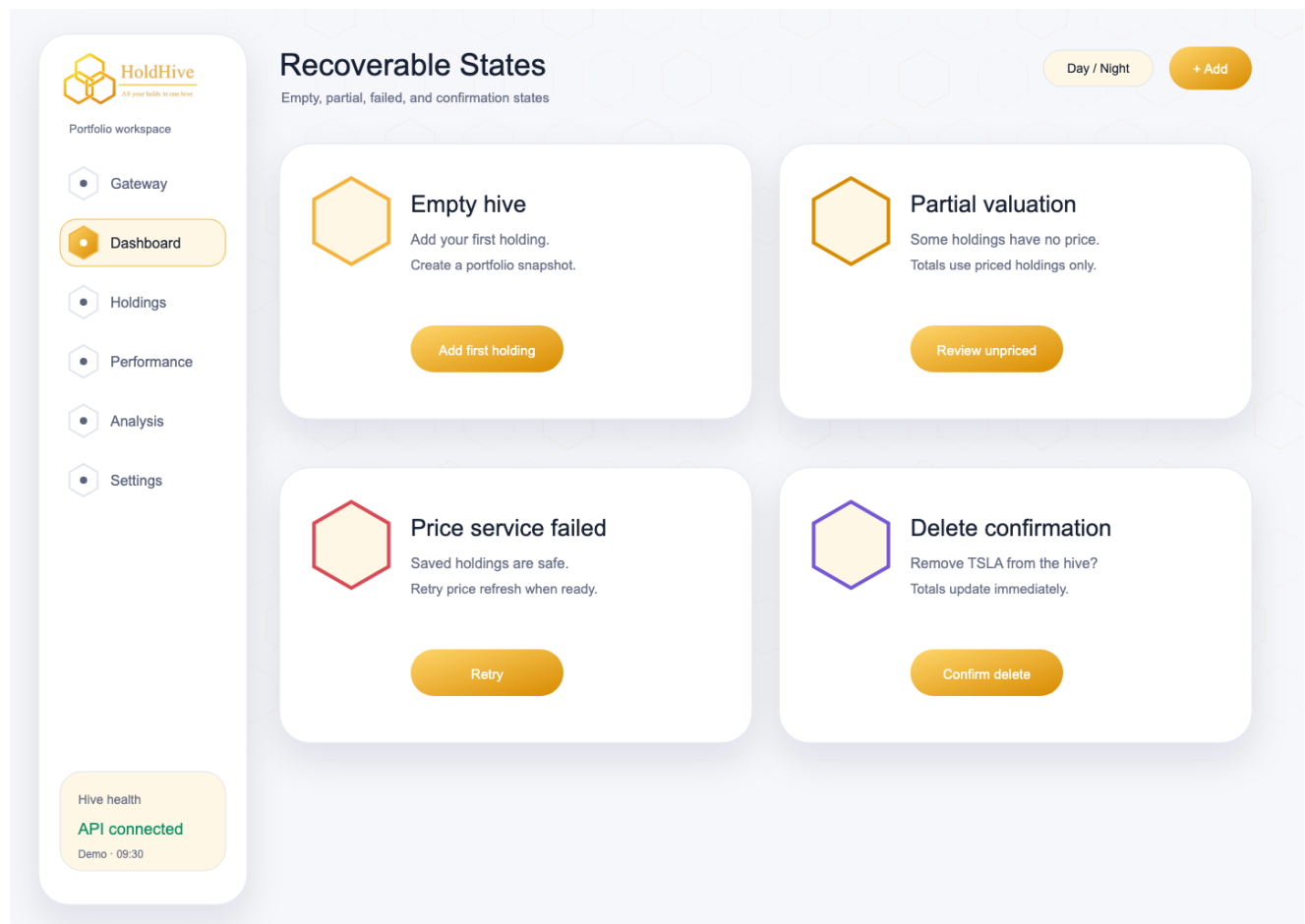
蓝湖画板 15：States 夜间主题 - 空态、部分估值、失败和确认

文件：docs/design/lanhu/png/08-states-dark.png。上传蓝湖时保留文件名前缀排序。



蓝湖画板 16：States 日间主题 - 用户友好状态

文件：docs/design/lanhu/png/08-states-light.png。上传蓝湖时保留文件名前缀排序。



HoldHive 技术栈定案

1. 决策摘要

HoldHive 采用前后端分离的模块化单体架构。目标不是追求最多技术，而是在两天编码时间内交付可运行、可测试、可解释的 MVP。

范围	技术选型	用途
前端语言	TypeScript 5.x	页面与 API 类型安全
前端框架	React 18+	Dashboard、表单和状态渲染
前端构建	Vite 5+	本地开发、构建和测试集成
前端样式	CSS Modules 或普通 CSS	控制依赖和样式作用域
图表	Recharts 2.x	资产配置图
动画	Framer Motion 或 CSS transition	页面进入、数字刷新、Toast、表格行变化
图标	Lucide React	导航、按钮、状态标签
提示反馈	React Hot Toast 或自建 Toast	添加、删除、错误和价格状态提示
数字动画	react-countup 或轻量自定义 hook	总市值、盈亏和持仓数变化
后端语言	Java 21 LTS	REST API 与业务逻辑
后端框架	Spring Boot 3.x	Web、校验、数据访问和配置
实时价格源	东方财富公共接口；CoinGecko Simple Price；Tiingo/Finnhub 为股票/ETF 备选；Binance/Coinbase 为加密资产后备	股票、ETF、加密资产和现金估值
构建工具	Maven 3.9+	依赖、测试和打包
数据库	MySQL 8.4 LTS	持仓、证券和价格快照
数据访问	Spring Data JPA / Hibernate	Repository 与实体映射
数据库迁移	Flyway	版本化建表和数据迁移
API 文档	springdoc-openapi / Swagger UI	OpenAPI 文档与调试
后端测试	JUnit 5、Mockito、AssertJ、MockMvc	单元与 Web 层测试
覆盖率	JaCoCo	后端行覆盖率门槛 70%
前端测试	Vitest、React Testing Library	组件和交互测试
E2E 冒烟测试	Playwright	最核心用户流程，时间允许时启用
本地数据库	本机 MySQL 8.x	每位成员独立启动数据库，不依赖 Docker
CI	GitHub Actions	构建、测试、覆盖率和前端检查

版本号以初始化项目当天最新的兼容稳定版本为准，并锁定在依赖文件中。不得在编码最后一天进行大版本升级。

2. 选型前置条件

项目规则要求：前端框架应至少有两名队员已经掌握；否则团队需共同学习 3-4 小时并记录过程。

因此 Day 1 必须执行一次技术能力检查：

- 若至少两人能独立使用 React、TypeScript、组件、Props、Hook 和表单事件，使用本文默认方案。
- 若不足两人具备上述能力，前端立即改为原生 HTML、CSS、JavaScript 和 Chart.js；API、后端、数据库与测试方案不变。
- 不允许在 Day 3 才因框架困难切换技术栈。

后端优先采用团队已掌握并能在两天内稳定交付的技术。如果最终后端框架不是 Java/Spring，应在 Day 1 经技术评审方确认后等价替换，并写一条 ADR 说明原因。数据库定为 MySQL；若项目环境的 MySQL 小版本不同，应验证 Flyway 迁移兼容性，而不是临时切换数据库产品。

3. 前端技术栈

3.1 核心工具

React + TypeScript + Vite

- React 负责组件化 Dashboard、持仓表、添加表单、删除确认和状态反馈。
- TypeScript 为 API DTO、表单值和组件属性提供静态检查。
- Vite 提供快速本地启动和简单构建配置。

样式：CSS Modules 或普通 CSS

本期不引入大型 UI 框架。团队建立少量 CSS 变量统一颜色、间距、字体和边框，通过原生语义 HTML 保证表单和表格可访问。

图表：Recharts

只实现一个资产配置环形图。Recharts 与 React 组件模型一致，适合直接根据 summary allocations 渲染。图例必须同时展示 STOCK、ETF、CRYPTO、CASH 的分类和金额，不让图表成为唯一的信息载体。

3.2 前端额外库建议

这些库不是必须全部引入，但它们能直接改善图表、用户提示和现代交互体验。两天编码时间内建议只选必要项，不堆依赖。

能力	推荐库	是否建议	用途
资产配置 donut	Recharts	P0 推荐	与 React 数据流一致，快速实现股票、ETF、加密资产和现金的 Pie/Donut、Legend、Tooltip
组合趋势 line/area	Recharts	P1 推荐	如果有演示历史数据，用 AreaChart 展示组合价值趋势
原生 JS 备选图表	Chart.js	备选	如果放弃 React，则用 Chart.js 实现 donut 和 line
页面/卡片过渡	Framer Motion	P1 推荐	AnimatePresence、卡片进入、表格行添加删除过渡
简单过渡	CSS transition/keyframes	P0 推荐	没时间引入 Framer Motion 时，用 CSS 完成 hover、fade、highlight
图标	lucide-react	P0 推荐	Dashboard、Holdings、Plus、Trash、Alert、Refresh 等图标
Toast	react-hot-toast	P0 推荐	添加成功、删除成功、API 失败、演示价格提示
数字变化	react-countup 或自定义 hook	P1 推荐	总市值、盈亏、持仓数刷新时更直观
精确数值格式化	Intl.NumberFormat	P0 必须	金额、百分比和币种展示；优先原生 API
日期格式化	Intl.DateTimeFormat 或 date-fns	P1 可选	价格更新时间和图表横轴；简单场景优先原生 API

推荐前端最小依赖组合：

React + TypeScript + Vite
Recharts
lucide-react
react-hot-toast
CSS Modules / plain CSS

如果 Day 3 仍未完成 P0，不再引入 Framer Motion、react-countup 或 Playwright，优先保证 CRUD、图表和联调稳定。

3.3 前端不采用的技术

- 不使用 Redux：MVP 状态范围小，使用组件状态和自定义 Hook 足够。
- 不使用 Next.js：本期不需要 SSR、SEO 或复杂路由。
- 不同时使用多个组件库或图表库。
- 不为单页 Dashboard 引入复杂路由。
- 不在前端重复实现后端的估值公式。

3.4 建议目录

```
frontend/
├── src/
│   ├── api/           # HTTP 客户端和 DTO
│   ├── components/    # 通用展示与交互组件
│   ├── features/
│   │   └── portfolio/ # Dashboard、持仓、摘要和图表
│   ├── hooks/         # 数据加载与交互 Hook
│   ├── styles/        # 全局变量和基础样式
│   ├── test/          # 测试初始化和 fixtures
│   ├── App.tsx
│   └── main.tsx
├── package.json
└── vite.config.ts
```

3.5 前端数据策略

- 使用浏览器 `fetch` 的薄封装，不额外引入 Axios。
- API 类型集中放在 `src/api`，组件不得直接拼接 URL。
- 页面加载时并行请求 `holdings` 和 `summary`，分别处理失败状态。
- 创建或删除成功后重新请求服务端数据，避免客户端自行计算权威市值。
- 表单提交期间禁用按钮，防止重复请求。

3.6 前端依赖边界

前端依赖方向必须保持单向：页面组合组件调用 feature hook，feature hook 调用 `src/api`，`src/api` 再调用后端 REST API。底层模块不得反向依赖上层页面，也不得让通用组件知道 `portfolio` 业务细节。

```
App.tsx
-> features/portfolio/PortfolioDashboard
-> features/portfolio/hooks
-> api/portfolioApi + api/types
-> backend REST API
```

```
components/
-> styles/
-> no business API calls
```

具体规则：

- `components/` 只能放可复用 UI，不直接读取 `portfolio` 数据，不调用 `fetch`。
- `features/portfolio/components/` 可以展示持仓、摘要和图表，但数据获取统一走 `features/portfolio/hooks/`。
- `features/portfolio/hooks/` 负责加载、刷新、提交和删除动作，不直接处理 DOM 或样式。
- `api/` 只负责 HTTP、DTO 和错误转换，不包含 React 组件或图表逻辑。
- `styles/` 只提供 token、主题和基础样式，不引入业务字段。
- 图表组件接收已经整理好的 `allocation` 数据，不自己计算权威市值；权威计算来自后端 `summary`。
- 新增库时优先放在使用范围最小的模块内，不因为一个页面需求把依赖扩散到全局。

4. 后端技术栈

4.1 核心工具

Java 21 + Spring Boot 3 + Maven

- Spring Web 提供 REST Controller。
- Spring Validation 处理请求字段校验。
- Spring Data JPA 提供持久化访问。
- Flyway 管理数据库版本，保证四名成员和 CI 使用同一套建表、改表和 seed 流程。
- springdoc-openapi 生成 Swagger UI。

采用模块化单体，不拆微服务。外部价格服务通过接口和适配器隔离，以便在真实数据、缓存和演示价格之间切换。实时股价接口、链接、用法、缓存和限流策略见 [实时股价 API 方案](#)。

4.2 建议包结构

```
backend/src/main/java/.../holdhive/
├── portfolio/
│   ├── api/           # Controller、请求和响应 DTO
│   ├── application/   # 用例服务和事务边界
│   ├── domain/        # 业务对象、计算和领域异常
│   └── persistence/   # JPA Entity 和 Repository
├── pricing/
│   ├── application/   # 价格查询服务
│   ├── domain/        # PriceQuote、PriceStatus
│   └── infrastructure/ # 外部 API、缓存、演示适配器
├── common/
│   ├── error/         # 统一异常响应
│   └── config/        # CORS、OpenAPI 等配置
└── HoldHiveApplication.java
```

按业务能力而非传统全局 **controller/service/repository** 大目录拆分，使 portfolio 和 pricing 可以独立理解与测试。

4.3 后端职责边界

- Controller 只负责 HTTP 映射、请求校验和响应状态。
- Application Service 负责用例编排、事务和权限边界。
- Domain 负责估值公式和业务规则，不依赖 HTTP 或数据库。
- Repository 负责持久化，不承载业务计算。
- Pricing Adapter 负责外部调用和数据转换，不修改持仓。
- 所有金额使用 **BigDecimal**，禁止使用 **double**。

4.4 后端依赖方向

后端采用模块化单体，但模块内必须保持清晰依赖方向。外层可以依赖内层，内层不能反向依赖外层；接口定义放在靠近业务调用方的位置，实现放在基础设施层。

```
api
-> application
-> domain

application
-> persistence repository interface / Spring Data repository
-> pricing application interface

pricing application
-> pricing domain
-> pricing infrastructure adapter

persistence
-> entity / database mapping
```

禁止依赖：

- `domain` 禁止 import `org.springframework.*`、`jakarta.persistence.*`、HTTP client 或外部 API SDK。
- `api` 禁止直接写估值公式或直接调用外部行情 API。
- `persistence` 禁止返回 HTTP DTO；它只返回 Entity 或 repository 查询结果。
- `pricing.infrastructure` 禁止调用 `HoldingRepository` 修改持仓；它只返回报价结果。
- `common` 禁止依赖 `portfolio` 或 `pricing` 的业务类，否则会变成隐藏的全局业务层。

推荐做法：

- 复杂计算先放在 `PortfolioCalculator` 这类纯 domain 类中，用普通单元测试覆盖。
- 外部行情通过 `PricingAdapter` 接口隔离，真实实现、演示实现和测试 stub 可替换。
- Controller response DTO 由 mapper 组装，避免把 Entity 直接暴露给前端。
- 每个 service 方法只负责一个用例，例如 create holding、delete holding、get summary，不做“万能服务”。
- 若两个模块需要共享字段，优先通过 DTO 或小型 value object 表达，不共享大型可变对象。

4.5 配置策略

使用 Spring Profile：

Profile	数据库/价格来源	用途
local	本机 MySQL + 演示或外部价格	本地开发
test	H2 MySQL compatibility mode；CI 可另接 GitHub MySQL service	自动测试
demo	MySQL + 固定演示价格	稳定演示

数据库 URL、用户名、密码和外部 API 配置通过环境变量提供。仓库只提交 `.env.example`，不得提交真实 `.env`。

5. 数据库与迁移

默认使用 MySQL 8.4 LTS 和 InnoDB。它提供事务、外键、`CHECK` 约束、`DECIMAL` 定点数和复合索引，能够支持当前持仓模型以及后续交易流水和历史价格。数据库字符集统一为 `utf8mb4`，排序规则在首次迁移中固定，避免不同环境产生字符串比较差异。

Flyway 是数据库结构的唯一版本入口。所有表、索引、约束和初始演示数据都通过 `db/migration` 下的 SQL 文件进入数据库；本地启动、CI 和 QA 环境按相同版本顺序执行 migration，避免手工建表导致环境不一致。

Flyway 迁移建议：

```
backend/src/main/resources/db/migration/
├── V1__create_portfolio_tables.sql
├── V2__create_price_snapshot_table.sql
└── V3__seed_demo_portfolio.sql
```

- `V3` 只在 `demo` 配置中执行或通过独立演示数据加载器完成。
- 已在共享环境执行的迁移文件不得修改；变更使用新版本迁移。
- 生产代码不得依赖 Hibernate 自动建表，`ddl-auto` 使用 `validate`。

详细表设计见 [数据库设计](#)。

6. 测试技术栈与分层

6.1 后端测试

测试层	工具	测试重点	是否为 MVP 必须
Domain 单元测试	JUnit 5 + AssertJ	市值、成本、盈亏、占比、除零和部分价格	必须
Service 单元测试	JUnit 5 + Mockito	创建、重复、删除、不存在、价格失败	必须
Controller 测试	MockMvc	状态码、校验、JSON、统一错误结构	必须
Repository 测试	@DataJpaTest	唯一约束、查询和映射	必须
数据库集成测试	本机 MySQL 或 GitHub Actions MySQL service	Flyway 迁移、约束和 MySQL 特有行为	时间允许；至少在合并前运行一次

Mockito 用于隔离 Service 层和适配器层依赖，例如 Repository、PricingAdapter 或 HTTP client。它不用于替代数据库约束、Flyway migration、Repository 查询和 Controller JSON 契约测试；这些仍由 @DataJpaTest、本机 MySQL 或 CI MySQL service、MockMvc 覆盖。

JaCoCo 在 Maven `verify` 阶段检查后端行覆盖率不低于 70%。覆盖率只是一条门槛，PR 仍需人工检查关键分支是否真正被验证。

推荐命令：

```
cd backend
./mvnw clean verify
```

6.2 前端测试

使用 Vitest、React Testing Library 和 @testing-library/user-event：

- 摘要组件正确展示完整、部分和空估值。
- 添加表单拒绝非法 assetType、空 ticker、非正数量和负成本。
- 现金类型显示固定估值提示，不调用外部行情。
- 加密资产显示 live/cache/demo 标签，不把缓存或演示价格伪装为实时价格。
- 提交中按钮禁用，失败后保留输入。
- 删除前显示确认，取消不调用 API，确认成功后刷新。
- 价格状态为 DEMO 或 UNAVAILABLE 时出现明确标签。
- 图表同时提供可读的文字图例。

推荐命令：

```
cd frontend
npm ci
npm run test -- --run
npm run build
```

6.3 E2E 测试

Playwright 只覆盖一条高价值烟雾流程：

打开空组合 -> 添加 AAPL -> 看到摘要和配置图 -> 删除 AAPL -> 回到空状态

若 Day 4 前 P0 尚未完成，优先修复功能和补后端测试，不引入 Playwright。团队仍需按相同步骤执行并记录人工验收。

6.4 测试数据

- 单元测试使用 fixture/builder 创建数据，避免每个测试复制大段对象。
- 测试不访问真实 Yahoo、东方财富、CoinGecko、Binance、Coinbase 或其他外部服务，价格适配器使用 stub 或 Mockito mock。

- Demo 使用虚构持仓和固定价格，确保结果可重复。
- 每个测试独立运行，不依赖执行顺序或其他测试留下的数据。

7. API 与前后端契约

后端以 OpenAPI 为契约来源，接口设计见 [REST API 文档](#)。

推荐流程：

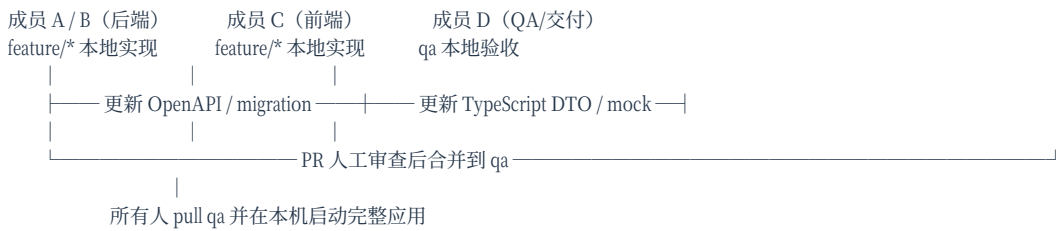
1. Day 2 冻结 MVP request/response schema。
2. 前端根据 JSON 示例定义 TypeScript 类型并使用 mock 数据开发。
3. 后端实现后通过 Swagger 和集成测试验证契约。
4. 联调发现契约变化时，先更新 API 文档和双方类型，再修改实现。

本期不引入自动代码生成，避免两天编码周期增加工具链风险。

8. 本机协作、变更同步与联调

8.1 基本原则

四名成员都在自己的机器上开发，但不共享某人的本地数据库，也不依赖某个人一直运行后端。每位成员均可从 **qa** 分支在本机启动完整系统：MySQL、Spring Boot API 和 React 前端。Git、OpenAPI、Flyway 和演示数据才是团队共享的事实来源。



8.2 代码、数据库和接口的同步边界

变化类型	唯一事实来源	作者必须同步提交的内容	接收方动作
API 字段、路径、状态码	OpenAPI 文件和 REST API 文档	OpenAPI、后端 DTO/测试、前端 mock/类型的更新或兼容说明	前端更新类型和 mock，QA 更新验收用例
数据库结构	Flyway migration	新的 V<n>_*.sql、实体映射和迁移测试	每人 pull 后重新启动本地服务，让 Flyway 自动执行
估值或校验规则	后端 domain 测试和 API schema	规则说明、单元测试、错误码或响应字段	前端只展示服务端结果，不复制计算公式
前端页面需求	User Story、验收标准和 API mock	组件测试、状态说明、所需接口字段	后端确认接口能满足展示需求

所有破坏性变化必须先创建或更新 User Story，再提出“契约变更 PR”。两天项目中优先采用向后兼容方式，例如新增可选字段；不得悄悄重命名、删除或改变已被前端使用字段的语义。

8.3 每日同步节奏

时间点	时长	参与者	输出
上午开始	10 分钟	全员	昨日变更、当天接口依赖、阻塞项
开发前	5 分钟	A、B、C	检查 OpenAPI、示例 JSON、错误码是否一致
中午	15 分钟	A、B、C、D	后端展示已完成 endpoint；前端确认 mock 能否替换为真实 API
下午集成窗口	30-45 分钟	全员	合并已审查 PR 至 qa，每人 pull 后本机跑完整流程
收工前	10 分钟	全员	记录未完成接口、已知缺陷、迁移版本和次日计划

qa 只接收已经通过本地测试和人工审查的 PR。若 qa 红灯，停止合并新功能，先恢复可运行状态。

8.4 前端无等待开发

前端不需要等待后端完成才开始：

- Day 2 先冻结 GET /holdings、POST /holdings、PATCH /holdings/{id}、DELETE /holdings/{id} 和 GET /portfolio/summary 的示例 JSON。
- 前端依据相同 TypeScript DTO 使用 fixture/mock 数据完成 Dashboard、表单、空态和错误态。
- API 适配层集中在 src/api，组件不直接调用 fetch 或硬编码 URL。
- 后端 endpoint 合并到 qa 后，前端只替换 API 适配层的 mock 实现为真实调用，组件不应重写。
- 每次契约变更均由前端在本机用真实 API 验证一次，并由 QA 记录结果。

建议通过环境变量切换数据源：

```
# frontend/.env.development
VITE_API_MODE=live
VITE_API_BASE_URL=/api/v1

# frontend/.env.mock
VITE_API_MODE=mock
```

Vite 开发服务器将 /api 代理到本机 <http://localhost:8080>，避免 CORS 干扰本地调试。不要将某位成员的局域网 IP、个人设备或临时隧道写入默认配置。

8.5 每位成员的本机启动方式

角色	日常启动	联调时额外动作
后端 A/B	首次执行 <code>mysql -u root -p < scripts/mysql/init-local-mysql.sql</code> ，再运行 Spring Boot	<code>./mvnw clean verify</code> ；通过 Swagger 或 curl 展示真实响应
前端 C	<code>npm run dev</code> ，默认 mock 模式	pull qa 后切换 <code>VITE_API_MODE=live</code> ，本机启动后端和 MySQL
QA/交付 D	pull qa 后启动三项服务	按验收清单执行浏览器流程和 API 请求，记录实际响应

前端 C 和 QA D 需要能够启动后端，即使他们不修改 Java。为降低门槛，后端 A/B 负责维护一条复制即可执行的启动命令和固定 demo 数据；这比远程访问他人电脑更可靠，也更符合可复现演示要求。

8.6 调试流程

- 在浏览器 DevTools 的 Network 面板记录请求 URL、方法、状态码、请求体和响应体。
- 前端将可复现步骤、时间和失败响应发到任务卡；不要只写“接口坏了”。
- 后端在统一错误响应中返回 `traceId`，并用该 ID 在本机日志中定位对应请求。
- 后端先用断点、日志和单元测试确认问题，再修复；必要时补一个回归测试。

5. 修复进入 `qa` 后，前端 C 与 QA D 都在自己机器上复测相同步骤，并在任务卡标记结果。

示例缺陷记录：

```
HH-02 创建持仓失败
环境：qa, commit abc1234, VITE_API_MODE=live
步骤：提交 ticker=AAPL、quantity=0、averagePurchasePrice=175.50
预期：400 VALIDATION_FAILED, fieldErrors 指向 quantity
实际：500 INTERNAL_ERROR, traceId=8f0c2d15d1e44dc8
```

8.7 联调完成标准

- [] `qa` 的 OpenAPI、前端 DTO、后端 DTO 和 API 文档字段一致。
- [] 每人可在本机从零启动 MySQL、后端和前端，且不依赖他人电脑。
- [] 新的 Flyway migration 可在空 MySQL 数据库成功执行。
- [] mock 模式和真实 API 模式覆盖相同的关键页面状态。
- [] 创建、查询、修改、删除和部分价格失败在 `qa` 上均由前端 C 与 QA D 验证。
- [] 每个已修复缺陷都有复现步骤、回归测试或明确的人工验收记录。

9. 本地开发与 CI

推荐仓库结构：

```
HoldHive/
├── backend/
├── frontend/
├── docs/
│   ├── guideline/
│   └── design/
├── scripts/mysql/
└── README.md
```

本地数据库由每位成员机器上的 MySQL 提供；前后端仍可通过 IDE 或命令直接启动，以提高调试效率。CI 如需真实 MySQL，可使用 GitHub Actions 的 MySQL service container，不要求成员本机安装 Docker。

GitHub Actions 对每个 PR 执行：

1. 后端 `./mvnw clean verify`。
2. JaCoCo 70% 行覆盖率门槛。
3. 前端 `npm ci`、测试和生产构建。
4. 检查 Flyway migration 能在空数据库执行。

CI 失败的 PR 不得合并到 `qa`、`main` 或 `prod`。完整分支策略、required checks、PR 门禁和可执行 GitHub Actions YAML 模板见 [Git 分支与 GitHub 自动化流程](#)。

10. 明确不采用

- 微服务、消息队列、Kubernetes：当前规模没有收益。
- Redis：MVP 价格缓存可以先存 MySQL 或内存。
- GraphQL：REST 已满足固定页面需求。
- Redux：状态规模不足以证明必要性。
- Tailwind 与大型组件库并用：避免样式系统重复。
- 真实券商 OAuth、用户认证和云部署：超出 MVP 范围。
- 直接调用未经验证的市场数据源：Demo 必须有稳定降级方案。

11. Day 1 技术栈确认清单

- [] 至少两名成员可以使用 React + TypeScript；否则切换原生 JavaScript 方案。

- [] 技术评审方已确认前后端框架和 MySQL 符合项目要求。
- [] 四人可以运行 Java 21、Node.js 和 MySQL；Maven 通过 `backend/mvnw` 下载。
- [] 前后端骨架、健康检查和数据库连接已经打通。
- [] OpenAPI 示例和前端 mock 使用相同字段。
- [] `local`、`test`、`demo` 环境边界清晰。
- [] 测试命令和 JaCoCo 70% 门槛已在 CI 中验证。
- [] 技术栈及替代方案已经记录为 ADR，并由团队共同确认。

HoldHive Git 分支与 GitHub 自动化流程

1. 决策摘要

HoldHive 采用“轻量 GitFlow + QA 集成门禁”。核心是 `qa` 作为团队集成分支，`main` 作为稳定交付快照。若后续有真实部署环境，可再启用 `prod`；当前 1.0.0 交付采用 `qa -> main -> tag`，不强制维护 `prod`。

分支流向固定为：



核心原则：

- `main`、`qa` 是受保护分支，不允许直接 push；如果启用 `prod`，也必须受保护。
- `feature/*` 只从 `qa` 创建，只通过 PR 合并回 `qa`。
- `docs/*` 也从 `qa` 创建，只通过 PR 合并回 `qa`，避免文档直接改动 `main` 的发布快照。
- `main` 代表稳定可交付版本；1.0.0 这类 tag 代表不可移动的发布快照。
- `hotfix/*` 只用于 `main` 或 `prod` 已发布后出现必须立即修复的问题。
- 发布后必须打 tag；本项目当前使用无 v 前缀的语义化版本，例如 1.0.0。
- 所有 PR 必须通过 GitHub Actions；CI 失败不得合并。

2. 分支何时创建

分支	创建时间	创建来源	创建人	是否长期存在	用途
<code>main</code>	仓库初始化时已有	初始仓库	仓库 owner	是	稳定可交付版本，最终从这里做 release candidate
<code>qa</code>	Day 1 规划结束前	<code>main</code>	成员 D 或组长	是	团队集成、联调、QA 验收
<code>feature/<story-id>-<short-name></code>	每个 User Story 开始编码前	<code>qa</code>	对应开发者	否	单一功能开发，例如 <code>feature/HH-02-add-holdings</code>
<code>bugfix/<issue-id>-<short-name></code>	<code>qa</code> 或 <code>main</code> 验收发现普通 bug 时	目标 bug 所在分支	修复人	否	未发布问题修复，不直接碰 <code>prod</code>
<code>hotfix/<issue-id>-<short-name></code>	<code>main</code> 已发布后出现阻断演示/部署的问题	<code>main</code>	最熟悉问题的人	否	已发布版本紧急修复
<code>docs/<topic></code>	文档或设计稿单独修改时	<code>qa</code>	文档负责人	否	文档、API、ERD、设计稿等非代码变更
<code>prod</code>	只有存在真实部署/演示环境隔离需求时	<code>main</code>	成员 D 或组长	可选	已部署环境快照，不承接日常开发

当前项目没有单独部署环境，因此使用 `main` + tag 替代 `prod`：`main` 保持稳定，1.0.0 固定指向正式交付 commit。后续普通文档更新不要移动 tag，应通过新 PR 进入 `qa`。

3. 初始化命令

Day 1 完成仓库骨架后创建 `qa`：

```
git switch main
git pull origin main
git switch -c qa
git push -u origin qa
```


如果团队决定启用 `prod`，Day 4 验收通过后创建 `prod`：

```
git switch main
git pull origin main
git switch -c prod
git push -u origin prod
git tag -a 1.0.0 -m "HoldHive MVP demo release"
git push origin 1.0.0
```

不要提前创建大量空分支。`feature/*`、`bugfix/*`、`docs/*` 和 `hotfix/*` 都应在具体任务开始时创建，任务合并后删除。

4. 日常使用流程

4.1 功能开发

```
git switch qa
git pull origin qa
git switch -c feature/HH-02-add-holding

# coding + local tests
git add backend frontend docs
git commit -m "feat(holding): add holding creation flow"
git push -u origin feature/HH-02-add-holding
```

然后在 GitHub 上创建 PR：

```
base: qa
compare: feature/HH-02-add-holding
```

合并要求：

- 至少 1 名队友 review。
- `backend`、`backend mysql smoke`、`frontend`、`docs` 通过。
- PR 描述包含 Story ID、实现摘要、测试结果、验收标准和已知限制。
- 合并方式使用 squash merge，保持 `qa` 历史简洁。

4.2 QA 集成到 main

当 `qa` 完成一批功能并通过人工验收时，创建 release PR：

```
base: main
compare: qa
```

合并要求：

- 成员 D 完成验收清单。
- GitHub Actions 中 `backend`、`backend mysql smoke`、`frontend`、`docs` 全部通过。
- API 文档、OpenAPI、数据库 migration 与实现一致。
- 合并方式使用 merge commit，保留一次清晰的 release 节点。

`qa -> main` 不是每天随手合并。建议 Day 3 晚上做一次候选合并，Day 4 最终演示前做一次正式合并。`1.0.0` 已按该流程由 PR #36 合入 `main` 后打 tag。

发布 tag：

```
git switch main
git pull --ff-only origin main
git tag -a 1.0.0 -m "Release 1.0.0"
git push origin 1.0.0
```

tag 是发布证据，不随普通文档或小修订移动。如果需要发布补丁，使用新 tag，例如 `1.0.1`。

4.3 main 发布到 prod

`main` 稳定后创建 PR：

```
base: prod
compare: main
```

合并要求：

- 所有 CI 通过。
- QA/交付负责人确认演示脚本通过。
- 不允许包含未验收的 P1/P2。
- 合并后打 tag。

如果启用 **prod**，推荐 tag：

```
git switch prod
git pull origin prod
git tag -a 1.0.0 -m "HoldHive MVP demo release"
git push origin 1.0.0
```

4.4 Hotfix 流程

只有 **main** 或可选的 **prod** 已发布后才使用 **hotfix/***。当前项目未启用 **prod** 时，从 **main** 创建 hotfix：

```
git switch main
git pull origin main
git switch -c hotfix/HH-99-fix-demo-startup

# fix + tests
git commit -m "fix(startup): restore demo database migration"
git push -u origin hotfix/HH-99-fix-demo-startup
```

合并顺序：

1. PR **hotfix/*** -> **main**，只包含紧急修复。
2. 合并后打补丁 tag，例如 **1.0.1**。
3. 将同一修复 back-merge 或 cherry-pick 到 **qa**，避免后续功能覆盖 hotfix。
4. 如果启用了 **prod**，再同步到 **prod**。

如果问题尚未发布到 **main** 或 **prod**，不要使用 **hotfix/***；从 **qa** 创建 **bugfix/*** 即可。

5. 分支保护规则

GitHub 仓库设置中为 **qa**、**main** 和 **prod** 分别配置 branch protection 或 ruleset。官方文档说明，受保护分支可以要求 status checks、限制删除和 force push，并要求 PR review 后才能合并。

分支	必须开启	推荐规则
qa	Pull request、status checks	1 个 approval；禁止 force push；要求 backend、backend mysql smoke、frontend、docs
main	Pull request、status checks、清晰历史	1 个 approval；禁止直接 push；要求 backend、backend mysql smoke、frontend、docs
prod	Pull request、status checks、tag/release gate	1 个 approval；只允许从 main 或 hotfix/* 合并；禁止删除；发布后打 tag

注意：

- 每个 required check 的 job name 必须唯一，避免 GitHub 无法判断哪个 check 应作为门禁。
- 不允许管理员绕过保护规则，除非项目负责人明确要求紧急演示。
- 不使用 force push 修复共享分支；错误提交通过 revert 或新 PR 修复。

6. Merge 策略

合并方向	合并方式	原因
feature/* -> qa	Squash merge	一个 Story 对应一个清晰集成提交，便于回滚
docs/* -> qa	Squash merge	文档变更保持清晰
bugfix/* -> qa/main	Squash merge	普通修复独立可追溯
qa -> main	Merge commit	保留一次 release candidate 边界
main -> prod	Merge commit 或 fast-forward	代表正式发布节点
hotfix/* -> prod	Squash 或 merge commit	视修复复杂度；必须 back-merge/cherry-pick 到 main 和 qa

禁止：

- feature/* 直接合并到 main 或 prod。
- 从 prod 反向开发新功能。
- 把多个无关 Story 合并在同一个 feature PR。
- 为了“赶时间”跳过 PR 和 CI。

7. GitHub Actions 自动化设计

7.1 工作流文件

当前仓库已有主工作流：

```
.github/workflows/
└── pr-check.yml # PR/push 门禁：后端、MySQL smoke、前端、文档检查
```

后续如有部署环境，再新增 release.yml 或环境审批 workflow；当前 1.0.0 以 pr-check.yml 作为发布质量门禁。

7.2 PR 门禁

触发条件：

```
on:
  pull_request:
    branches: [qa, main]
```

检查内容：

Job	技术栈	命令
backend	Java 21、Spring Boot、Maven、Flyway、JaCoCo	./mvnw -B verify
backend mysql smoke	MySQL 8.4 service container + Spring Boot local profile	package 后启动后端，访问 health、market search、create cash holding、summary
frontend	React、TypeScript、Vite、Vitest	npm ci、npm run test -- --run、npm run build
docs	文档存在性检查	检查核心 guideline/design 文档存在

7.3 QA 集成

当前 pr-check.yml 已覆盖 push 到 qa：

```
on:
  push:
    branches: [qa]
```

检查内容：

- 后端执行 ./mvnw -B verify。

- `backend mysql smoke` 连接 MySQL service container, 执行 Flyway migration 和基础 API smoke。
- 前端执行 test + build。
- 文档 job 检查核心文档仍存在。
- 如果后续有时间, 可追加 Playwright 冒烟: Gateway -> Add Holding -> Dashboard -> Analysis。

7.4 Release 验证

触发条件：

```
on:
  push:
    branches: [main]
    tags: ["*.*.*"]
```

检查内容：

- 完整后端 verify。
- 前端生产 build。
- 本地 MySQL 环境模板、Flyway migration 和 profile 配置检查。
- 上传后端 jar、前端 dist、测试报告和覆盖率报告。
- 如果后续新增 `prod` 或专门 release workflow, 再设置 `environment: production-demo`, 由成员 D 手动确认。

8. 当前 GitHub Actions

当前仓库使用 `.github/workflows/pr-check.yml`, 覆盖 PR 和 push：

```
on:
  pull_request:
    branches: [qa, main]
  push:
    branches: [qa, main]
```

Job	目的	关键命令/动作
<code>backend</code>	后端单元、集成、Flyway 和 JaCoCo 检查	<code>cd backend && ./mvnw -B verify</code>
<code>backend mysql smoke</code>	用 MySQL 8.4 service container 验证真实数据库启动、Flyway migration 和基础 API	package 后启动后端；curl health、market search、创建现金持仓、summary
<code>frontend</code>	前端依赖安装、Vitest 和生产构建	<code>npm ci</code> 、 <code>npm run test -- --run</code> 、 <code>npm run build</code>
<code>docs</code>	防止核心文档缺失	检查 API、数据库、团队指南和蓝湖说明存在

8.1 可选后续增强

如果项目继续演进, 可新增：

- `release.yml`：仅在 tag 或 `main` push 后打包后端 jar 和前端 dist。
- Playwright smoke：启动前后端后跑浏览器级路径, 覆盖 Gateway -> Add Holding -> Dashboard -> Analysis。
- OpenAPI diff：防止 DTO 字段变更没有同步 API 文档和前端类型。

9. 自动化门禁矩阵

合并目标	必须通过	人工动作
feature/* -> qa	backend、backend mysql smoke、frontend、docs	1 名队友 review
docs/* -> qa	docs；若改动 API/运行流程，也确认 backend、frontend	1 名队友 review
bugfix/* -> qa	受影响模块 + 全量 pr-check	1 名队友 review
qa -> main	backend、backend mysql smoke、frontend、docs	成员 D 完成验收清单
hotfix/* -> main	受影响模块 + 全量 pr-check	最少 1 名相关负责人 review

10. 四天落地计划

时间	Git/CI 动作	负责人
Day 1 Planning	创建 qa；配置 branch protection；提交 workflow 草案	成员 D + 成员 A
Day 2 Planning	冻结 API 契约；让 pr-check 至少能跑通空项目或骨架项目	成员 A + 成员 C
Day 3 Coding	所有 Story 用 feature/* -> qa；中午和收工前各做一次 qa 集成	全员
Day 4 Coding	qa -> main；验收通过后在 main 打 1.0.0	成员 D + 全员

11. 常见冲突处理

- 后端 migration 冲突：不改已合并 migration，新建更高版本 V<n>__*.sql。
- OpenAPI 字段冲突：先更新 API 文档和 mock，再合并实现。
- 前后端都改 DTO：以 api_documentation_zh.md 和 OpenAPI 为准。
- qa 红灯：暂停合并新 feature，优先建 bugfix/* 修 CI。
- 已发布版本出问题：从 main 建 hotfix/*，合回 main 后再同步回 qa；不要从 qa 带入未验收功能。

12. 参考链接

主题	链接
GitHub protected branches	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
GitHub Actions workflow syntax	https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
GitHub Actions concurrency	https://docs.github.com/en/actions/how-to-write-workflows/choose-when-workflows-run/control-workflow-concurrency
actions/setup-java	https://github.com/actions/setup-java
actions/setup-node	https://github.com/actions/setup-node
actions/upload-artifact	https://github.com/actions/upload-artifact

HoldHive 实时股价 API 方案

1. 决策摘要

本项目需要的是“足够稳定、可解释、适合本地联调”的报价能力，不是交易级行情系统。前端不直接请求第三方行情接口，统一请求后端；后端通过 `PricingAdapter` 对接外部行情、缓存和演示数据。

推荐路径：

1. MVP 免费主路径：东方财富公共行情接口。已在本机验证可返回 A 股、美股和港股示例；无需 API key，适合股票和 ETF 的快速联调。
2. 免费 fallback：腾讯行情接口或新浪行情接口。当东方财富不可用时，仅用于 A 股备用解析。
3. 免费官方备选：Tiingo Starter。需要注册 token，官方额度是 `50 requests/hour`、`1,000 requests/day`；它是免费且合规的备选，但要把缓存 TTL 调到 90-120 秒，避免四人频繁刷新触发小时额度。
4. 可选研究/后备：AKShare / AKTools。AKShare 覆盖面广，适合调研和手工验证；AKTools 可把 AKShare 暴露为 HTTP API。但它会引入 Python/FastAPI/Uvicorn 运行时，不适合作为当前 Java 后端两天编码的默认主链路。
5. 不进入主链路：Alpha Vantage 免费档和 Yahoo Finance 网页接口。Alpha Vantage 官方免费额度只有 `25 requests/day`，明显不适合联调；本机在 2026-07-25 调用 Yahoo Chart 接口返回 `Edge: Too Many Requests`，也不满足稳定要求。
6. 付费只作为长期选项。Tiingo Power、Finnhub 付费/团队 token 或其他商业 API 适合公开部署或长期运行，不是本项目默认方案。

所有外部行情都必须经过后端缓存。前端只能调用 HoldHive 自己的 `/api/v1/market/*` 或 `/api/v1/portfolio/summary`，避免暴露 token、绕过缓存或被 CORS/限流影响。现金不访问外部行情，后端固定按组合基准币种 `1.00000000` 估值。

“频率不太低”的项目标准定义为：四人本地联调时，Dashboard 每 30-60 秒刷新一次、搜索框带防抖、手动刷新有冷却，不能因为几轮演示就触发 429。按这个标准，东方财富公共接口 + 后端批量请求 + 30-60 秒缓存是最佳免费方案；如果必须使用官方免费 token，Tiingo Starter 也可用，但缓存建议提高到 90-120 秒。Alpha Vantage 免费档只有 25 次/日，排除出主链路。

2. 候选接口与使用结论

接口	覆盖范围	使用结论	频率判断
东方财富 push2 批量报价	A 股、美股，港股需结合单标的接口验证	P0 推荐，作为默认实时/准实时报价源	无需 key，未公开正式 SLA；本项目必须缓存并限制后端请求
东方财富单标的报价	单只证券，适合搜索后校验	P0 推荐，用于新增持仓时校验代码和名称	无需 key，适合低频校验
东方财富搜索建议	ticker/name 到 QuoteID 映射	P0 推荐，把用户输入转成 provider quote id	用户输入时防抖，不在每次键盘输入都请求
腾讯 A 股行情	A 股	P1 fallback，解析成本略高	无需 key；仅失败兜底
新浪 A 股行情	A 股	P1 fallback，请求建议带 Referer	无需 key；仅失败兜底
Tiingo Equity Realtime / IEX API	美股和 ETF，需 token	官方免费备选，适合需要正式 API 条款时使用	Starter 免费但只有 50/hour、1,000/day；必须缓存
Tiingo Pricing	额度参考	选型依据	Starter 为 50/hour、1,000/day；Power 为 10,000/hour、100,000/day
AKShare	A 股、港股、美股、基金、期货、指数等多类财经数据	P2 可选研究/后备，不作为默认 Java 主链路	Python 库，主要用于学术研究；接口依赖公开数据源，需经常跟进版本
AKTools HTTP	将 AKShare 封装成本地 HTTP API	P2 可选，仅在团队接受额外 Python 服务时使用	依赖 AKTools、AKShare、FastAPI、Uvicorn、Typer；增加联调与启动复杂度
Alpha Vantage Free API	美股等，需 key	不推荐主链路	免费仅 25/day，不满足联调刷新
Finnhub Stock Quote API	美股、部分全球市场，需 token	长期备选，接口简单	超出套餐会 429，另有 30 calls/second 上限；免费额度需以账号页为准
CoinGecko Simple Price	BTC、ETH 等主流加密资产	已接入为加密资产主路径，失败后走缓存/demo	Demo 免费计划可用，但有分钟和月度额度；必须缓存
Binance public market data	交易所现货交易对，例如 BTC/USDT	P1 备选，接口简单	受地区和交易所服务可用性影响；只做价格展示，不做交易
Coinbase Exchange ticker	交易所产品，例如 BTC-USD	P1 备选	公共端点按 IP 限流；适合少量展示，不做高频刷新
Yahoo Chart / Quote 网页接口	美股等	不作为默认源	本机实测返回 Too Many Requests，不满足稳定联调要求

2.1 资产类型与行情策略

资产类型	MVP 策略	说明
STOCK	东方财富主链路，腾讯/新浪 A 股 fallback，Tiingo/Finnhub 作为官方备选	当前实时行情工作的主要范围
ETF	与股票同链路处理	ETF 像股票一样交易，有 ticker 和价格
MUTUAL_FUND	MVP 使用演示/缓存净值；P1 接入基金净值与持仓披露	场外基金没有盘中股票式报价，穿透数据只用于分析
CRYPTO	CoinGecko 实时价格 -> 有效缓存 -> demo/unavailable	支持 BTC/ETH 展示和估值，不接钱包或交易所账户
CASH	后端固定价格 1.00000000，priceStatus = FIXED	不请求任何外部 API，不写价格快照
BANK_DEPOSIT	后端固定价格 1.00000000，priceStatus = FIXED	按本金纳入组合总值；利息和到期收益后续扩展

3. 东方财富接口用法

3.1 secid 规则

HoldHive 不应只保存用户输入的 ticker，还应保存 provider quote id，避免每次查询都重新搜索。

用户输入	市场	东方财富 <i>secid</i> / <i>QuoteID</i>	验证状态
600519.SH 或 600519	上海 A 股	1.600519	已验证
000001.SZ 或 000001	深圳 A 股	0.000001	已验证
AAPL	美股 NASDAQ	105.AAPL	已验证
00700.HK	港股	116.00700	单标的接口已验证；批量接口联调时再确认

用户新增持仓时，后端执行：

1. 标准化输入，例如去空格、转大写、识别 .SH / .SZ / .HK 后缀。
2. 可直接映射的 A 股使用规则映射。
3. 美股或不确定市场调用东方财富搜索建议接口，读取 *QuoteID*、*Name*、*JYS* 和 *SecurityTypeName*。
4. 保存 *instrument.provider* = *EASTMONEY* 和 *instrument.provider_quote_id* = 105.AAPL。
5. 报价失败时仍允许保存持仓，但返回 *priceStatus* = *UNAVAILABLE*。

3.2 批量报价

GET <https://push2.eastmoney.com/api/qt/ulist.np/get?fltt=2&invt=2&fields=f12,f14,f2,f3,f4,f5,f6,f13,f15,f16,f17,f18,f20,f21,f124&secids=1.600519,0.000001,105.AAPL>

关键字段映射：

字段	含义	HoldHive 字段
f12	证券代码	<i>ticker</i>
f13	市场编号	<i>providerMarketCode</i>
f14	中文或本地显示名	<i>displayName</i>
f2	最新价	<i>currentPrice</i>
f3	涨跌幅百分数	<i>changePercent</i>
f4	涨跌额	<i>changeAmount</i>
f5	成交量	<i>volume</i>
f6	成交额	<i>turnover</i>
f15	最高价	<i>dayHigh</i>
f16	最低价	<i>dayLow</i>
f17	开盘价	<i>openPrice</i>
f18	昨收价	<i>previousClose</i>
f124	Unix 秒级行情时间	<i>priceObservedAt</i>

字段为 -、空值或明显无效时，后端应将该持仓报价标记为 *UNAVAILABLE*，不得把未知价格当作 0。

3.3 单标的报价

GET <https://push2.eastmoney.com/api/qt/stock/get?fltt=2&invt=2&fields=f43,f44,f45,f46,f47,f48,f49,f57,f58,f59,f60,f86,f107&secid=105.AAPL>

关键字段映射：

字段	含义	HoldHive 字段
f57	证券代码	<i>ticker</i>
f58	名称	<i>displayName</i>
f43	最新价	<i>currentPrice</i>

字段	含义	HoldHive 字段
f44	最高价	dayHigh
f45	最低价	dayLow
f46	开盘价	openPrice
f60	昨收价	previousClose
f86	Unix 秒级行情时间	priceObservedAt
f107	市场编号	providerMarketCode

单标的接口适合新增持仓时校验一个 ticker 是否存在，或者在批量接口对某市场返回异常时做 fallback。

3.4 搜索建议

GET <https://searchapi.eastmoney.com/api/suggest/get?input=AAPL&type=14&token=D43BF722C8E33BD5A6040B1F8FD653E5>

前端搜索框不直接调用该接口。后端提供 `/api/v1/market/search`，并加 300-500ms 防抖和服务端短缓存。返回候选项时只暴露必要字段：

```
{
  "query": "AAPL",
  "results": [
    {
      "ticker": "AAPL",
      "displayName": "苹果",
      "exchangeCode": "NASDAQ",
      "provider": "EASTMONEY",
      "providerQuoteId": "105.AAPL",
      "assetType": "US_STOCK"
    }
  ]
}
```

4. 腾讯与新浪 fallback

腾讯接口：

GET <https://qt.gtimg.cn/q=sh600519,sz000001>

腾讯返回 ~ 分隔字符串。MVP 只在东方财富失败时解析：名称、代码、最新价、昨收、开盘、成交量、成交额、行情时间。解析失败时直接降级为缓存或演示价格，不在页面暴露原始字符串。

新浪接口：

curl -H "Referer: https://finance.sina.com.cn" "https://hq.sinajs.cn/list=sh600519,sz000001"

新浪返回逗号分隔字符串。MVP 只作为 A 股备用源，不做港股、美股支持。

5. 官方 API 备选

5.1 Tiingo Starter 免费备选

Tiingo 提供带 token 的官方 API 文档和价格页。其文档说明使用账号 token 认证，并按小时、天和月度带宽限制；同时明确“不按分钟或秒限流”。价格页显示 Starter 为 50 requests/hour、1,000 requests/day，Power 为 10,000 requests/hour、100,000 requests/day。因此 Starter 只适合个人测试，Power 更适合四人小组联调和稳定演示。

如果坚持免费，Tiingo Starter 的使用方式是：

- 只作为东方财富失败时的美股备用源。
- 后端批量合并 symbol，不允许每个组件单独请求。
- MARKET_CACHE_TTL_SECONDS 设置为 90 或 120。

- 手动刷新按钮冷却设置为 30 秒。
- 超出额度时立刻降级到缓存或 DEMO，不要在前端重复重试。

示例：

```
GET https://api.tiingo.com/iex/?tickers=AAPL,MSFT&token=${TIINGO_TOKEN}
```

后端配置：

```
MARKET_PROVIDER=TIINGO
TIINGO_TOKEN=<team-token>
MARKET_CACHE_TTL_SECONDS=30
```

5.2 Finnhub

Finnhub 的 Quote API 简单，适合美股价格查询。Finnhub 文档说明 GET 请求可用 token 参数或 X-Finnhub-Token 认证，超出额度返回 429，且所有套餐之上还有 30 API calls/second 上限。这个秒级上限不低，但套餐自身额度仍要以账号实际页面为准；多人联调时必须用后端缓存，不能让每个浏览器直接请求 Finnhub。

示例：

```
GET https://finnhub.io/api/v1/quote?symbol=AAPL&token=${FINNHUB_TOKEN}
```

返回字段通常包括 c 最新价、d 涨跌额、dp 涨跌幅、h 最高、l 最低、o 开盘、pc 昨收和 t 时间戳。后端必须转换成统一 MarketQuote。

5.3 Alpha Vantage 不推荐作为主链路

Alpha Vantage 提供免费 API key，但官方支持页说明免费股票 API 只有 25 requests/day。这个额度适合一天内少量手工验证，不适合多人开发时反复刷新 Dashboard，因此不进入默认方案。

5.4 AKShare / AKTools 判断

结论：可以作为“数据调研工具”或“P2 后备行情服务”，但不建议作为 HoldHive MVP 的默认实时股价源。

理由：

- AKShare 是 Python 财经数据接口库，不是可直接被 Java/Spring Boot 调用的远程官方行情 API。
- 官方说明 AKShare 数据来自公开数据源，主要用于学术研究；网页变化可能导致接口需要持续维护和升级。
- AKTools 可以将 AKShare 暴露为 HTTP API，但会额外引入 Python、AKTools、FastAPI、Uvicorn、Typer 等运行时。当前项目已经明确“不依赖 Docker”，再增加一个 Python HTTP 服务会提高成员本机启动和联调成本。
- 两天编码期内，成员 B 若同时维护 Java pricing adapter 和 Python/AKTools 服务，职责边界会变重，不利于快速稳定交付。

可接受用法：

```
MARKET_PROVIDER=EASTMONEY
MARKET_FALLBACK_PROVIDERS=TENCENT,SINA,DEMO
# P2 optional only
AKTOOLS_BASE_URL=http://127.0.0.1:8081
```

如果团队后续决定引入 AKTools，必须满足：

1. 成员 D 把 Python 版本、安装命令和启动命令加入 CONTRIBUTING.md。
2. 成员 B 只通过 AkToolsPricingAdapter 调用本地 HTTP，不在 Java 服务里直接运行 Python 脚本。
3. 失败时仍按 BEST_AVAILABLE -> cache -> demo -> unavailable 降级。
4. PR 里明确说明为什么东方财富、腾讯/新浪和 Tiingo 不满足当期需求。

5.5 加密资产行情

当前后端已为 CRYPTO:BTC 和 CRYPTO:ETH 接入 CoinGecko Simple Price，并保留缓存和 demo fallback。加密资产仍不接钱包、交易所账户或链上流水，只把外部价格转换成统一 MarketQuote。

如果后续扩展更多币种，可按以下顺序接入：

1. CoinGecko Simple Price：按 coin id 查询，例如 **bitcoin**、**ethereum**；适合把 **BTC**、**ETH** 转成统一 USD 价格。
2. Binance public market data：按交易对查询，例如 **BTCUSDT**；适合熟悉交易所 symbol 的团队，但要注意地区可用性。
3. Coinbase Exchange ticker：按产品查询，例如 **BTC-USD**；适合少量查询和手工联调。

后端统一转换为 **MarketQuote**，前端不感知第三方来源：

```
GET /api/v1/market/quotes?providerQuoteIds=CRYPTO:BTC,CRYPTO:ETH&priceMode=BEST_AVAILABLE
```

示例响应片段：

```
{
  "ticker": "BTC",
  "displayName": "Bitcoin",
  "assetType": "CRYPTO",
  "providerQuoteId": "CRYPTO:BTC",
  "currency": "USD",
  "currentPrice": 66250.00000000,
  "priceStatus": "DEMO",
  "priceObservedAt": "2026-07-25T09:10:00Z"
}
```

实时加密行情接入后，仍必须遵守：

- 后端缓存 TTL 不低于 30 秒。
- 测试使用 stub 或 demo adapter，不访问真实 CoinGecko、Binance 或 Coinbase。
- 任何 provider 失败都降级为缓存、演示价格或 **UNAVAILABLE**。
- 不接钱包、不读取账户余额、不发起交易、不保存交易所 API key。

6. 后端统一接口

第三方接口只由后端访问。前端需要行情时调用 HoldHive 自己的 API。

6.1 搜索证券

```
GET /api/v1/market/search?query=AAPL
```

响应：

```
{
  "query": "AAPL",
  "results": [
    {
      "ticker": "AAPL",
      "displayName": "苹果",
      "exchangeCode": "NASDAQ",
      "provider": "EASTMONEY",
      "providerQuoteId": "105.AAPL",
      "assetType": "STOCK"
    }
  ],
  "source": "MIXED",
  "cached": false
}
```

6.2 批量获取报价

```
GET /api/v1/market/quotes?providerQuoteIds=1.600519,0.000001,105.AAPL&priceMode=BEST_AVAILABLE
```

响应：

```
{
  "provider": "MIXED",
  "priceMode": "BEST_AVAILABLE",
  "quotes": [
    {
      "provider": "EASTMONEY",
      "providerQuoteId": "105.AAPL",

```

```
"ticker": "AAPL",
"display": "AAPL",
"currency": "USD",
"currentPrice": 333.02000000,
"priceStatus": "LIVE",
"priceObservedAt": "2026-07-25T00:00:00Z"
},
"unavailable": []
}
```

6.3 前端使用规则

- Dashboard 初次加载时请求 `/api/v1/portfolio/summary`，不要并发打第三方行情。
- 添加持仓时可请求 `/api/v1/market/search`，搜索框至少 300ms 防抖。
- 用户手动刷新报价时，请求 `/api/v1/market/quotes` 或刷新 `summary`；按钮 10 秒冷却。
- 如果后端返回 `DEMO`、`CACHED` 或 `UNAVAILABLE`，UI 必须显示状态标签。

7. 缓存、限流和失败策略

7.1 后端缓存

场景	TTL 建议	说明
交易时段 Dashboard	30-60 秒	足够演示实时感，避免多人刷新打爆接口
非交易时段	5-30 分钟	行情不频繁变化，减少请求
搜索建议	10-30 分钟	ticker 映射基本稳定
失败后的 last-known price	最长 24 小时	必须标记 <code>CACHED</code> ，不能伪装为实时

7.2 后端限流

- 单个 provider 默认最多 1 次外部请求/秒。
- 批量报价优先合并请求，一次最多 50 个 `providerQuoteId`。
- 同一用户或同一浏览器手动刷新至少 10 秒间隔。
- 连续 3 次 provider 失败后进入 60 秒熔断，只返回缓存或 `UNAVAILABLE`。

7.3 失败降级顺序

```
BEST_AVAILABLE = fresh external quote
-> fresh cache
-> last-known cache with CACHED status
-> explicit demo quote if DEMO_ALLOWED
-> UNAVAILABLE
```

`CASH` 不进入以上链路，始终由后端返回固定估值。其他资产的任何降级都必须在响应中暴露 `priceStatus`、`provider` 和 `priceObservedAt`。

8. 配置建议

```
HOLDHIVE_MARKET_EXTERNAL_ENABLED=true
HOLDHIVE_MARKET_CACHE_TTL=60s
HOLDHIVE_MARKET_HTTP_TIMEOUT=3s
```

如果使用官方 token 接口：

```
MARKET_PROVIDER=TIINGO
TIINGO_TOKEN=<team-token>
FINNHUB_TOKEN=<team-token-if-used>
```

不要把 token 写入 Git。仓库只提交 `.env.example`，真实值由每位成员本机配置。

9. 测试与验收

后端测试不得依赖真实行情服务。真实接口只在手工联调或专门的 integration profile 下运行。

必须覆盖：

- 东方财富正常返回 A 股、美股报价时能转换成统一 DTO。
- ETF 报价按股票链路转换，assetType 保持 **ETF**。
- 加密资产在 live/cache/demo 模式下可估值，且状态分别返回 **LIVE**、**CACHED** 或 **DEMO**，不得伪造实时价格。
- 现金不触发外部行情请求，固定返回 `priceStatus = FIXED`。
- 单只证券价格为 -、空或缺字段时返回 **UNAVAILABLE**。
- Provider 超时或返回非 JSON 时，服务不崩溃，并按降级顺序返回缓存或不可用状态。
- 前端不会直接访问第三方 URL。
- 手动刷新按钮有冷却，重复点击不会触发多次后端刷新。
- 演示价格永远显示 **DEMO**，缓存价格永远显示 **CACHED**。

10. 实测记录

实测时间：2026-07-25，地点：当前本机网络。

接口	示例	结果
东方财富批量报价	1.600519,0.000001,105.AAPL	返回 rc=0 和最新价字段
东方财富单标的	105.AAPL、116.00700	返回 rc=0 和最新价字段
东方财富搜索	AAPL、00700	返回 QuoteID 映射
腾讯行情	sh600519,sz000001	返回行情字符串
新浪行情	sh600519,sz000001	返回行情字符串
Yahoo Chart	AAPL	返回 Edge: Too Many Requests，不作为默认源

以上公共网页接口没有正式 SLA。它们满足本项目本地训练和演示要求，但不适合公开生产服务。若项目需要公开部署或长期稳定运行，必须改用 Tiingo、Finnhub 或其他正式商业行情服务。

11. 最终推荐

两天编码项目按以下免费优先组合实现，兼顾可用性和请求频率：

场景	推荐实现	原因
默认本地演示	东方财富 <code>search + stock/get + ulist</code>	本机已验证，无 key，A 股/美股/港股覆盖足够演示
A 股备用	腾讯或新浪接口	返回简单，作为东方财富失败兜底
免费官方备用	Tiingo Starter	免费且有官方文档；用 90-120 秒缓存规避 50/hour 限制
可选研究/后备	AKShare / AKTools	覆盖面广，但引入 Python HTTP 服务和学术研究用途限制，不作为默认主链路
不推荐免费源	Alpha Vantage 免费档	25/day 太低，不适合联调
长期/公开部署	Tiingo Power 或 Finnhub 团队 token	有文档、认证和明确额度，适合项目需要正式 API 时切换
断网或 provider 失败	DemoProvider + MySQL 缓存	保证展示不被外部服务阻断

不要在前端直接访问 Yahoo、东方财富、腾讯、新浪、Tiingo 或 Finnhub。所有第三方请求统一进后端 `PricingAdapter`，并通过 `/api/v1/market/search`、`/api/v1/market/quotes` 和 `/api/v1/portfolio/summary` 暴露给前端。

HoldHive 数据库设计

1. 设计目标

本设计优先支持两天编码周期内的多资产 MVP，同时保留向多组合、历史价格、交易流水和多币种扩展的路径。

设计原则：

- 使用 MySQL 8.4 LTS 和 InnoDB，并确保项目环境的小版本能够执行全部 Flyway 迁移。
- 金额和数量使用定点数，不使用浮点数。
- 持仓事实与市场价格分离，外部价格失败不能破坏持仓数据。
- MVP 不引入用户、券商账户、交易、税务和公司行动等复杂模型。
- 通过外键、唯一约束和检查约束保护数据，而不是只依赖前端校验。
- 数据库迁移脚本纳入 Git，并按版本顺序执行。

2. 分阶段模型

MVP 必须实现

- `portfolio`：组合基本信息。当前虽然只有一个组合，仍保留组合边界，避免持仓表以后整体重构。
- `instrument`：可估值资产的稳定身份，不把 ticker 文本重复存入每条持仓。
- `holding`：某个组合当前持有某个证券的数量和平均成本。
- `price_snapshot`：市场价格缓存和数据来源状态。

后续扩展

- `portfolio_transaction`：买入、卖出、股息、费用、存取款等不可变流水。
- `portfolio_valuation`：按日保存组合价值，用于历史表现图。
- `app_user`、`portfolio_member`：认证和多用户授权。
- `account`：券商、现金或托管账户。
- `exchange_rate`：多币种换算。

MVP 不应提前创建所有扩展表。扩展模型用于锁定演进方向，防止当前字段设计堵死后续能力。

3. 实体关系

```
erDiagram
    PORTFOLIO ||--o{ HOLDING : contains
    INSTRUMENT ||--o{ HOLDING : identifies
    INSTRUMENT ||--o{ PRICE_SNAPSHOT : priced_by

    PORTFOLIO {
        bigint id PK
        varchar name
        char base_currency
        datetime created_at
        datetime updated_at
    }

    INSTRUMENT {
        bigint id PK
        varchar ticker
        varchar exchange_code
        varchar display_name
        varchar asset_type
        varchar provider
        varchar provider_quote_id
        char currency
        datetime created_at
        datetime updated_at
    }

    HOLDING {
        bigint id PK
        bigint portfolio_id FK
        bigint instrument_id FK
        decimal quantity
        decimal average_purchase_price
        bigint version
        datetime created_at
        datetime updated_at
    }

    PRICE_SNAPSHOT {
        bigint id PK
        bigint instrument_id FK
        decimal price
        char currency
        varchar provider
        varchar status
        boolean is_demo
        datetime observed_at
        datetime created_at
    }
```

4. 表结构

4.1 portfolio

字段	建议类型	约束	说明
id	BIGINT	PK, 自动生成	组合 ID
name	VARCHAR(100)	NOT NULL	组合名称
base_currency	CHAR(3)	NOT NULL, 默认 USD	ISO 4217 基准币种
created_at	DATETIME(6)	NOT NULL	UTC 创建时间
updated_at	DATETIME(6)	NOT NULL	UTC 最后更新时间

MVP 启动时创建一个默认组合，例如 **My Portfolio**，但业务代码不应把 ID 1 散落在各处。默认组合 ID 应由配置或查询获得。

4.2 instrument

字段	建议类型	约束	说明
id	BIGINT	PK, 自动生成	资产 ID
ticker	VARCHAR(32)	NOT NULL	标准化为大写，例如 AAPL、VOO、BTC、USD
exchange_code	VARCHAR(16)	NOT NULL, 默认 UNKNOWN	交易所或分类代码；现金使用 CASH
display_name	VARCHAR(160)	NULL	展示名称
asset_type	VARCHAR(24)	NOT NULL, 默认 STOCK	MVP 允许 STOCK、ETF、MUTUAL_FUND、CRYPTO、CASH、BANK_DEPOSIT
provider	VARCHAR(32)	NULL	行情来源，例如 EASTMONEY、DEMO、FIXED
provider_quote_id	VARCHAR(64)	NULL	外部行情 ID，例如 105.AAPL；现金为 NULL
currency	CHAR(3)	NOT NULL, 默认 USD	报价币种
created_at	DATETIME(6)	NOT NULL	UTC 创建时间
updated_at	DATETIME(6)	NOT NULL	UTC 最后更新时间

唯一约束：UNIQUE (asset_type, ticker, exchange_code)。这样 BTC 作为加密货币不会与同名股票或其他资产类别冲突。

资产类型规则：

- **STOCK** 和 **ETF**：通常需要 provider_quote_id，可以来自 /api/v1/market/search；ETF 估值像股票，底层持仓只进入穿透分析。
- **MUTUAL_FUND**：保存基金代码或自定义代码，MVP 可使用演示/缓存净值；持仓披露数据进入独立的 fund lookthrough 数据，不写回主持仓。
- **CRYPTO**：MVP 可以使用 provider = DEMO 和演示价格；接入实时接口后再保存真实 provider quote id。
- **CASH**：ticker 使用币种代码，例如 USD；exchange_code = CASH；provider = FIXED；不需要 price_snapshot。
- **BANK_DEPOSIT**：ticker 使用可读代码，例如 HSBC_USD；exchange_code = BANK；provider = FIXED；MVP 按本金固定估值，不自动计提利息。

4.3 holding

字段	建议类型	约束	说明
id	BIGINT	PK, 自动生成	持仓 ID
portfolio_id	BIGINT	NOT NULL, FK	所属组合
instrument_id	BIGINT	NOT NULL, FK	对应证券
quantity	DECIMAL(24,8)	NOT NULL, > 0	持仓数量，支持小数份额
average_purchase_price	DECIMAL(24,8)	NOT NULL, >= 0	每单位平均买入价
version	BIGINT	NOT NULL, 默认 0	后续更新时用于乐观锁
created_at	DATETIME(6)	NOT NULL	UTC 创建时间
updated_at	DATETIME(6)	NOT NULL	UTC 最后更新时间

唯一约束：UNIQUE (portfolio_id, instrument_id)。MVP 中同一组合不得重复创建同一证券；重复请求返回 409 HOLDING_ALREADY_EXISTS。未来若需要分批成本，应引入交易或批次表，而不是复制持仓行。

外键删除策略：删除组合时可以级联删除其持仓；删除已被持仓引用的证券应被拒绝。MVP 只删除持仓，不物理删除证券主数据。

4.4 price_snapshot

字段	建议类型	约束	说明
id	BIGINT	PK, 自动生成	价格记录 ID
instrument_id	BIGINT	NOT NULL, FK	对应证券
price	DECIMAL(24,8)	NOT NULL, >= 0	市场价格
currency	CHAR(3)	NOT NULL	报价币种
provider	VARCHAR(64)	NOT NULL	数据提供方, 例如 <code>SAMPLE_API</code>
status	VARCHAR(32)	NOT NULL	写入时的价格状态, 例如 <code>LIVE</code> ; 兼容初始 migration
is_demo	BOOLEAN	NOT NULL, 默认 <code>FALSE</code>	是否为演示价格
observed_at	DATETIME(6)	NOT NULL	价格对应的 UTC 市场时间
created_at	DATETIME(6)	NOT NULL	UTC 系统写入时间

唯一约束：UNIQUE (instrument_id, provider, observed_at)。

MVP 查询只取每个资产最新的一条有效价格。市场服务失败时，可以读取未过期缓存；若使用演示数据，必须设置 `is_demo = true` 并通过 API 返回该状态。现金不写入 `price_snapshot`，由后端按固定价格规则直接估值。

5. 参考 DDL

以下 DDL 面向 MySQL 8.4。所有连接必须使用 UTC，会话初始化执行 `SET time_zone = '+00:00'`；应用层仍以 ISO 8601 UTC 对外传输时间。

```
CREATE TABLE portfolio (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  base_currency CHAR(3) NOT NULL DEFAULT 'USD',
  created_at DATETIME(6) NOT NULL,
  updated_at DATETIME(6) NOT NULL,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE instrument (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ticker VARCHAR(32) NOT NULL,
  exchange_code VARCHAR(16) NOT NULL DEFAULT 'UNKNOWN',
  display_name VARCHAR(160),
  asset_type VARCHAR(24) NOT NULL DEFAULT 'STOCK',
  provider VARCHAR(32),
  provider_quote_id VARCHAR(64),
  currency CHAR(3) NOT NULL DEFAULT 'USD',
  created_at DATETIME(6) NOT NULL,
  updated_at DATETIME(6) NOT NULL,
  PRIMARY KEY (id),
  CONSTRAINT ck_instrument_asset_type
    CHECK (asset_type IN ('STOCK', 'ETF', 'MUTUAL_FUND', 'CRYPTO', 'CASH', 'BANK_DEPOSIT')),
  CONSTRAINT uq_instrument_symbol UNIQUE (asset_type, ticker, exchange_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE holding (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  portfolio_id BIGINT UNSIGNED NOT NULL,
  instrument_id BIGINT UNSIGNED NOT NULL,
  quantity DECIMAL(24,8) NOT NULL,
  average_purchase_price DECIMAL(24,8) NOT NULL,
```

```

version BIGINT UNSIGNED NOT NULL DEFAULT 0,
created_at DATETIME(6) NOT NULL,
updated_at DATETIME(6) NOT NULL,
PRIMARY KEY (id),
CONSTRAINT ck_holding_quantity CHECK (quantity > 0),
CONSTRAINT ck_holding_average_price CHECK (average_purchase_price >= 0),
CONSTRAINT uq_holding_instrument UNIQUE (portfolio_id, instrument_id),
CONSTRAINT fk_holding_portfolio FOREIGN KEY (portfolio_id)
    REFERENCES portfolio(id) ON DELETE CASCADE,
CONSTRAINT fk_holding_instrument FOREIGN KEY (instrument_id)
    REFERENCES instrument(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

```

CREATE TABLE price_snapshot (
id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
instrument_id BIGINT UNSIGNED NOT NULL,
price DECIMAL(24,8) NOT NULL,
currency CHAR(3) NOT NULL,
provider VARCHAR(64) NOT NULL,
status VARCHAR(32) NOT NULL,
is_demo BOOLEAN NOT NULL DEFAULT FALSE,
observed_at DATETIME(6) NOT NULL,
created_at DATETIME(6) NOT NULL,
PRIMARY KEY (id),
CONSTRAINT ck_price_non_negative CHECK (price >= 0),
CONSTRAINT uq_price_observation
    UNIQUE (instrument_id, provider, observed_at),
CONSTRAINT fk_price_instrument FOREIGN KEY (instrument_id)
    REFERENCES instrument(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

6. 索引设计

```

CREATE INDEX idx_holding_portfolio ON holding(portfolio_id);
CREATE INDEX idx_price_latest
    ON price_snapshot(instrument_id, observed_at DESC);

```

MVP 数据量很小，不应创建大量推测性索引。新增索引前应通过查询计划或可复现的性能问题证明必要性。

7. 一致性与事务

- 创建持仓时，在同一事务中查找或创建 **instrument**，随后创建 **holding**。
- 依靠唯一约束处理并发重复，不使用“先查后插”作为唯一保护。
- 删除持仓只删除 **holding**；不删除 **instrument** 和历史价格。
- 从外部获取价格不应与持仓写事务绑定，避免网络超时长时间占用数据库事务。
- 时间统一以 UTC 保存，API 使用 ISO 8601，例如 **2026-07-24T08:30:00Z**。
- MySQL 服务、JDBC 连接和应用时区全部显式设为 UTC；不得依赖开发机本地时区。
- 金额计算使用 **BigDecimal**、**Decimal** 或等价定点类型，并在展示层统一舍入。

8. 扩展到交易账本

当产品需要买卖历史、已实现盈亏、股息或历史收益时，增加不可变流水表：

```

portfolio_transaction
- id
- portfolio_id
- instrument_id
- transaction_type: BUY | SELL | DIVIDEND | FEE | DEPOSIT | WITHDRAWAL
- trade_date
- quantity
- unit_price
- fee_amount
- currency

```

- external_reference
- created_at

届时 **holding** 可继续作为由交易聚合得到的读模型，以提高查询速度；交易表成为事实来源。迁移步骤应为：创建交易表、为现有持仓生成期初交易、双写校验、切换计算来源，而不是直接删除现有持仓数据。

9. 数据库验收清单

- [] 所有表通过版本化迁移创建，不依赖手工建表。
- [] 数量、价格和唯一性约束同时在服务层和数据库层生效。
- [] 同一组合不能重复创建相同证券持仓。
- [] 删除持仓不会删除证券或价格历史。
- [] 市场价格失败不会修改或丢失持仓。
- [] 测试覆盖空组合、重复持仓、非法数值、外键失败和并发重复。
- [] 数据库中不保存浮点金额、密钥、券商凭据或真实个人资产信息。

HoldHive REST API 文档

1. 文档范围

本文定义 HoldHive MVP 的 REST API 契约，并标记后续扩展接口。实现后应使用 OpenAPI/Swagger 维护机器可读版本；本文和 OpenAPI 必须保持一致。

- Base URL：/api/v1
- 数据格式：application/json
- 字符编码：UTF-8
- 时间格式：ISO 8601 UTC，例如 2026-07-24T08:30:00Z
- 金额和数量：JSON number；服务端必须使用定点数处理
- 认证：MVP 无认证，仅允许在本地或受控演示环境运行

2. 通用约定

2.1 标识符

资源 ID 在示例中使用正整数。客户端必须将 ID 当作不透明标识，不推断创建顺序或业务含义。

2.2 数值与币种

- quantity 必须大于 0，最多 8 位小数。
- averagePurchasePrice 必须大于等于 0，最多 8 位小数。
- currentPrice 和金额字段最多返回 8 位小数；UI 通常显示 2 位。
- 币种使用 ISO 4217 三位代码，例如 USD。
- 百分比直接返回百分数，例如 12.34 表示 12.34%，不是 0.1234。

2.3 资产类型

MVP 支持六类可展示资产：

类型	含义	估值方式
STOCK	普通股票，例如 AAPL、600519	通过行情适配器或演示价格估值
ETF	场内基金 / 交易所交易基金，例如 VOO、SPY、QQQ	与股票使用同一套行情和估值公式
MUTUAL_FUND	场外基金，例如开放式基金、货币基金	MVP 使用手工、演示或缓存净值；实时净值为 P1
CRYPTO	加密资产，例如 BTC、ETH	MVP 先支持演示/缓存价格；实时接口为 P1
CASH	现金余额，例如 USD	以组合基准币种固定按 1.00000000 估值
BANK_DEPOSIT	银行存款，例如活期或定期存款本金	以组合基准币种固定按 1.00000000 估值

本期不支持债券、基金申赎流水、存款利息自动计提、现金流水、链上钱包、交易所账户同步或多币种汇率换算。

2.4 数据状态

价格状态使用以下枚举：

状态	含义
LIVE	从外部数据服务获得的新鲜价格
CACHED	使用仍在允许时效内的缓存价格
DEMO	使用演示数据，不代表真实市场价格

状态	含义
FIXED	不依赖外部行情的固定估值，MVP 用于 CASH 和 BANK_DEPOSIT
UNAVAILABLE	没有可用价格，该持仓不计入当前估值

2.5 错误响应

所有非 2xx 响应使用统一结构：

```
{
  "code": "VALIDATION_FAILED",
  "message": "Request validation failed",
  "fieldErrors": [
    {
      "field": "quantity",
      "message": "must be greater than 0"
    }
  ],
  "traceId": "8f0c2d15d1e44dc8",
  "timestamp": "2026-07-24T08:30:00Z"
}
```

生产响应不得包含堆栈、SQL、内部类名或密钥。traceId 用于关联服务端日志。

2.6 错误码

HTTP	code	使用场景
400	VALIDATION_FAILED	字段缺失、格式或范围错误
400	MALFORMED_JSON	JSON 无法解析
404	HOLDING_NOT_FOUND	持仓不存在
409	HOLDING_ALREADY_EXISTS	同一组合已存在相同证券
409	CONCURRENT_MODIFICATION	资源版本冲突，供后续更新接口使用
503	PRICE_SERVICE_UNAVAILABLE	请求要求实时价格但价格服务不可用
500	INTERNAL_ERROR	未预期服务端错误

3. 数据模型

3.1 Holding

```
{
  "id": 101,
  "instrumentId": 201,
  "ticker": "AAPL",
  "exchangeCode": "NASDAQ",
  "displayName": "Apple Inc.",
  "assetType": "STOCK",
  "provider": "EASTMONEY",
  "providerQuoteId": "105.AAPL",
  "currency": "USD",
  "quantity": 10.00000000,
  "averagePurchasePrice": 175.50000000,
  "currentPrice": 210.25000000,
  "marketValue": 2102.50000000,
  "costBasis": 1755.00000000,
  "unrealizedGainLoss": 347.50000000,
  "unrealizedGainLossPercent": 19.80056980,
  "allocationPercent": 42.50000000,
  "priceStatus": "LIVE",
  "priceObservedAt": "2026-07-24T08:29:00Z",
  "createdAt": "2026-07-24T08:00:00Z",
  "updatedAt": "2026-07-24T08:00:00Z"
}
```

`id` 是持仓记录 ID，用于 `GET/PATCH/DELETE /api/v1/holdings/{holdingId}`。`instrumentId` 是证券主数据 ID，用于 `GET /api/v1/funds/{instrumentId}/lookthrough` 等按证券维度查询的接口。前端不得用 `holdingId` 代替 `instrumentId`。

当价格不可用时，`currentPrice`、`marketValue`、`unrealizedGainLoss`、`unrealizedGainLossPercent`、`allocationPercent` 和 `priceObservedAt` 为 `null`，`priceStatus` 为 `UNAVAILABLE`。`CASH` 和 `BANK_DEPOSIT` 不请求外部行情，`currentPrice` 固定为 `1.00000000`，`priceStatus` 为 `FIXED`。

3.2 PortfolioSummary

```
{
  "portfolioId": 1,
  "portfolioName": "My Portfolio",
  "baseCurrency": "USD",
  "holdingCount": 3,
  "pricedHoldingCount": 2,
  "valuationStatus": "PARTIAL",
  "totalCostBasis": 4200.00000000,
  "totalMarketValue": 4680.50000000,
  "totalUnrealizedGainLoss": 480.50000000,
  "totalUnrealizedGainLossPercent": 11.44047619,
  "priceAsOf": "2026-07-24T08:29:00Z",
  "allocations": [
    {
      "holdingId": 101,
      "ticker": "AAPL",
      "assetType": "STOCK",
      "marketValue": 2102.50000000,
      "allocationPercent": 44.91934622
    }
  ],
  "unpricedHoldings": [
    {
      "holdingId": 103,
      "ticker": "UNKNOWN",
      "assetType": "STOCK",
      "reason": "PRICE_UNAVAILABLE"
    }
  ]
}
```

valuationStatus 取值：

- **COMPLETE**：所有持仓都有有效价格。
- **PARTIAL**：部分持仓没有价格，总市值只包括有效持仓。
- **UNAVAILABLE**：没有任何持仓可以估值。
- **EMPTY**：组合没有持仓。

4. MVP 接口

4.1 查询全部持仓

GET /api/v1/holdings

可选查询参数：

参数	默认值	说明
sort	ticker,asc	支持 ticker、marketValue、unrealizedGainLoss
priceMode	BEST_AVAILABLE	BEST_AVAILABLE、LIVE_ONLY 或 DEMO_ALLOWED

成功响应：200 OK

```
{
  "items": [
    {
      "id": 101,
      "instrumentId": 201,
      "ticker": "AAPL",
      "exchangeCode": "NASDAQ",
      "displayName": "Apple Inc.",
      "assetType": "STOCK",
      "provider": "EASTMONEY",
      "providerQuoteId": "105.AAPL",
      "currency": "USD",
      "quantity": 10.00000000,
      "averagePurchasePrice": 175.50000000,
      "currentPrice": 210.25000000,
      "marketValue": 2102.50000000,
      "costBasis": 1755.00000000,
      "unrealizedGainLoss": 347.50000000,
      "unrealizedGainLossPercent": 19.80056980,
      "allocationPercent": 42.50000000,
      "priceStatus": "LIVE",
      "priceObservedAt": "2026-07-24T08:29:00Z",
      "createdAt": "2026-07-24T08:00:00Z",
      "updatedAt": "2026-07-24T08:00:00Z"
    }
  ],
  "count": 1
}
```

空组合返回 200 和 {"items": [], "count": 0}，不返回 404。

4.2 查询单个持仓

GET /api/v1/holdings/{holdingId}

- 成功：200 OK，响应为 Holding。
- 不存在：404 HOLDING_NOT_FOUND。

4.3 创建持仓

POST /api/v1/holdings

Content-Type: application/json

请求：

```
{
  "assetType": "STOCK",
  "ticker": "AAPL",
  "exchangeCode": "NASDAQ",
  "providerQuoteId": "105.AAPL",
  "quantity": 10.00000000,
  "averagePurchasePrice": 175.50000000
}
```

规则：

- `assetType` 必填，允许 `STOCK`、`ETF`、`MUTUAL_FUND`、`CRYPTO`、`CASH`、`BANK_DEPOSIT`；缺省策略如需保留，只能后端明确按 `STOCK` 处理并返回标准化结果。
- `ticker` 必填，去除首尾空格后转为大写，长度为 1-32。
- `exchangeCode` 可选；缺省为 `UNKNOWN`。若市场数据服务要求交易所，服务端返回字段错误。
- `providerQuoteId` 可选；搜索接口返回该字段时应随创建请求提交，后端仍需重新校验。
- `quantity` 必填且大于 0。
- `averagePurchasePrice` 必填且大于等于 0。
- 同一组合、`assetType`、`ticker` 和 `exchangeCode` 已存在时不自动合并，返回 409，避免未经用户确认地改变平均成本。
- `ETF` 表示场内基金，像股票一样有交易所、`ticker` 和盘中价格。
- `MUTUAL_FUND` 表示场外基金，`ticker` 使用基金代码或自定义代码；MVP 使用手工、演示或缓存净值，不承诺盘中实时价格。
- `CASH` 的 `ticker` 使用币种代码，例如 `USD`；`exchangeCode` 使用 `CASH`；`quantity` 表示现金金额，`averagePurchasePrice` 固定为 1.00000000。
- `BANK_DEPOSIT` 的 `ticker` 使用可读代码，例如 `HSBC_USD` 或 `USD_DEPOSIT`；`exchangeCode` 使用 `BANK`；`quantity` 表示存款本金，`averagePurchasePrice` 固定为 1.00000000。MVP 不自动计算利息或到期收益。

现金请求示例：

```
{
  "assetType": "CASH",
  "ticker": "USD",
  "exchangeCode": "CASH",
  "quantity": 4500.00000000,
  "averagePurchasePrice": 1.00000000
}
```

银行存款请求示例：

```
{
  "assetType": "BANK_DEPOSIT",
  "ticker": "HSBC_USD",
  "exchangeCode": "BANK",
  "displayName": "HSBC USD Deposit",
  "quantity": 3000.00000000,
  "averagePurchasePrice": 1.00000000
}
```

成功响应：201 Created

Location: /api/v1/holdings/101

响应体为创建后的 `Holding`。如果持仓已保存但暂时取不到价格，仍返回 201，其中 `priceStatus` 为 `UNAVAILABLE`。

失败：

- 400 `VALIDATION_FAILED`
- 409 `HOLDING_ALREADY_EXISTS`

4.4 修改持仓数量和平均成本

PATCH /api/v1/holdings/{holdingId}?priceMode=DEMO_ALLOWED
Content-Type: application/json

请求：

```
{
  "quantity": 8.00000000,
  "averagePurchasePrice": 320.00000000
}
```

可选查询参数：

参数	默认值	说明
priceMode	BEST_AVAILABLE	控制响应体估值口径；本地演示价格使用 DEMO_ALLOWED

规则：

- 只修改持仓的 quantity 和 averagePurchasePrice，不改变 assetType、ticker、exchangeCode 或 providerQuoteId。
- quantity 必填且大于 0。
- averagePurchasePrice 必填且大于等于 0。
- CASH 和 BANK_DEPOSIT 始终保持 averagePurchasePrice = 1.00000000，即使请求里提交其他值。
- 成功响应为更新后的 Holding，并按请求的 priceMode 返回 currentPrice、marketValue、priceStatus 等估值字段。

失败：

- 400 VALIDATION_FAILED
- 404 HOLDING_NOT_FOUND

4.5 删除持仓

DELETE /api/v1/holdings/{holdingId}

- 成功：204 No Content，无响应体。
- 不存在：404 HOLDING_NOT_FOUND。

重复删除返回 404，客户端可以据此刷新本地列表。删除持仓不会删除证券主数据或价格历史。

4.6 查询组合摘要

GET /api/v1/portfolio/summary

可选查询参数：

参数	默认值	说明
priceMode	BEST_AVAILABLE	BEST_AVAILABLE、LIVE_ONLY 或 DEMO_ALLOWED

成功响应：200 OK，响应为 PortfolioSummary。

即使部分价格不可用也返回 200 和 valuationStatus = PARTIAL，因为持仓和部分估值仍可使用。只有调用方明确要求 LIVE_ONLY 且实时价格服务整体不可用时，才返回 503 PRICE_SERVICE_UNAVAILABLE。

4.7 搜索证券

GET /api/v1/market/search?query=AAPL

该接口由后端访问外部行情搜索源，例如东方财富搜索建议接口。前端不得直接请求第三方行情服务。

可选查询参数：

参数	默认值	说明
query	必填	用户输入的 ticker 或名称片段
market	空	可选市场过滤，例如 US、SH、SZ、HK

成功响应：200 OK

```
{
  "query": "AAPL",
  "results": [
    {
      "ticker": "AAPL",
      "displayName": "苹果",
      "exchangeCode": "NASDAQ",
      "provider": "EASTMONEY",
      "providerQuoteId": "105.AAPL",
      "assetType": "STOCK"
    }
  ],
  "source": "MIXED",
  "cached": false
}
```

搜索结果只用于辅助新增持仓。最终保存时，后端仍需重新校验 providerQuoteId 是否可报价。

4.8 批量获取报价

GET /api/v1/market/quotes?providerQuoteIds=1.600519,0.000001,105.AAPL&priceMode=BEST_AVAILABLE

可选查询参数：

参数	默认值	说明
providerQuoteIds	必填	第三方行情源的 quote id，多个值逗号分隔
priceMode	BEST_AVAILABLE	BEST_AVAILABLE、LIVE_ONLY 或 DEMO_ALLOWED

成功响应：200 OK

```
{
  "provider": "MIXED",
  "priceMode": "BEST_AVAILABLE",
  "quotes": [
    {
      "provider": "EASTMONEY",
      "providerQuoteId": "105.AAPL",
      "ticker": "AAPL",
      "displayName": "苹果",
      "currency": "USD",
      "currentPrice": 333.02000000,
      "priceStatus": "LIVE",
      "priceObservedAt": "2026-07-25T00:00:00Z"
    }
  ],
  "unavailable": []
}
```

providerQuoteIds 中某个标的不可用时，不应让整个响应失败；该项进入 unavailable，并在 portfolio summary 中体现为 PARTIAL 或 UNAVAILABLE。

4.9 查询基金穿透

GET /api/v1/funds/{instrumentId}/lookthrough

该接口用于解释 ETF 或 MUTUAL_FUND 的底层持仓。基金穿透只用于提示和分析，不改变基金本身的持仓数量、成本或估值。

成功响应：200 OK

```
{
  "fundInstrumentId": 102,
  "ticker": "VOO",
  "displayName": "Vanguard S&P 500 ETF",
  "assetType": "ETF",
  "asOfDate": "2026-06-30",
  "source": "DEMO_DISCLOSURE",
  "coveragePercent": 41.15000000,
  "holdings": [
    {
      "ticker": "AAPL",
      "displayName": "Apple Inc.",
      "assetType": "STOCK",
      "weightPercent": 7.12000000
    }
  ],
  "warnings": [
    "Fund holdings are based on the latest available disclosure and may lag current positions."
  ]
}
```

- 已知演示基金按 ticker 返回披露样例。
- 真实数据库中的基金 instrument id 也可查询；未知基金返回空持仓和 warning。
- 非基金或不存在的 instrument 返回 404 FUND_LOOKTHROUGH_NOT_FOUND。

4.10 查询组合穿透暴露

```
GET /api/v1/portfolio/exposure?lookthrough=true&priceMode=DEMO_ALLOWED
```

可选查询参数：

参数	默认值	说明
lookthrough	false	true 时将基金披露持仓拆到底层资产，并保留未披露残余
priceMode	BEST_AVAILABLE	与 holdings/summary 相同

成功响应：200 OK

```
{
  "portfolioId": 1,
  "portfolioName": "My Portfolio",
  "baseCurrency": "USD",
  "lookthrough": true,
  "priceMode": "DEMO_ALLOWED",
  "totalMarketValue": 2500.00000000,
  "items": [
    {
      "ticker": "AAPL",
      "displayName": "Apple Inc.",
      "assetType": "STOCK",
      "directMarketValue": 1500.00000000,
      "fundLookthroughMarketValue": 100.00000000,
      "totalExposureValue": 1600.00000000,
      "exposurePercent": 64.00000000,
      "sources": ["DIRECT", "FUND:VOO"]
    }
  ],
  "warnings": [
    "AAPL appears both as direct holding and inside fund holdings."
  ]
}
```

前端可用该接口在 Dashboard 或 Analysis 页面提示基金与直持股票重叠；不要在浏览器里自行拆分基金权重。

4.11 健康检查

GET /api/v1/health

成功响应：200 OK

```
{
  "status": "UP",
  "database": "UP",
  "priceProvider": "DEGRADED",
  "timestamp": "2026-07-24T08:30:00Z"
}
```

该接口不得返回数据库连接串、凭据或内部网络地址。

4.12 组合分析

POST /api/v1/portfolio/analysis

请求体：

```
{
  "baseCurrency": "CNY",
  "holdings": [
    { "ticker": "600519", "assetType": "STOCK", "quantity": 100, "marketValue": 180000,
      "costBasis": 150000 }
  ]
}
```

成功响应：200 OK，字段说明：

字段	说明
overview	总市值 + 按资产类型的配置占比
concentration	HHI 集中度指数、最大持仓、Top5 占比、风险等级
fundOverlap	基金重仓股与直接持股的重叠情况
lookThrough	基金穿透后按标的合并的有效敞口及穿透后 HHI
sectorExposure	穿透后按行业汇总的占比及行业 HHI
profitLoss	每笔持仓的浮动盈亏及百分比收益率
llmInsights	AI 基于以上数字生成的解读文本（JSON），仅解读不重新计算
warning	AI 生成失败时的原因（如 LLM_UNAVAILABLE）；此时 llmInsights 为 null，但其余字段仍正常返回

这是兼容手工测试和外部调用的阻塞接口：调用方自行传入 holdings，后端返回规则型事实和 LLM JSON 解读。应用内 Analysis 页面优先使用 4.15 的当前组合合并 SSE 端点，避免前端把市值等权威数据传回后端。

4.13 当前组合分析 — 结构化事实

GET /api/v1/portfolio/analysis/insights

从服务端当前默认组合读取持仓（不接受客户端传入的市值），计算 L0-L4 结构化事实。不调用 LLM，响应快。

查询参数：

参数	默认值	说明
priceMode	BEST_AVAILABLE	与 holdings/summary 相同，控制估值口径

成功响应：200 OK，返回 PortfolioAnalysisFacts JSON 对象：

字段	说明
overview	总市值 + 按资产类型的配置占比
concentration	HHI 集中度指数、最大持仓、Top5 占比、风险等级

字段	说明
fundOverlap	基金重仓股与直接持股的重叠情况
lookThrough	基金穿透后按标的合并的有效敞口及穿透后 HHI
sectorExposure	穿透后按行业汇总的占比及行业 HHI
profitLoss	每笔持仓的浮动盈亏及百分比收益率

4.14 当前组合分析 — AI 流式解读

```
GET /api/v1/portfolio/analysis/insights/stream
Accept: text/event-stream
```

从服务端当前默认组合读取持仓，计算 L0-L4 事实后调用 LLM 生成流式英文解读。每个 SSE data: 事件携带一小段文本。

查询参数：

参数	默认值	说明
priceMode	BEST_AVAILABLE	与 holdings/summary 相同，控制估值口径

- LLM 未配置或调用失败时，降级为单条说明文本后正常结束，不返回 HTTP 错误。
- 前端可用 EventSource 消费。

4.15 当前组合分析 — 合并 SSE（推荐）

```
GET /api/v1/portfolio/analysis/insights/full
Accept: text/event-stream
```

推荐前端使用的端点。合并了 4.13 和 4.14 的功能：先发送结构化事实，再流式发送 AI 解读。避免前端并发调用两个接口导致的重复数据库查询和基金持仓预取。

SSE 事件序列：

顺序	事件名	数据格式	说明
1	facts	{"type":"facts","payload":{...}}	L0-L4 结构化事实（毫秒级到达）
2+	token	{"type":"token","payload":"文本片段"}	LLM 流式输出的英文 Markdown 文本
最后	done	{"type":"done"}	流正常结束

前端消费示例：

```
const es = new EventSource('/api/v1/portfolio/analysis/insights/full?priceMode=BEST_AVAILABLE');

es.addEventListener('facts', (e) => {
  const { payload } = JSON.parse(e.data);
  renderDataCards(payload); // 立即渲染数据卡片
});

es.addEventListener('token', (e) => {
  const { payload } = JSON.parse(e.data);
  appendAiText(payload); // 流式追加 AI 解读
});

es.addEventListener('done', () => es.close());
es.onerror = () => { /* LLM 失败时 facts 已收到，降级展示 */ es.close(); };
```

降级策略：如果 LLM 调用失败，客户端仍会收到 facts 事件，可正常展示数据卡片；仅 AI 解读区域显示降级提示。

5. 计算规则

```
costBasis = quantity × averagePurchasePrice
marketValue = quantity × currentPrice
unrealizedGainLoss = marketValue - costBasis
unrealizedGainLossPercent = unrealizedGainLoss ÷ costBasis × 100
allocationPercent = marketValue ÷ totalPricedMarketValue × 100
```

- `costBasis = 0` 时, `unrealizedGainLossPercent` 返回 `null`。
- 价格不可用的持仓不计入 `totalMarketValue` 和 `allocation` 分母。
- `CASH` 和 `BANK_DEPOSIT` 固定按 `currentPrice = 1.00000000` 计入总市值和 `allocation` 分母。
- `MUTUAL_FUND` 使用最新可用净值或演示净值；如果净值缺失, 按 `UNAVAILABLE` 处理。
- `totalCostBasis` 包括全部持仓成本, 便于用户理解投入；若摘要为部分估值, UI 必须明确提示总盈亏并非完整组合结果。
- 中间计算不提前舍入, API 最多返回 8 位小数, UI 再按币种规则展示。
- MVP 的表现只是当前快照未实现盈亏, 不是时间加权或资金加权收益。

6. 一致性、缓存与失败处理

- 创建、修改和删除持仓成功后, 前端重新获取 `holdings` 和 `summary`, 不在客户端自行推算权威结果。
- 市场价格调用设置短超时, 并通过适配器与核心持仓服务隔离。
- `BEST_AVAILABLE` 顺序建议为：新鲜外部价格、有效缓存、明确允许的演示价格、不可用。
- 演示价格永远返回 `priceStatus = DEMO`, 页面必须可见地标注。
- 外部价格失败不能回滚已经成功保存的持仓。
- 未预期异常写入带 `traceId` 的服务日志, 对外只返回通用错误。

7. 后续扩展接口

以下接口不属于两天编码 MVP, 只有 P0 全部完成后才进入新 Story：

方法与路径	用途
GET /api/v1/portfolios	多组合列表
POST /api/v1/portfolios	创建组合
GET /api/v1/portfolios/{id}/transactions	查询不可变交易流水
POST /api/v1/portfolios/{id}/transactions	记录买入、卖出、股息等交易
GET /api/v1/portfolios/{id}/performance	查询历史估值与表现
GET /api/v1/portfolios/{id}/insights	获取规则型集中度或再平衡提示

API 演进使用 `/api/v1` 保持兼容。新增可选字段属于兼容变更；删除字段、改变字段含义或收紧已有输入规则需要新版本或迁移期。

7.1 基金穿透展示规则

基金本身也是一个投资组合, 可能持有股票、债券、现金或其他基金。HoldHive 不能把基金简单当成普通股票处理后就结束；当用户添加 `ETF` 或 `MUTUAL_FUND` 时, 前端应提示“基金可能包含股票, 可能与已有直持股票形成重叠暴露”。穿透信息不得写进主持仓表, 也不得改变基金本身的估值结果, 它只用于解释和分析。

当前可用接口见 4.9 和 4.10。前端实现时遵守：

- 新增或编辑 `ETF`、`MUTUAL_FUND` 时显示基金重叠风险提示。
- Dashboard 或 Analysis 页面使用 `GET /api/v1/portfolio/exposure?lookthrough=true` 展示穿透暴露, 不在浏览器里自行计算基金权重。
- `sources` 包含 `DIRECT` 和 `FUND:{ticker}` 时, UI 应显示“直接持仓 + 基金内含”的合并含义。

- `warnings` 直接展示为用户友好的提示；不要把基金披露缺失解释为接口失败。
- 穿透数据通常有披露滞后，界面应展示 `source` 或 `warning`，不把它当作实时持仓。

7.2 大模型组合分析接口

1.0.0 已实现当前组合 LLM 解读，推荐路径为：

```
GET /api/v1/portfolio/analysis/insights/full?priceMode=BEST_AVAILABLE
Accept: text/event-stream
```

该接口先发送规则型事实，再发送英文 Markdown insight token。它只读取后端整理后的结构化组合快照、基金穿透摘要和风险提示，不接收前端传入的任意市值，不直接暴露数据库表，也不保存用户密钥。

本地配置：

```
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=deepseek-chat
DEEPSEEK_API_KEY=<local-secret>
```

安全规则：

- API key 只放在 `.env` 或后端运行环境变量，不提交到 Git，也不进入前端。
- 后端向大模型发送最小必要字段，不发送用户姓名、账户号、数据库连接信息或原始异常。
- LLM 只负责解释后端已计算的事实，不能重新计算 HHI、盈亏、穿透比例等核心数字。
- LLM 未配置或失败时，接口仍返回 facts；前端只降级 AI 文本区域。
- 前端必须把大模型结论标记为解释性分析，不能显示成买入、卖出或收益预测。

8. 安全约束

- MVP 没有认证，因此不得公开部署或存储真实个人投资信息。
- 服务端必须限制请求体大小，并拒绝未知或非法字段。
- 所有数据库操作使用参数化查询或 ORM，不拼接用户输入。
- 日志不得记录密钥、完整请求体中的潜在敏感信息或内部异常堆栈到客户端。
- CORS 只允许实际前端来源；不要在公开环境使用无限制*。

9. API 验收与测试矩阵

场景	预期结果
查询空组合	200, 空数组; 摘要状态为 <code>EMPTY</code>
创建合法持仓	201, 包含 <code>Location</code> 和持仓响应
ticker 含小写和空格	标准化后保存为大写
quantity 为 0、负数或缺失	400 <code>VALIDATION_FAILED</code> , 定位 quantity
averagePurchasePrice 为负数	400 <code>VALIDATION_FAILED</code>
重复持仓	409 <code>HOLDING_ALREADY_EXISTS</code>
查询不存在的持仓	404 <code>HOLDING_NOT_FOUND</code>
删除存在的持仓	204, 随后查询返回 404
删除不存在的持仓	404 <code>HOLDING_NOT_FOUND</code>
单个证券价格不可用	200, 该项为 <code>UNAVAILABLE</code> , 摘要为 <code>PARTIAL</code>
所有价格不可用	200, 摘要为 <code>UNAVAILABLE</code> ; <code>LIVE_ONLY</code> 可返回 503
使用演示价格	响应明确返回 <code>DEMO</code> , UI 显示演示标签
成本为 0	盈亏金额可计算, 盈亏率为 <code>null</code>
未预期异常	500 <code>INTERNAL_ERROR</code> , 响应含 <code>traceId</code> 且无内部堆栈

10. 完成标准

- [] 所有 MVP 接口均有 OpenAPI schema、示例和状态码说明。
- [] Controller、Service 和数据访问层的职责清晰。
- [] 接口测试覆盖成功、校验、冲突、不存在和价格服务失败。
- [] 后端总体行覆盖率至少 70%。
- [] API 与数据库文档中的字段、精度和约束一致。
- [] 前端不依赖未文档化字段或错误格式。
- [] 产品负责人和技术评审方已确认 MVP 接口和演示数据方案。